

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 13:15:58
Файлов исходного кода: 111
Суммарный объём кода: 1.96 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommen
          # На Ubuntu политика ImageMagick иногда запре
          # разрешаем то, что нужно пайплайну (text -> p
          for policy in /etc/ImageMagick-6/policy.xml /
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-страниц"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, initial-scale=1" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискретке
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```



```

 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'compiler'
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }}
 >
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
 title="Скачать полный PDF сборник всех стра
 >
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center

```



```

export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }, [code, stdin]);
}
```

```

 <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
 </button>
</div>
</div>

<textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented()) {
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
 placeholder="Напишите Python-код..."

```

```

 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-indigo-600" >
 Ctrl или Cmd + Enter запускает код. В
 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-amber-600" >
 Первый запуск скачивает примерно неск
 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-emerald-600" >
 Код выполняется в браузере и не отпра
 </div>
 </div>
 </div>
 </main>
);
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и
+name = "алгоритмы"
+

```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```
-----
+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...
```

`vite.config.ts`

Полное содержимое после изменений

```
```typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},
},
```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas). При запросе пользователя создать или сгенерировать новую страницу **\*\*строго\*\*** придерживаться готовых шаблонов разметки, стилей и компонентов.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-шаблоны.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 }
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

| Глава                                      | id                    | Комментарий             |
|--------------------------------------------|-----------------------|-------------------------|
| 15. Планарность: K5, K3,3 и формула Эйлера | `planarity-euler`     | т «на пальцах».         |
| 16. Флойд (Флойд–Уоршелл)                  | `floyd`               | Есть матричный алгоритм |
| 17. Алгоритм Джонсона                      | `johnson-algo`        | Перевзвешивание         |
| Обзор: Кратчайшие пути и Графы             | `intro`               | Вводная страница        |
| Эйлер: Путь ≠ Цикл                         | `euler-path-vs-cycle` | Ключевая концепция      |

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

| Глава                                | id                   | Чего не хватает                     |
|--------------------------------------|----------------------|-------------------------------------|
| 3. Декартово дерево (Treap)          | `treap`              | Нет ни визуализации, ни интерактива |
| 8. Графы. Компоненты связности       | `graph-components`   | Нет визуализации                    |
| 24. Классы сложности, сведение задач | `complexity-classes` | Нет визуализации                    |

## Сводная таблица по всем главам

| #   | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|-----|-------|----------|---------|-----|----------|
| --- | ---   | ---      | ---     | --- | ---      |



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.inn
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 /* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}` }
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 /* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className="
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 /* Шапка главы с быстрыми переходами */
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсальн
 </div>
 <h2 className="text-base sm:text-xl font
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
)}
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок накрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрывае
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{k_x} \times 2^{k_y}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; vertical-align: middle;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; vertical-align: middle;">ум на его части не вычислишь. Приходится хранить матрицу.  
<span class="font-mono text-emerald-300">log N × log M</span>. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: сложение матриц – перекрывающиеся блоки посчитают  
</b> – у минимума нет обратной операции. Стрелка

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">Проблема разреженных матриц

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text-white">

<li><b>Сумма через разреженную таблицу – неэффективна (билет 5).</b></li>

<li><b>Память.</b> N = M = 1000 даёт 10<sup>6</sup> элементов. Если считать ячейки, потом пишете код.</li>

<li><b>Один log вместо двух.</b> В запросе  
<span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i][j - 1]</span>  
</li>

<li><b>Выход за границу в запросе.</b> В запросе  
<span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i][j - 1]</span>

<li><b>Нужны обновления.</b> Разреженная матрица (билет 1), а не она.</li>

</ul>

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого из нескольких сотен на сторону), а запросов очень много.

Вторая половина этого сравнения:

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text-sm">
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">С
 х». Спуск от корня: узел, который вместе с
 скрытия – досматривается только одна ветвь
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">М
 аг – одно сравнение: чей у у корня больше,
 сохраняет порядок х). Итого $O(h)$ сравнений
 </div>
 </div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```
<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
```

Код: вставка, merge, erase (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def insert(root, key, pri):
```

```
 if root is None:
```

```
 return Node(key, pri)
```

```
 if key <= root.key: # равные x — левый
```

```
 root.left = insert(root.left, key, pri)
```

```
 else:
```

```
 root.right = insert(root.right, key, pri)
```

```
 return root
```

```
for key, pri in pairs_sorted_by_pri_desc: # порядок э
```

```
 root = insert(root, key, pri)</pre>
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def merge(a, b): # все x в a < все x в b
```

```
 if not a or not b: return a or b
```

```
 if a.pri > b.pri:
```

```
 a.right = merge(a.right, b); return a
```

```
 b.left = merge(a, b.left); return b
```

```
def erase(root, key): # удалить произвольный элемент
```

```
 if key < root.key: root.left = erase(root.left, key)
```

```
 elif key > root.key: root.right = erase(root.right, key)
```

```
 else: root = merge(root.left, root.right)
```

```
 return root</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
 id: "splay-tree",
```

```
 title: "4. Splay-дерево",
```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
x, и под p.</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

 / \ \ x выше / \ \
 x y -----> α p
 / \ \ p выше / \ \
 α β <----- β γ</pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + s
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Тонкая: одна матрица
</div>
 <p class="text-slate-300 text-sm mb
одной матрицы (плюс рамка из нулей). Никаки
прямоугольника от угла. Всё остальное полу
</div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычес
</div>
 <p class="text-slate-300 text-sm mb
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data
платит она за это памятью <span class="fon
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 </tr>
 <tr class="border-b border-slate

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
 - часть демонстрации: лента подсвечивает, из
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Графы. Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">
 1. Планарный граф — граф, который можно нарисовать так, чтобы никакие две его
 ребра не пересекались.
 2. Планарный граф можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Если представить карту мира, то границы стран можно считать ребрами графа.
 Границы не пересекаются в одной точке по-прежнему, поэтому можно раскрасить
 карты, чтобы соседние страны не сливались цветами. Всегда можно всего 4
 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Макро-вершины — это вершины орграфа, которые принадлежат одной и той же компоненте сильной связности.
 <p class="text-xl font-mono text-white">Они инвертированы в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где все улицы имеют одностороннее движение. Если вы можете пройти по улицам в обоих направлениях, то эти улицы образуют цикл. В таком случае, все улицы, входящие в этот цикл, будут принадлежать одной и той же макро-вершине.
 </div>
 <p class="text-slate-300 text-sm mb-4">ВЫХ направлениях по односторонним улицам. Как это выглядит на карте.</p>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </h2>

 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </p>
 </div>
 </h3>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Пример: класс P (простые задачи) vs класс NP (нестрогие задачи).</p>
 <p class="text-base sm:text-lg font-mono">
 P: класс decision-задач, решаемых за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 <p>).

 Сведение (Reduction) A ≤_p B: если A ∈ P, то B ∈ P.
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс NP – нестрогие задачи (например, задачу о наборе чисел, дающих заданную сумму (ответ), он БЫСТРО (за класс P) решить.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
</div>

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл",
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
</div>

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный в
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
 <thead>
```

```
<tr class="text-slate-500 border-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg bor-
```

```
 <div class="absolute -top-3 left-4 bg-s-
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района сое-
 её перекроют (удалят вершину) — районы буд-
 g>хрупкость системы.
```

```
 </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor-
```

```
 <div class="absolute -top-3 left-4 bg-e-
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одно-
 абелем (мостом). Сами свитчи — это точки со
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg bord-
```

```
 <summary class="p-4 font-bold text-slate-40
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
 </summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

 # PEP 709: с 3.12 компрекеншны встроены во фрейм
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: " $O(n)$ ",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",

```





```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок вершин произвольный.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица с односторонним движением)",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целочисленного порядкового номера.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д-во"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева через них. Ожидаемая высота $O(\log n)$).",
 complexity: "ожидаемо $O(\log n)$ ",
}
```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бордер-лист).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция — $O(N)$.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встанет на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 411 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Базовая база
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база – это структура данных, которая
 работает как массив, куча, очередь и стек, бинарное дерево.
 Кодим по минимуму: там, где за нас уже написали.
 </p>
 </div>

 <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за $O(\log n)$
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, и список с <span class="font-mono text-sm">

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-sky-400 mb-4">Очереди
 <p class="text-slate-300 text-sm mb-4">Очередь (BFS) отличается от стека (DFS) тем, что первый вышел) отличаются одной операцией снятия элемента: оба конца работают за $O(1)$. Обычный список сдвигает весь хвост и стоит $O(n)$.</p>
 <pre class="bg-slate-950 border border-slate-700">
```

```
from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
 <p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>
```

</div>

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-amber-400 mb-4">Сортировка
 <p class="text-slate-300 text-sm mb-4">Если массив отсортирован, то можно найти элемент за серединой – понял, в какой половине жить, – и за $O(\log n)$ это 20 сравнений. В Python за это отвечает модуль bisect.</p>
 <pre class="bg-slate-950 border border-slate-700">
```

```
from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньшего 30
 <p class="text-slate-300 text-sm">Без сортировки это будет $O(n)$.</p>
```

</div>

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка
 <p class="text-slate-300 text-sm mb-4">Писать сортировку можно и вручную, но это не очень интересно. В Python есть модуль sorted, который сортирует любой iterable. Это не совсем сортировка, но это очень удобно. В Python есть модуль sorted, который сортирует любой iterable. Это не совсем сортировка, но это очень удобно.
```

```

 left(x) = z # z становится левым р
 высоты пересчитываются только у z и x — ост
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-teal-400 mb-4
 <p class="text-slate-300 text-sm mb-4">Оба обхо
 тару». Вся разница в таре: стек уходит вглуби
 ext-emerald-300">0(V + E): каждая верш
 <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank): # tank: список-стек
 seen, tank = {start}, tank([start])
 while tank:
 v = tank.pop() if isinstance(tank, list) else t
 for to in graph[v]:
 if to not in seen:
 seen.add(to); tank.append(to)
 return seen</pre>
 <p class="text-slate-300 text-sm">Дальше эта па
 надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-slate-300 mb-
 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-indigo-500 shadow-md">
 Аналогия 1: Схема кинозала
 </div>
 <p class="text-slate-300 text-sm">Дерев
 азателей-«usher'ов» не нужно. Место n, дети
 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Дейкстра
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 Аналогия 1: Позитивное планирование

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 (нельзя поехать и заработать). Он всегда в
 <p class="text-xl font-mono text-white">
 <div id="slot-chapter-image-dijkstra">
 </div>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (неожиданное закрытие
 охождение улицы), Дейкстра сломается, так как

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12"
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд

```



```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка — это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрывают
 обратная операция, но применить её к перекрытию
 ммы, билет 5."
},
{
 question: "Какие операции подходят разреженной та
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
],
 correctIndex: 1,
 explanation: "Нужно $op(x, x) = x$: тогда дважды на
 приём и уходит в префиксные суммы."
},
{
 question: "Между запросами элементы матрицы меняю
 options: [
 "Разреженную таблицу: она перестраивается за $O(n^2)$ ",
 "Префиксную матрицу: достаточно поправить одну
 "Дерево отрезков: и таблица, и префиксы статичны",
],
 correctIndex: 2,
 explanation: "Обе структуры статичны: изменение э
 ксной. С обновлениями работает дерево отрезков,
 }
},
"prefix-sums-2d": [
 {
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну

```

```

 ной операции нет. Для min, max и gcd берут разро
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
],
"euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это
 ётной, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная
 options: [
 "Потому что Гамильтон — болезнь, и лечить её т
 "Потому что для эйлерова цикла есть простой кри
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критери
 ого критерия. Это NP-полная задача, которую не
 },
 {

```

```

 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventD
 chapterId ? latestByChapter.get(chapterId) ?? null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDet
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !==
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId]);

```

```

 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
 }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
 typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
 Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь
 * ввёл 5 вершин, визуализация рисует 5x5, а не подставляет его
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list] d = [[0, 4], [999, 0]]
 * list[dict] d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict] d = {0: {0: 0, 1: 4}}
 * dict[(i, j)] d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
 rows: number;
 cols: number;
 /** Значение ячейки; null — таких данных в коде пользователя нет
 at: (i: number, j: number) => number | null;
}

```

```

 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
 };

const asArray = vizArray(value);
if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 return;
 });
 }
 const record = vizRecord(row);
 if (record) {
 for (const [key, cell] of Object.entries(record)) {
 const j = Number(key);
 if (Number.isInteger(j)) put(i, j, cell);
 }
 }
 });
} else {
 const record = vizRecord(value);
 if (!record) return null;
 for (const [key, cell] of Object.entries(record)) {
 const pair = parsePairKey(key);
 if (pair) {
 put(pair[0], pair[1], cell);
 continue;
 }
 const i = Number(key);
 if (!Number.isInteger(i)) continue;
 const inner = vizArray(cell);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```

tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы"},
 { name: "i", role: "индекс элемента массива; его левый и правый дети"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
 { name: "m", role: "середина отрезка $m = (l + r) // 2$ "},
],
},
treap: {
 vizTitle: "Декартово дерево: build → split → merge",
 stepNote: "Шаг = один спуск в split/merge/erase (перезаписывается целиком.",
 code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3]]
n = len(points) # сколько пар (x; y) – столько вершин
mid = 7 # разрез для split: $x \leq mid < y$
key = 9 # какую точку удаляем в erase
step = 0 # номер шага – листает кадры дерева
height = 0 # высота построенного дерева
root = None

--- тело алгоритма ---
class Node:
 def __init__(s, x, y):
 s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
 """Декартово дерево из отсортированных по x точек:
 pts = sorted(pts, key=lambda p: p[0])
 stack = []
 for x, y in pts:
 cur = Node(x, y)
 last = None
```



```

while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # x и p -
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root))
 return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
 { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\", \"zig\" },
 { name: "step", role: "номер применения случая – 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000 },
 { name: "xk / x", role: "ключ узла, который поднимается", value: 40 },
 { name: "pk / p", role: "ключ родителя x перед поворотом", value: 20 },
 { name: "gk / g", role: "ключ деда – нужен в Zig-Zag", value: 60 },
 { name: "root", role: "корень дерева после расшейки", value: 40 }
],

```

```

 stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] # $P[0] = 0$ – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)` ,
 variables: [
 { name: "a", role: "исходный массив 1D-демонстрация" },
 { name: "n", role: "длина массива a; префиксов будет n" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0] + \dots + a[i-1]$ " },
 { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = событие внутри fibМемо(k): вызов,
code: `n = 5`
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
 variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного вызова" },
 { name: "memo", role: "кэш уже посчитанных значений" }
]
}

```

```

code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущ
q.append(u) # enqueue соседа — в конец
front = q[0] # голова очереди: её обрабо
v = q.popleft() # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
 variables: [
 { name: "front", role: "голова очереди — вершина,
 { name: "v", role: "вершина, извлечённая из очере
 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```

v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то ребру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "n", role: "число вершин; в демонстрации задано 10" },
 { name: "adj", role: "списки смежности — тот же граф, что в примере" },
 { name: "comp", role: "разметка: номер компоненты, в которой находится вершина", range: "-1 ... comps" },
 { name: "used", role: "булев вариант разметки — для BFS не нужен" },
 { name: "comps", role: "счётчик найденных компонент" },
 { name: "c", role: "номер компоненты, которую крашим сейчас" },
 { name: "v", role: "вершина, где обход находится" },
 { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
 { name: "i / step", role: "номер шага — перематываем кадр" },
],
},
"top-sort": {
```

```
def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины — она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)
print("обратный граф:", adjT)

фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
```



```
},
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверяе
 { name: "tin[v]", role: "время входа в вершину – m
 { name: "low[v]", role: "минимум tin, куда можно ,
 { name: "timer", role: "глобальный счётчик времени
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров маршрут: алгоритм Иера",
 stepNote: "Шаг = переход по неиспользованному ребру
 идти нельзя.",
 code: `n = 5 # вершин: 0...4 (на
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)} # степени ве
odd = [v for v in range(n) if deg[v] % 2 == 1] # нечёт
start = odd[0] if odd else 0 # путь начинае
used = [False] * len(edges) # какие рёбра :
path = [start] # маршрут: пос.
last = start # где мы сейча
stack = [start] # стек для отк
v = start
step = 0
```

```

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежн
F = 2 - V + E # граней: из $V - E + F = 2$

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| $V - E + F = 2$ ")
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "n", role: "число вершин графа на демонстрации" },
 { name: "adj", role: "списки смежности — тот же граф, что и n" },
 { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
 { name: "v", role: "вершина, которую красим сейчас" },
 { name: "u", role: "сосед вершины v, чей цвет уже известен" },
 { name: "delta", role: "максимальная степень $\Delta = \max \deg(v)$ " },
 { name: "chi", role: "сколько цветов реально понадобится" },
 { name: "i / step", role: "номер шага — перематываем видео" },
 { name: "V, E, F", role: "для вкладки «Планарность графа»" },
 { name: "9", role: "номер шага — перематываем видео" },
],
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

```

## Универсальное пособие • полный исходный код

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База
```

```
--- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":
 print(f"итерация {i}: изменений нет – сошлось дельта")
 break
 print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
 if dist[u] != INF and dist[u] + w < dist[v]:
 cycle_edge = (u, v, w)
 break
```

```
if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
 print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
 variables: [
 { name: "n", role: "число вершин графа – ровно n-1"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
 { name: "dist[v]", role: "расстояние; если обновилось, то новое"},
],
```

```

 на-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивает",
 { name: "d[v]", role: "расстояние Дейкстры в новом графе"
 },
],
},
mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "E"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "B", "G"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "n", role: "длина строки s", range: "n = len(s)" },
 { name: "i", role: "позиция, для которой считаем z[i]" },
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
 { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:z[i]]" }
],
},
"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
 stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень автомата. Если найдено, то переходят с картинкой «водопада».",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую переходим, если в автомате нет ребра с вершиной v`

```

```

 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0: # собираем все образцы
 if v in term:
 matches.append((i, term[v]))
 v = fail[v]
 print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
 { name: "words", role: "набор образцов – по нему",
 },
 { name: "text", role: "сканируемый текст – подста",
 { name: "state", role: "текущее состояние автомата",
 { name: "i / c", role: "индекс и сам символ текста",
 { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
 { name: "fail", role: "fail[v] – вершина наибольшей",
 { name: "term", role: "в каких вершинах заканчивае",
 { name: "queue", role: "очередь BFS, которой натя",
 { name: "matches", role: "найденные вхождения: (ин",
 { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа

```

```

 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },

 /* — Дополнительные демонстрации внутри страниц (вкл.)

 "planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет",
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
 variables: [
 { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n" },
 { name: "v", role: "текущая вершина", range: "0 ... n-1" },
 { name: "delta / chi", role: "максимальная степень / количество цветов", range: "0 ... n" },
 { name: "i / step", role: "номер кадра", range: "0 ... n-1" },
]
 }
}
 `

```

```

],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:
 c += 1
 ecolor[e] = c
 inc[a][e] = c
 inc[b][e] = c
 chi_e = max(chi_e, c)
 print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
 { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
 { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
 { name: "v, u", role: "концы текущего ребра – подмножества вершин", range: "0..n-1" },
 { name: "inc", role: "для каждой вершины: какие цвета рёбер к ней примыкают", range: "0..delta" },
 { name: "chi_e / delta", role: "χ' – сколько цветов нужно", range: "0..delta+1" }
],
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",

```



```

 },
 "graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает",
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева – по ней реш
comps = n # сначала каждая вершина – о
e = -1 # номер обрабатываемого ребра
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага – двигает по ка

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # ребро внутри
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)

```

```

k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
 { name: "step", role: "номер кадра сведения (0..6)" },
 { name: "cls", role: "класс задачи A, которую сводят к КЛИКА" },
 { name: "n", role: "размер входа x; после сведения к КЛИКА" },
 { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
 code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0] # сертификат: x1=1, x2=0, x3=1
k = 0 # номер проверяемого дизъюнкта
ok = True # вердикт верификатора

--- тело алгоритма ---
for k, cl in enumerate(clauses):
 hit = False
 for l in cl:
 val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
 if val == 1:
 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение переменных" },
],
},

```

```

edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представителем компонента
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе = 51)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "i", role: "номер текущего ребра – перебираем"},
 { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
 { name: "parent", role: "DSU-массив: цвета компонентов"},
 { name: "mst", role: "рёбра, уже взятые в остов", "array"},
 { name: "total", role: "вес остова (51 на этом графе)"},
],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E)",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:
 phase += 1
 cheapest = {}
 for (w, a, b) in edges: # КАЖДАЯ компонента ОДНОВЕРХОВА
 ra, rb = find(a), find(b)
 if ra == rb:
 continue
 u, v = a, b
 if ra not in cheapest or w < cheapest[ra][0]:
 cheapest[ra] = (w, a, b)
 if rb not in cheapest or w < cheapest[rb][0]:
 cheapest[rb] = (w, a, b)
 if not cheapest:
 break # граф несвязен
 for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты

```

```

 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[{r,c,k,k}]", role: "минимум квадратн
 },
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

=====

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

-----

ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.2 КБ

-----

```
import { Chapter } from '../../types';
```

```

pan class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
 — это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ — любая операция, котор
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается — возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента — спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

 </div>
 <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]) и
свой баланс. Это и есть отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
за $O(1)$, но строятся один раз и умирают при
ми.</p>
```

```
 <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
 <p class="text-slate-400 text-sm mb-1">
n ячеек (не $2n!$) – иначе на $n = 6$ поймать п
n ... $2n-1$] и n – степень двойки (или использ
 <p class="text-slate-400 text-sm">•
ть лень во всё дерево – это $O(n)$ и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

```
Код: построение и запрос суммы (C
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

 y[4 * MAXN]; // «обещание»: прибавит

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

void add(int v, int l, int r, int L, int R, int x) { /
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 return;
 }
 push(v, l, r); // о
 int m = (l + r) / 2;
 add(2 * v, l, m, L, R, x);
 add(2 * v + 1, m + 1, r, L, R, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounde  
 # для удобств

```
</section>
```

```
`,
 },
];
```

---

ФАЙЛ 27/111: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТР0К: 1212 | РАЗМ:

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
 {
```

```
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
ершины
требование к графу – неотрицательные вес
```

```


Ограничения: минус ломает жадно

s→a = 2
b→a = -2. Верн
начала вытащит 3 - 2 = 1 придёт
Джонсона (билет 17).


```

```


Демонстрация

Демонстрация (D, Дом 2 Е, ТЦ F, Метро G) и 11 односторонних
ршины (u → v): переменные u
, v ают картинку напрямую.

- Инициализация. ∞; в кучу кладём prev пуст.
- Извлечь минимум. Достаём пару du. Если done – пара устарела.
- Зафиксировать. dist[v] меняется.
- Релаксировать рёбра. Для каж
u: если dist[v]
{d


```

```

<table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border-b"
 <th class="py-1.5 pr-2 font-normal">Исходный код
 <th class="py-1.5 pr-2 font-normal">Описание
 <th class="py-1.5 font-bold">Пример
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200"
 y-2 pr-2">лучший известный путь $s \rightarrow v$ </td><td>
 <tr class="border-b border-slate-200"
 y-2 pr-2">предок v на текущем лучшем пути</td><td>
 <tr class="border-b border-slate-200"
 lass="py-2 pr-2">вершины с окончательным dist
 </td></tr>
 <tr class="align-top"><td class="py-2">Исходный код
 ие, вершина)</td><td class="py-2">извлечение
 </td></tr>
 </tbody>
</table>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">Пример
 </h3>
 <p class="text-slate-300 text-sm mb-4">Исходный код
 извлекается из кучи, s → u. Отсюда
 dist[v] = δ(s, v) для всех достижимых вершин.
 dist[v] = ∞ для недостижимых вершин.
 </p>
 <details class="bg-slate-900/40 rounded-lg border"
 <summary class="p-4 font-bold text-slate-300">Доказательство инварианта
 <b border-slate-700/50">Доказательство инварианта
 <div class="p-5 text-sm text-slate-300"
 <p>Пусть dist[v] = δ(s, v) для
 dist[v] = δ(s, v) для

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-400">
 <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе

 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left border-collapse">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-normal">Гр.
 <th class="py-2 pr-3 font-normal">Гр.
 <th class="py-2 font-bold">Гр.
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-3 font-normal">priority_queue (C++)</td><td class="py-2 pr-3 font-normal">приоритетный выбор</td></tr>
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-3 font-normal">ease-key</td><td class="py-2 pr-3 font-normal">очередь</td></tr>
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-3 font-normal">поиск минимума</td><td class="py-2 pr-3 font-normal"></td></tr>
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-3 font-normal"></td><td class="py-2 pr-3 font-normal"></td></tr>
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-3 font-normal"></td><td class="py-2 pr-3 font-normal"></td></tr>
 </tbody>
 </table>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-2 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">Код: Дейкстра на графе
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
 INF = float("inf")

```

```

dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">» Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>;,
 vector<pair<long long,int>;>,
 greater<pair<long long,int>;>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {
 if (du + w < d[v]) {
 d[v] = du + w;
 p[v] = u;
 pq.push({d[v], v});

```

```

 <p class="text-slate-400 text-xs">Т
 . Память – матрица
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-amber-400

 <div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Отрицательные веса. Алгор
 ксация вершины необратима. Нужен Форд–Беллм
 Отрицательные циклы. При
 ечное число. Сначала проверьте граф Фордом–
 Ленивое удаление. Без пр
 одна вершина обрабатывается несколько раз:
 Переполнение INF. <span c
 уходит в отрицательное число и «улу
 "font-mono">u.
 Ориентированность. Демо-
 ом графе каждое ребро добавляется в оба спи
 Дейкстра ≠ BFS. BFS – ча
 С разными весами FIFO ломается, нужна куча
 «Одна пара». Алгоритм сч
 ий выход при извлечении цели (билет 17), но

 </div>

 <p class="text-slate-300 text-sm mt-4 mb-4"
 ов ценой скорости <span class="font-mono te
 font-mono text-emerald-300"> $O(V^3)$; б
 после перевзвешивания потенциалов). Предста
 </div>
</div>
</section>`
},

```

```

 <p class="font-mono text-emerald-300">
 </div>
 <div class="bg-slate-900 border border-rose-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-emerald-300">
 </div>
 <div class="bg-slate-900 border border-rose-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-emerald-300">
 </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Патруль Бэтмена на улицах Готэма
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ночей подряд, объезжая абсолютно все улицы
 ровно 1 выходной в году, когда он отде-
 -лю ночь Бэтмен делает контрольный проезд: е-
 о-то района снова уменьшилось – значит, Джо
 </div>

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Отрицательный цикл: временная паника
 </div>
 <p class="text-slate-300 text-sm mb-4">
 меньше:
 и ответ « $-\infty$ ». Контрольная итерация – это де-
 .</p>
 </div>
 </div>
</div>
</div>

```



```

 <p class="text-slate-300 text-sm mb-3">
тава A имеет суммарный вес <span cla
веет» на 2.</p>
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
 <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
 <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
 <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
 <tr class="align-top"><td class:
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательн
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
остижимые из цикла вершины (A, C, D, E, F).
х, до которых из него есть путь.</p>
 </div>

```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
```

```

 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
 <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
 </tr>

 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает {d[i][i] < 0}<
 <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
 <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
 </tbody>
</table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-
<h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900/60 p-4 rounded-lg l
 <p class="text-xs font-bold text-amber-:
 :</p>
<pre class="bg-slate-950 p-3 rounded text-xs font-mono
 d = [float("inf")] * n
 d[start] = 0
 # Бэтмен патрулирует n - 1 ночей
 for _ in range(n - 1):
 for u, v, w in edges:
 if d[u] + w < d[v]:
 d[v] = d[u] + w

```

```

 cycle = [y]
 v = prev[y]
 while v != y:
 cycle.append(v)
 v = prev[v]
 cycle.append(y)
 cycle.reverse()
 return dist, prev, cycle

демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D", "A", -10]])
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None —

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D", "A", -10]])
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # всё

<p class="text-slate-400 text-xs">П
имая вершина с INF<
ля float('inf')</sp
</div>
</details>

<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (структур
<div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
t;
#include <iostream>
using namespace std;

```

```
int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout && "отрицательных циклов не
 else {
 cout && "цикл: ";
 for (int v : negative_cycle(x)) cout && v
 cout && "\n";
 }
}
</code></pre>
```

```
</div>
</details>
```

```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Оптимизации: ранни
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 rt deque
```

```
def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые»
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True
 while q:
 u = q.popleft()
 in_queue[u] = False
 for v, w in adj[u]:
 if dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 if not in_queue[v]:
 q.append(v)
```

```

 . Нужна проверка всего графа – добав
 а, билет 17) и запустите от неё.
 «Ответ $-\infty$ ». Если отрицат
 s="font-mono">dist[v] = $-\infty$: печатать
 ходом в обратном графе.
 SPFA как «быстрый Форд–Беллм
 в условии V, E ≤ 1

 </div>

 <p class="text-slate-300 text-sm mt-4">С
 отенциалы <span class="font-mono text-emera
 я Дейкстрой. Флойд–Уоршелл (16) решает все
 порядок для DAG – билет 9.</p>
 </div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-indigo-300 italic
 <p class="text-xl font-mono text-white"
 </div>

```

```

 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 </div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в
 </div>
 <p class="text-slate-300 text-sm mb-4">
 це только соседние станции. Затем говорите:
 через 1 там, где
 плакат окончательный. Порядок «разрешений»
 an>.</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-rose-500 shadow-md">
 Аналогия 2: Почему k обязан быть
 </div>
 <p class="text-slate-300 text-sm mb-4">
 s="font-mono">k один раз и бо
 садку не соберётся. На демо-графе этой стра
 ono text-emerald-300">d[2][1] = ∞, х
 пользуется «поздние» посредники). Проверка пер
 </div>
 </div>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <p class="text-slate-300 text-sm mb-4">Демо
 <p class="text-slate-300 text-sm mb-4">вершины 0..6, <span class="font-mono text-em

```

```

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 font-bold"
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">конечное</td><td class=
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">INF</td><td class="py-2"
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">0</td><td class="py-2">
 <tr class="align-top"><td class:
se-300">< 0</td><td class="py-2 text-rose-400">
делены</td></tr>
 </tbody>
 </table>
</div>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400"

 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">> Формальная динамика
 <div class="p-5 text-sm text-slate-300"
 <p>Обозначим <span class="font-mono"
o">i → j, все промежуточные вершины
font-mono">i и <span class="font-mono"
 <p>База: <span class="font-mono"
ли 0.</p>
 <p>Переход: путь из <span cl

```

```

 d[i][j] = d[i][k] + d[k][j]

 return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good[i][j] != bad[i][j]))
4 клетки отличаются; при порядке (i,k,j) их 6</code><
 <p class="text-slate-400 text-xs">В
 угих «почти работают» (большинство клеток с
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Алгоритм Флойда-Уоршелла
 </h3>
 <details class="bg-slate-900/40 rounded-lg border-1 border-slate-800">
 <summary class="p-4 font-bold text-slate-200">Алгоритм Флойда-Уоршелла
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-200">
def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ обнулена

 for u, v, w in edges:
 if w < d[u][v]: # кратчайший путь
 d[u][v] = w
 nxt[u][v] = v
 if not directed and w < d[v][u]:
 d[v][u] = w
 nxt[v][u] = u

 for k in range(n): # k – ВНЕШНИЙ

```



```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-white">
 t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
 for (int i = 0; i < n; ++i) d[i][i] = 0;
 for (int k = 0; k < n; ++k) // k
 for (int i = 0; i < n; ++i) {
 if (d[i][k] == INF) continue; // защищаем от переполнения
 for (int j = 0; j < n; ++j) {
 if (d[k][j] == INF) continue;
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
 }
 }
}

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return false;
}

long long diameter() { // самая длинная дистанция
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (d[i][j] < INF) res = max(res, d[i][j]);
 return res;
}

int center() { // вершина с минимальной суммой расстояний
 int best = -1; long long bestEcc = INF;
 for (int i = 0; i < n; ++i) {
 long long ecc = 0;
 for (int j = 0; j < n; ++j)
 if (d[i][j] < INF) ecc += d[i][j];
 if (ecc < bestEcc) best = i, bestEcc = ecc;
 }
 return best;
}
```

```

 <p class="text-slate-400 text-xs mt-4">
 > при r[i][k]
 V^3).</p>
 </div>
 </details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-800">
 <h3 class="text-xl font-bold text-amber-400">

```

```

 <div class="bg-rose-950/30 p-5 rounded-lg border-1 border-rose-900">
 <p class="text-sm font-bold text-rose-300">
 <ul class="list-disc pl-5 space-y-2 text-rose-400">
 Порядок циклов. (i, j, k) на демо-графе 4 клетки
] = 999 вместо 4).
 Диагональ. d[i][j]
 «починит» её только при наличии цикла, а по
 Переполнение INF. long long
 уходит в минус и «улучшает» всё подряд. Пр
 no">long long.
 Параллельные рёбра. Во в
 no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Формулы
 ых с циклом, ответ d[i][j] = -∞, если существуе
 d[i][k] < ∞
 Недостижимость ≠ 0. В ор
 таких пар из 49). Выводить 999 или 0 как «р
 Память. nx
 ачений (≈100 МБ на nx
 истока берите Дейкстру.
 Сам путь. Матрица nxt[i][j] (как

 </div>
 </div>

```

```

 <p class="text-slate-300 text-sm mt-4">C

```

```
<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

Аналогия 1: Такси с навигатором

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
 у одинаково. А* — тот же таксист, но с навигатором.
 /span> прибавляется оценка остатка пути по карте. Если идти
 т туда, где сумма меньше. Навигатор обязан подсказывать
 /span>): занижать можно, завышать нельзя — это не таксист, а
 0">h = 0 это снова Дейкстра.</p>
```

```
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg">
```

```
<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

Аналогия 2: Две бригады туннеля

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
 пускают две: от А и от В. Каждая зона роста имеет свой
 я не в момент встречи, а когда сумма стоимостей
 — иначе можно пропустить более дешёвый объект.
 ирует.</p>
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left">
```

```
<thead>
```

```
<tr class="text-slate-500 border-b">
```

```
<th class="py-2 pr-3 font-bolder">А
```

```
<th class="py-2 pr-3 font-bolder">В
```

```
<th class="py-2 pr-3 font-bolder">С
```

```
<th class="py-2 font-bolder">Дейкстра
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-700">
)</td><td class="py-2 pr-3">разрешает Дейкстра (V)
 g V)</td><td class="py-2">нет отрицательных
```

```
<p class="text-slate-300 text-sm mb-3">
 Дейкстра (билет 14) быстрая, но не универсаль-
 ленный. Алгоритм Джонсона делает финт: пере-
 ь теми же.
```

```
</p>
<ol class="text-sm text-slate-300 space-y-2">
 Добавляем фиктивную вершину s .
 Запускаем от s Форда-Беллмана. Ва-
 жный факт:</i> Джонсон НЕ «удаляет» отрица-
 кой и прекращает работу, так как при отрица-
 Перевзвешиваем: $h(u, v) = d(u, v) + h(u)$
 тво следует из того, что $h(v) \leq h(u) + w(u, v)$.
 Магия: для любого пути P из s в t
) – $h(\text{финиш})$, то есть все пути между фиктивной вершиной и t имеют одинаковую длину.
```

```
 Запускаем быструю Дейкстру из t .

d(u, v) = d(u, s) + d(s, t) + d(t, v)

```

```
<p class="text-slate-400 text-xs mt-3">
 В разреженном графе ($E \approx V$) это $O(V^2 \log V)$ против $O(V^3)$ для Флойда-Уоршелла.
 ойд – для маленьких или очень плотных.</p>
```

```
</div>
```

```
</div>
```

```
<h3 class="text-lg font-bold text-white mt-3">Алгоритм Джонсона</h3>
<p class="text-slate-300 text-sm mb-3">Если $g(t)$ – оценка стоимости пути из t (по перевёрнутым рёбрам). Волны р-
: в «широком» графе (дорожная сеть) это эквивалентно тому, что
огда волны «встретились» в одной вершине; но в «узком» графе это
огда $g(u, v) = g(u) + d(u, v)$
более короткий путь.</p>
```

```
<h3 class="text-lg font-bold text-white mt-3">Алгоритм Джонсона</h3>
<p class="text-slate-300 text-sm mb-3">Дейкстра (билет 14) быстрая, но не универсаль-
 по $g(v)$ (уже известная оценка стоимости пути из s в v).
 $h(v) = g(v) + h(v)$, где $h(v)$ – оценка стоимости пути из v в t .
 а требования: допустимость ($h(v) \leq \text{стоимость}(v, t)$) и монотонность/согласованность ($h(u) \leq w(u, v) + h(v)$).
```

</div>

<p class="text-slate-300 text-sm mt-4"><b c  
py-0.5 rounded text-xs font-mono text-emera  
class="bg-slate-950 px-2 py-0.5 rounded tex  
м числом раскрытий, 0-1 BFS <code class="bg  
te-800">0(V + E)</code>, Dial <code class="la  
late-800">0(E + V·C)</code>, Дейкстра с куч  
emerald-300 border border-slate-800">0(E + V  
o text-rose-300 border border-slate-800">0('

<details class="bg-slate-900/40 rounded-lg l  
    <summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Код: Джонсон целиком (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300  
    <pre class="bg-slate-950 p-4 rounded

heapq

INF = float("inf")

```
def johnson(n, edges):
 # edges: (u, v, w)
 # 1) потенциалы: Форд–Беллман от фиктивной вершины
 h = [0] * (n + 1)
 # из s все рёбра
 for _ in range(n):
 # V итераций: V-1
 changed = False
 for u, v, w in edges:
 if h[u] + w < h[v]:
 h[v] = h[u] + w
 changed = True
 if not changed:
 break
 else:
 return None
 # отрицательный цикл
```

# 2) перевзвешивание: все  $w' \geq 0$

g = [[] for \_ in range(n)]

for u, v, w in edges:

```

 dist, v = heapq.heappop(qf)
 if dist > df.get(v, INF): continue
 for to, w in g[v]:
 nd = dist + w
 if nd < df.get(to, INF):
 df[to] = nd; heapq.heappush(qf, (nd, to))
 if to in db:
 mu = min(mu, nd + db[to]) # встреча:
 dist, v = heapq.heappop(qb)
 if dist > db.get(v, INF): continue
 for to, w in rg[v]:
 nd = dist + w
 if nd < db.get(to, INF):
 db[to] = nd; heapq.heappush(qb, (nd, to))
 if to in df:
 mu = min(mu, nd + df[to])
 return mu

--- A*: приоритет f = g + h ---
def astar(g, s, t, h): # h(v) – допустимая оценка
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход: g
 if v in closed:
 continue
 closed.add(v)
 for to, w in g[v]:
 nd = gv + w
 if nd < gdist.get(to, INF):
 gdist[to] = nd
 heapq.heappush(heap, (nd + h(to), nd, to))
 return INF

```

```
</section>
```

```
 },
```

```
];
```

```
ФАЙЛ 28/111: src/data/tickets/ticket18_20_mst.ts
```

```
КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 224 | РАЗМ
```

```
import { Chapter } from '../..types';
```

```
/**
```

```
 * Билеты 18 · 19 · 20 — одна страница на три алгоритма
```

```
 * минимального остоного дерева (MST). Так же, как в с
```

```
 * KruskalSimulator, три алгоритма живут на одном графе
```

```
 * переключаются вкладками «Краскал / Прима / Борувка».
```

```
 *
```

```
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + под
```

```
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
```

```
 * через emitVizDemo, код в панели следует за ней).
```

```
 */
```

```
export const tickets18to20Mst: Chapter[] = [
```

```
 {
```

```
 id: "mst",
```

```
 title: "18–20. Графы. Остовное дерево: Краскал, При
```

```
 type: "html",
```

```
 description:
```

```
 "Три способа построить минимальное остовное дерев
```

```
 .",
```

```
 category: "Графы. Остовные",
```

```
 content: `
```

```
<section id="mst" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
 <span class="bg-blue-600 text-white px-4 py-1 r
```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-r
```

```
 </div>
```

```
 |
```

```

<div class="grid grid-cols-1 xl:grid-cols-3
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
der border-indigo-500 shadow-md">

```

Аналогия к билету 18 • Краск

```

</div>
<p class="text-slate-300 text-sm">С
». По всей карте растут независимые кусочки
дерево. Порядок рёбер важен, порядок верши
</div>

```

```

<div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">

```

★ Аналогия к билету 19 • Прим:

```

</div>
<p class="text-slate-300 text-sm">П
а захватывает ближайший непосещённый город.
ва» против «снаружи».</p>
</div>

```



```
<summary class="p-4 font-bold text-slate-300 border-slate-700/50">
```

Код 18: Краскал + DSU (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300"
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 lt;pair<int, pair<int,int>>>>
vector<int> p, rnk;
```

```
int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
bool unite(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false; // уже в одной компоненте
 if (rnk[a] < rnk[b]) swap(a, b);
 p[b] = a;
 if (rnk[a] == rnk[b]) rnk[a]++;
 return true;
}
```

```
sort(edges.begin(), edges.end()); // O(E log E)
long long cost = 0; int taken = 0;
for (auto [w, uv] : edges) {
 if (unite(uv.first, uv.second)) { // не замыкает цикл
 cost += w;
 if (++taken == n - 1) break; // V - 1 ребро
 }
}
}</pre>
```

```
</div>
</details>
```

```
<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
```

Код 19: Прим с кучей (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300"
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 heapq
```

```
comps = n
total = 0
while comps > 1:
 best = {} # компонента
 for w, u, v in edges:
 cu, cv = find(u), find(v)
 if cu == cv:
 continue # уже внутри
 if cu not in best or w < best[cu][0]:
 best[cu] = (w, u, v)
 if cv not in best or w < best[cv][0]:
 best[cv] = (w, u, v)
 if not best:
 break # граф несвя
 for w, u, v in best.values():
 if find(u) == find(v):
 continue # дубль: два
 parent[find(u)] = find(v)
 comps -= 1
 total += w
print(total)</pre>
```

<p class="text-slate-400 text-xs">3.  
ну), значит фаз не больше  $\log_2 V$ .</p>

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Вопросы, которые любят задавать (C  
</summary>

<div class="p-5 text-sm text-slate-300  
<p>• Почему жадность здесь работает  
меном ребра в предполагаемом MST.</p>

<p>• Чем Прим отличается от Дейкстры  
ного веса против дерева кратчайших путей).</p>

<p>• Какой алгоритм лучше на плотном

```
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
сом, оканчивающимся в позиции i.</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb
 Мы идём по строке слева направо. К
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
x</code>. Тупой алгоритм начнет искать занос
cab</code> суффикс <code>ab</code> – это ро
буквы шаблона». Это поиск за $O(N)$ без возвр
 </p>
 </div>
```

```
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
```

```

<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 p>
</div>
<p class="text-slate-300 text-sm mb-4">Префикс
 для шаблона (pattern) один раз, затем идём
 пало — $j += 1$. Не совпало — откатываемся на
 у заново сравнивать его не нужно. Когда j д
 ждения), делаем $j = \pi[j - 1]$.</p>
<p class="text-slate-300 text-sm mb-4">П
 ждый откат j уменьшает. Значит суммарно отк
 классическая амортизационная оценка, её люб
 <div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 льшего собственного префикса, равного суффи
 <p class="text-slate-400 text-sm mb-1">
 ый алгоритм.</p>
 <p class="text-slate-400 text-sm mb-1">
 т онлайн (символы могут приходить потоком),
 <p class="text-slate-400 text-sm">• P
 5] = 4, период 6 - 4 = 2 («ab»)</p>
 </div>
 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: поиск подстроки КМР (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
fix_function(s):
n = len(s)
pi = [0] * n
for i in range(1, n):
 j = pi[i - 1]
 while j > 0 and s[i] != s[j]:

```

```

type: "html",
category: "Строки",
content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «приклеиваю
 я начну читать строку отсюда, насколько длинная
 Это самый быстрый способ найти индекс
 >, считаешь Z-функцию, и там, где <code>Z[i]</code>
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>

```

```

 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
 }</pre>

```

```

 </div>

```

```

 </details>

```

```

 </div>

```

```

</div>

```

```

</section>`

```

```

 },

```

```

 {

```

```

 id: "aho-corasick",

```

```

 title: "23. Алгоритм Ахо-Корасик",

```

```

 type: "html",

```

```

 category: "Строки",

```

```

 content: `

```

```

<section id="aho-corasick" class="mb-12 scroll-mt-10">

```

```

 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```



```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

 </div>

```

```

<div class="space-y-8">

```

```

 <div class="bg-slate-700/50 p-6 rounded-xl border-l">

```

```

 <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">

```

```

 <p class="text-sm text-blue-300 italic mb-2">Осн

```

```

 <p class="text-lg font-mono text-white">Бор (Тр

```

```

 </div>

```

```

 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>

```

```

 </div>

```

```

<!-- Аналогии -->

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```

 <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">

```

```

 Аналогия 1: Паутина (Бор)

```

```

 </details>
 </div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description:
 "P, NP, co-NP, NP-трудные и NP-полные, PSPACE и E. Если ли задача NP-полна.",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 ат). Сведение A ≤p B – преобразовать вход A
 .</p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Клас
 машина Тьюринга) с выбранным бюджетом ресур
 про худший случай, а не про «в средн

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P против P
 </div>

```

```

 py-2 pr-3">детерминированно за $O(n^k)$ </td><td class="py-2 pr-3">удольности, максимальный поток</td></tr>
 <tr class="border-b border-slate-300">
 py-2 pr-3">сертификат проверяется за полиномиальную (не решающая версия), коммивояжёр «есть ли тип $\leq$
 <tr class="border-b border-slate-300">
 s="py-2 pr-3">сертификат есть у ответа «НЕТ» (не решающая версия)</td></tr>
 <tr class="border-b border-slate-300">
 lass="py-2 pr-3">не проще любой задачи из NP-полной (не решающая версия)</td></tr>
 <tr class="border-b border-slate-300">
 ass="py-2 pr-3">NP-трудные И из NP</td><td class="py-2 pr-3">сумм</td></tr>
 <tr class="border-b border-slate-300">
 "py-2 pr-3">полиномиальная память, время не зависит от размера входа</td></tr>
 <tr class="align-top"><td class="py-2 pr-3">за $2^{(n^k)}$ </td><td class="py-2">обобщённые NP-полные</td></tr>
 </tbody>
</table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">3.2. NP-полнота
<p class="text-slate-300 text-sm mb-3">Стандартная задача NP-полноты — это задача, для которой существует сертификат полиномиального размера и проверка сертификата полиномиальна.
 — «все вершины по разу и каждая соседняя пара вершин образует треугольник» — это полиномиальное сведение $A \leq_p B$. Тогда B — NP-полна.
<p class="text-slate-300 text-sm mb-3">Теорема 3.2.1. Задача SAT NP-полна. Доказывается «в лоб»: вычислительная сложность формулы размера $\text{poly}(n)$, где переменные — булевы, полиномиальна принятию входа. Дальше всё катится цепочкой:
 гамильтонов цикл $\rightarrow$ TSP.</p>

```

```

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">3.2. NP-полнота
 <ul class="list-disc list-inside text-slate-300">
 Точный перебор с отсеечениями


```



```

 Код: верификатор сертификата и пр
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
 ьтонов цикл NP-полон: сертификат = порядок
def verify_hamilton(n, edges, cert):
 if sorted(cert) != list(range(n)): # каждая
 return False
 allowed = {(u, v) for u, v in edges} | {(v, u) for
 for i in range(n):
 a, b = cert[i], cert[(i + 1) % n] # замык
 if (a, b) not in allowed:
 return False
 return True

3-SAT $\leq$ р КЛИКА: формула с k дизъюнктов $\rightarrow$ граф из $3k$ в
def reduce_3sat_to_clique(clauses):
 # вершина = (номер дизъюнкта, литерал); ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)
 edges = []
 for i in range(len(verts)):
 for j in range(i + 1, len(verts)):
 ci, li = verts[i]; cj, lj = verts[j]
 if ci == cj: # из од
 continue
 if li.lstrip("-") == lj.lstrip("-") and (li
 continue # x и n
 edges.append((i, j))
 return verts, edges, len(clauses) # клика

Проверка задачи из P для контраста: двудольность = 2-р
def is_bipartite(n, graph):
 from collections import deque
 col = [-1] * n
 for s in range(n):
 if col[s] != -1:
 continue
 col[s] = 0

```

```
];
```

ФАЙЛ 30/111: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 576 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```
{
```

```
 id: "graph-dfs-bfs",
```

```
 title: "6. Графы. Представление графов, DFS, BFS",
```

```
 type: "html",
```

```
 description: "Представление графов и базовые обходы
```

```
 category: "Графы. База",
```

```
 content: `
```

```
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
```

```
</div>
```

```
<div class="space-y-8">
```

```
 { /* База представлений */ }
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
```

```
 <h3 class="text-xl font-bold text-slate-300">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-slate-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 + E).</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 right-4 bottom-4">
```

```
 <div border-emerald-500 shadow-md">
```

```
 Аналогия: Матрица = Таблица
```

```
</div>
```

```

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-0">
 Что делает: Распространение по соседям. Работает через Очередь (FIFO). Гарантирует минимальное количество шагов.
 </p>
 <ul class="text-xs text-emerald-300 list-disc list-inside">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Проверка двудольности графа
 Алгоритм Ахо-Корасик (построение автомата)
 Двунаправленный BFS для поиска кратчайшего пути (в неориентированном графе)

 </div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #008080; border-radius: 10px 10px 0px 0px; box-shadow: 5px 5px 0px #008080;">
 Код обходов C++ (Основа)
 </summary>
 <div class="grid grid-cols-1 md:grid-cols-2" style="border: 1px solid #008080; border-radius: 10px 10px 10px 10px; box-shadow: 5px 5px 0px #008080;">
 <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #008080; border-radius: 10px 10px 10px 10px; box-shadow: 5px 5px 0px #008080;">
// DFS (Рекурсия с возвратом)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}
</pre>

```

```
<p class="text-2xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
</div>
```

```
<div class="space-y-4 text-slate-300 text-s">
 <p>Главные неравенства на число рёбер
 <ul class="list-disc list-inside space-y">
 Для простого планарного графа
 <i>Доказательство:</i> Каждая грань

 Для простого двудольного графа
 .

 <i>Доказательство:</i> В двудольном


```

```
<div class="bg-slate-950 p-4 rounded-lg">
 <p class="text-amber-300 font-bold">
 <p class="text-slate-300 text-xs le">
 1. K_5 (полный граф на 5 вершинах)
 $10 \leq 3(5) - 6 = 9$
 2. $K_{3,3}$ (полный двудольный 3×3)
 $9 \leq 12$ – формально при
 $2V - 4$: $9 \leq 2 \cdot 6 - 4 = 8$
 </p>
</div>
```

```
<p>Теорема Куратовского (1930) / Вагнера
 подразделением (subdivision)
</p>
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 рой вершин есть путь. Внешний цикл «для каж
 » даёт и разметку, и количество компонент.<
 </div>

 <p class="text-slate-300 text-sm mb-4">Два р
 ной запускаем DFS или BFS и присваиваем все
 mp[v] = c. Счётчик c увеличивается р
 Время $O(V + E)$, память $O(V)$. Чере
 каждое ребро делаем union(u, v), в конце чи
 ту и после каждого добавления спрашиваю

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm">Т
 ались серые (неисследованные) зоны? Прыгаеш
 и компонент. Краска — это comp[v], прыжок ч
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border border-emerald-500 shadow-md">
 Аналогия 2: Группы друзей (DS
 </div>
 <p class="text-slate-300 text-sm">К
 омпании в одну (union). Кто представитель к
 биты на компании, и вопрос «в одной ли комп
 рут, только если концы в разных компонентах

```

```
category: "Графы",
content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb">
екая сортировка — это расписание, которое
р>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 <div class="bg-rose-950/40 p-4 rounded-lg b">
 <p class="text-rose-300 font-bold text">
 <p class="text-slate-300 text-xs leadin`
```

```
bool topological_sort(int n) {
 fill(color, color + n + 1, 0);
 has_cycle = false;
 ans.clear();
 for (int i = 1; i <= n; ++i) {
 if (color[i] == 0) dfs(i);
 }
 if (has_cycle) return false; // Граф содержит цикл!
 reverse(ans.begin(), ans.end()); // Разворачиваем р
 return true;
}
```

</div>

</details>

</div>

</div>

</section>`,

},

{

id: "scc-kosaraju",

title: "10. Графы. Компоненты сильной связности (SCC)",

type: "html",

description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",

category: "Графы. Продвинутое",

content: `

<section id="scc-kosaraju" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-blue-600 text-white px-4 py-1 rounded">Теория</span>

<h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)</h2>

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-emerald-400">Разбиение орграфа на макро-вершины. Алгоритм Косарaju</h3>

<div class="bg-slate-900 p-4 rounded-lg mb-4">

<p class="text-lg text-emerald-300 italic">Граф состоит из  $n$  вершин и  $m$  ребер. Требуется разбить граф на макро-вершины (компоненты сильной связности), так чтобы каждая макро-вершина была максимальным набором вершин, между которыми есть путь в обоих направлениях.</p>

<div class="text-left font-mono text-sm">

```

content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="bg-slate-950 p-4 rounded-lg border">
 <p class="text-amber-300 font-bold text">
 <p class="text-slate-300 text-xs leading">
 Проверка <span class="font-mono text-
тиграфе между u и v может быть 2 параллельны
азуют цикл длиной 2.

 Решение: передавать в DFS не v
rald-300">p_edge_id (индекс ребра, по
 </p>
 </div>

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код поиска мостов C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
// Для мультиграфа с edge_id:
vector<pair<int, int>>> adj[MAXN]; // {to, edge_id}
int tin[MAXN], low[MAXN], timer;

void dfs_bridges(int v, int p, int edge_id = -1) {

```



```

 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex">
 <p class="text-xs text-slate-400">
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Особый случай: Корень дерева DFS
мого ребёнка в дереве DFS.
</p>

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код поиска точек сочленения (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
void dfs_cutpoints(int v, int p = -1) {
 tin[v] = low[v] = ++timer;
 int children = 0;
 for (int to : adj[v]) {
 if (to == p) continue;
 if (tin[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs_cutpoints(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1) {
 is_cutpoint[v] = true;
 }
 }
 ++children;
 }
}

```

```

 Эйлеров цикл: связан
-emerald-300">deg[v] % 2 == 0.
 Эйлеров путь: связан
o 2 – путь начинается в одной нечётной и за

 </div>

 <!-- Ориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg" style="background-color: #2d3748; color: white; padding: 10px;">
 <p class="text-indigo-300 font-bold">Графы с чётными степенями</p>
 <ul class="list-disc list-inside" style="list-style-type: none; padding-left: 20px;">
 Эйлеров цикл: слабос
ald-300">in[v] == out[v].
 Эйлеров путь: слабос
s] = 1 (старт), у одной <span class=
an class="font-mono text-emerald-300">in[v]

 </div>
</div>

<h3 class="text-lg font-bold text-white mt-4">Исполнение</h3>
<p class="text-slate-300 text-sm mb-3">Исполнение алгоритма:
 → рекурсивный переход. При возврате (на вы
ты: in[v]

<details class="bg-slate-900/40 rounded-lg" style="background-color: #1a202c; border-radius: 10px; padding: 10px;">
 <summary class="p-4 font-bold text-slate-300" style="background-color: #2d3748; color: white; padding: 5px 10px;">
 -b border-slate-700/50">
 Код построения Эйлерова цикла C++
 </summary>
 <div class="p-5 text-sm text-slate-300" style="background-color: #2d3748; color: white; padding: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a202c; border-radius: 10px; padding: 5px 10px;">
vector<unordered_multiset<int>>> adj;
vector<int> path;

void find_euler(int v) {
 while (!adj[v].empty()) {
 int u = *adj[v].begin();

```

```

}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
}

```

```

 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}
 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}]
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]
 v,
);

 if (currentLow[to] >= currentTin[v] && p !== -1) {
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
}

```

```

 }, [isPlaying, currentStepIndex, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />

```

```

 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
const stroke = isCurrent
? "#60a5fa"
: isAp
? "#f43f5e"
: isVisited
? "#34d399"
: "#334155";
const r = isAp ? 24 : 20;

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.label}
 </text>
 </g>
)

```

```

<div className="bg-slate-900 border border-slate-800" data-bbox="233 118 1000 235">
 <p className="text-white font-medium min-h-[100px]" data-bbox="273 148 1000 193">
 {currentStep.desc}
 </p>
</div>

```

```

<div className="space-y-4 flex-1 overflow-y-auto" data-bbox="233 262 1000 734">
 <div className="mb-4" data-bbox="273 292 1000 714">
 <h5 className="text-xs text-slate-500 font-medium" data-bbox="303 312 1000 332">Шаг 1: Подготовка
 <div className="flex items-center gap-2" data-bbox="303 337 1000 402">
 <div className="w-3 h-3 rounded-full bg-slate-400" data-bbox="338 362 608 382">
 Точка сочленения
 </div>
 <div className="flex items-center gap-2" data-bbox="303 432 1000 497">
 <div className="w-3 h-3 rounded-full bg-slate-400" data-bbox="338 457 608 477">
 Посещена
 </div>
 <div className="flex items-center gap-2" data-bbox="303 527 1000 592">
 <div className="w-3 h-3 rounded-full bg-slate-400" data-bbox="338 552 608 572">
 Текущая
 </div>
 <div className="flex items-center gap-2" data-bbox="303 622 1000 687">
 <div className="w-8 h-1 bg-rose-500 border" data-bbox="338 647 1000 667">
 Мост
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

```

<div>
 <h5 className="text-xs text-slate-500 font-medium" data-bbox="303 786 1000 831">ЛОГ ДЕЙСТВИЙ:
</h5>
 <div className="space-y-2" data-bbox="303 856 1000 996">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => (
 <div data-bbox="403 976 1000 996">

```

```
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";
```

```
export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];
```

```
 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
```



```
simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расст
 activeCodeLine: 2,
});
```

```
const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }
 }
}
```

```
simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v
 distances: { ...dist }, previous: { ...prev }
 log: `Итерация ${iter}: Проверяем ${edge.u} →
 updated ? ` Релаксация успешна! dist[${edge
 }`,
 activeCodeLine: updated ? 6 : 5,
});
}
```

```
if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distanc
 log: ` Ранний выход после итерации ${iter}.
 activeCodeLine: 10
 });
}
```

```

 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node.id, liveDist[node.id]])),
 log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} - ${liveV}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-blue-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-blue-300" >Полный
 </p>
 </div>
 </div>
 </div>
 </div>

```

```

<div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50
 ▲ Отрицательный цикл!
</div>
})

<svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-bf-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth=
 path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u)
 const vNode = nodes.find((n) => n.id === v)
 const isChecking = currentStep.checkingEdges
 v;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D'
 && edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b';
 else if (currentStep.hasNegativeCycle &&

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={
 asNegativeCycle && isCyclePart) ? 4 : 2} marker={isChecking ? 'url(#arrow-bf-active)' : isNeg ? 'url(#arrow-bf-cycle)' : 'url(#arrow-bf-normal)'} />
 <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} r={2} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#e11d48'} />
 <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Negative' : 'Cycle'} />
 </g>
)
 })}

```

```

 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge
 return (
 <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
 })}
 </div>
)}
 </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-300">
 <div className="grid grid-cols-4 gap-2 text-rose-300">
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`}>
 <div className="text-xs text-slate-400">
 <div className={`font-mono font-bo
 </div>
 </div>
 })}
 </div>
 </div>
 </div>
</div>

 {/* Код алгоритма */}

```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0).",
 });

 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 });
}
```

```

 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.`,
 });
 }
}
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`,
 });
}
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

```

```

 className="p-2 text-blue-400 hover:text-blue-500"
 title={isPlaying ? 'Пауза' : 'Воспроизведение'}
 >
 {isPlaying ? <Pause className="w-5 h-5" /> : <Play className="w-5 h-5" />}
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-emerald-500 hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-2 gap-4" >
 {/** Визуал графа */}
 <div className="bg-slate-800 border border-slate-700 p-4 min-h-[350px]" >
 <svg className="w-full h-full min-h-[350px]" >
 {/** Рёбра */}
 {EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1.id)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500 ${isVisited ? 'stroke-emerald-400' : 'stroke-blue-400'}`}
 />
)
 })}
 </svg>
 {/** Вершины */}

```

```

 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-slate-400"
 >
 {n.label}
 </text>
 </g>
);
}
</svg>
</div>

```

{/\* Код и Очередь \*/}

```

<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4 font-mono">
 bfs(graph, start)
 Mar {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visited.add(start);' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmpty() {',
 active: true },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node] {',
 active: true },
 { line: 7, code: ' if neighbor not in visited {',
 active: true },
 { line: 8, code: ' visited.add(neighbor); queue.push(neighbor);',
 active: true },
 { line: 9, code: ' }',
 active: true },
 { line: 10, code: ' }',
 active: true },
 { line: 11, code: '}' },
]}.map((cl) => {
 const isActive = step.activeLine === cl.line;
 return (
 <div key={cl.line} className={`px-2 py-1 border-l-2 border-blue-400 -ml-0.5` : `text-slate-400` : `text-blue-400`} >


```



```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string; }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Фриказоид (Декстер Дуглас)',
 caption: '⚡ Фриказоид (Декстер Дуглас) – работает то',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;
```

```
 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor} shadow-2xl`} style={{ width: 100%, height: 100%; position: relative; }}>

 <div style={info.captionColor} position: absolute; bottom: 10px; right: 10px; text-align: right; font-weight: bold; font-size: 1.2em; color: {info.captionColor};>{info.caption}</div>
 </div>
 </div>
);
}
```

```

 className="flex items-center gap-1 px-2.5 py-
 :bg-slate-700 enabled:hover:text-white disabl
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>
 <span className="text-slate-500 font-mono tabul
 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тем
 className="flex items-center gap-1 px-2.5 py-
 :bg-slate-700 enabled:hover:text-white disabl
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
}

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-
 transition-all"
 >
 <div className="flex items-center gap-1.5 tex
 400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Пре
 </div>
 <div className="text-sm font-bold text-slate-
 </button>
) : (

```

\* компонент-викторина существовали, но не были подключены  
\* контент был, а добраться до него было нельзя. Теперь  
\* демонстрации: прочитал → посмотрел, как работает → протестировал  
\*  
\* В текстовой версии (`?lite=1`) и в PDF те же вопросы  
\* с ответами и разбором – там интерактив не нужен.  
\*/

```
export const ChapterQuiz: React.FC<Props> = ({ chapterId }) => {
 const list = quizzes[chapterId];
 const [idx, setIdx] = useState(0);
 const [picked, setPicked] = useState<number | null>(null);
 const [score, setScore] = useState(0);
 const [finished, setFinished] = useState(false);
```

```
 // другая тема – другой блиц с нуля
```

```
 useEffect(() => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);
```

```
 if (!list || list.length === 0) return null;
```

```
 const q = list[idx];
 const answered = picked !== null;
 const correct = answered && picked === q.correctIndex;
```

```
 const pick = (i: number) => {
 if (answered) return;
 setPicked(i);
 if (i === q.correctIndex) setScore((s) => s + 1);
 };
```

```
 const next = () => {
 if (idx + 1 < list.length) {
 setIdx(idx + 1);
 setPicked(null);
 }
 };
```

```

 type="button"
 onClick={restart}
 className="inline-flex items-center gap-1
 -bold text-indigo-200 hover:bg-indigo-500/20
 >
 <RotateCcw className="w-3.5 h-3.5" /> Проп
 </button>
 </div>
) : (
 <>
 <p className="text-sm font-semibold text-wh
 <div className="space-y-2">
 {q.options.map((option, i) => {
 const isRight = i === q.correctIndex;
 const chosen = picked === i;
 const tone = !answered
 ? "border-slate-600 bg-slate-800/70 t
 : isRight
 ? "border-emerald-500 bg-emerald-500
 : chosen
 ? "border-rose-500 bg-rose-500/15
 : "border-slate-700 bg-slate-800/
 return (
 <button
 key={i}
 type="button"
 onClick={() => pick(i)}
 disabled={answered}
 className={`w-full text-left flex i
 <div disabled:cursor-default ${tone}`}
 >
 {answered && isRight && <CheckCircle
 {answered && chosen && !isRight &&
 {option}
 </button>
);
 })}
 </div>

```

```

import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermPopover";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";

```

```

interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлен код */
 onOpenCompiler?: () => void;
}

```

```

const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;

```

```

export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator, onOpenCompiler }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

```

```

 useTermHints(contentRef, [chapter.id]);

```

```

 /**

```

```

 * Упоминания «билет 15» становятся переходами на стр. 15

```

```

 * Разметка идёт ПОСЛЕ терминов: подсказки глоссария

```

```

 */

```

```

 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);
 return () => {
 window.removeEventListener("beforeprint", openAll);
 window.removeEventListener("afterprint", restore);
 };
 }, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");

```

```

 <Loader2 className="w-3.5 h-3.5 animate-s
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emeral
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Ска
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить эту тему в
страция, без шапки, содержания и панели ком
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-indigo-500
>
 <Printer className="w-3.5 h-3.5" /> PDF это
</button>
<a
 href={`?lite=1&topic=${encodeURIComponent(c
 target="_blank"
 rel="noreferrer"
 title="Текстовая версия темы без навигации,
еры"
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-sky-500 t
>
 <FileText className="w-3.5 h-3.5" /> Текст

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}

```

```

 </div>
 </div>
 {viz.hint && <p className="text-xs text-slate-500">
 <VizChapterContext.Provider value={chapter}>
 <VizLiveStatus demoTitle={viz.title} className="text-sm">
 <viz.Component />
 </VizChapterContext.Provider>
 </section>
)}

 {/* прочитал → посмотрел, как работает → проверить */}
 <ChapterQuiz chapterId={chapter.id} />

 <ChapterNav prev={prev} next={next} index={index} />
 </article>

 <TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
 </>
);
};

```

---

ФАЙЛ 38/111: src/components/ComplexityViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'lucide-react';

```



Универсальное пособие • полный исходный код

```

 ид", "MST", "2-SAT, двудольность"] },
];
```

```
const NP_HARD_NOTE = "NP-трудные — всё, к чему сводится
 амильтонов цикл. Проблема остановки NP-трудна, но в NP
```

```
/* ----- Сведение -----
```

```
const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма
 — то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x — конкретный
 \"x\"], active: 1 },
 { title: "Строим $f(x) = y$ за полиномиальное время", note: "Полиномиальное
 шинами из разных дизъюнктов, если литералы не противоречивы", active: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм B
 — но допустим, что он есть.", boxes: ["A", "x", "f"], active: 3 },
 { title: "Переводим ответ обратно: $A(x) = B(f(x))$ ", note: "Получили ответ
 $\Leftrightarrow$ формула выполнима. Полиномиальное время сохранилось", active: 5 },
 { title: "Вывод: B не проще A", note: "Раз $A \leq_p B$ и A NP-полна,
 -полна.", boxes: ["A", "x", "f", "y", "B", "ans"], active: 6 },
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> = {
 A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
 x: { t: "вход x\n|x| = n", c: "#64748b" },
 f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
 y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
 B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
 ans: { t: "ответ\nA(x) = B(f(x))", c: "#10b981" },
};
```

```
/* ----- Верификатор -----
```

```
/** 3-SAT формула: 4 дизъюнкта по 3 литерала, переменные 1..n
const CLAUSES: [number, number, number][] = [
 [1, 2, 3],
 [1, 2, 4],
 [1, 3, 5],
 [2, 3, 6],
 [2, 4, 7],
 [3, 4, 8],
 [4, 5, 9],
 [5, 6, 10],
 [6, 7, 11],
 [7, 8, 12],
 [8, 9, 13],
 [9, 10, 14],
 [10, 11, 15],
 [11, 12, 16],
 [12, 13, 17],
 [13, 14, 18],
 [14, 15, 19],
 [15, 16, 20],
 [16, 17, 21],
 [17, 18, 22],
 [18, 19, 23],
 [19, 20, 24],
 [20, 21, 25],
 [21, 22, 26],
 [22, 23, 27],
 [23, 24, 28],
 [24, 25, 29],
 [25, 26, 30],
 [26, 27, 31],
 [27, 28, 32],
 [28, 29, 33],
 [29, 30, 34],
 [30, 31, 35],
 [31, 32, 36],
 [32, 33, 37],
 [33, 34, 38],
 [34, 35, 39],
 [35, 36, 40],
 [36, 37, 41],
 [37, 38, 42],
 [38, 39, 43],
 [39, 40, 44],
 [40, 41, 45],
 [41, 42, 46],
 [42, 43, 47],
 [43, 44, 48],
 [44, 45, 49],
 [45, 46, 50],
 [46, 47, 51],
 [47, 48, 52],
 [48, 49, 53],
 [49, 50, 54],
 [50, 51, 55],
 [51, 52, 56],
 [52, 53, 57],
 [53, 54, 58],
 [54, 55, 59],
 [55, 56, 60],
 [56, 57, 61],
 [57, 58, 62],
 [58, 59, 63],
 [59, 60, 64],
 [60, 61, 65],
 [61, 62, 66],
 [62, 63, 67],
 [63, 64, 68],
 [64, 65, 69],
 [65, 66, 70],
 [66, 67, 71],
 [67, 68, 72],
 [68, 69, 73],
 [69, 70, 74],
 [70, 71, 75],
 [71, 72, 76],
 [72, 73, 77],
 [73, 74, 78],
 [74, 75, 79],
 [75, 76, 80],
 [76, 77, 81],
 [77, 78, 82],
 [78, 79, 83],
 [79, 80, 84],
 [80, 81, 85],
 [81, 82, 86],
 [82, 83, 87],
 [83, 84, 88],
 [84, 85, 89],
 [85, 86, 90],
 [86, 87, 91],
 [87, 88, 92],
 [88, 89, 93],
 [89, 90, 94],
 [90, 91, 95],
 [91, 92, 96],
 [92, 93, 97],
 [93, 94, 98],
 [94, 95, 99],
 [95, 96, 100],
 [96, 97, 101],
 [97, 98, 102],
 [98, 99, 103],
 [99, 100, 104],
 [100, 101, 105],
 [101, 102, 106],
 [102, 103, 107],
 [103, 104, 108],
 [104, 105, 109],
 [105, 106, 110],
 [106, 107, 111],
 [107, 108, 112],
 [108, 109, 113],
 [109, 110, 114],
 [110, 111, 115],
 [111, 112, 116],
 [112, 113, 117],
 [113, 114, 118],
 [114, 115, 119],
 [115, 116, 120],
 [116, 117, 121],
 [117, 118, 122],
 [118, 119, 123],
 [119, 120, 124],
 [120, 121, 125],
 [121, 122, 126],
 [122, 123, 127],
 [123, 124, 128],
 [124, 125, 129],
 [125, 126, 130],
 [126, 127, 131],
 [127, 128, 132],
 [128, 129, 133],
 [129, 130, 134],
 [130, 131, 135],
 [131, 132, 136],
 [132, 133, 137],
 [133, 134, 138],
 [134, 135, 139],
 [135, 136, 140],
 [136, 137, 141],
 [137, 138, 142],
 [138, 139, 143],
 [139, 140, 144],
 [140, 141, 145],
 [141, 142, 146],
 [142, 143, 147],
 [143, 144, 148],
 [144, 145, 149],
 [145, 146, 150],
 [146, 147, 151],
 [147, 148, 152],
 [148, 149, 153],
 [149, 150, 154],
 [150, 151, 155],
 [151, 152, 156],
 [152, 153, 157],
 [153, 154, 158],
 [154, 155, 159],
 [155, 156, 160],
 [156, 157, 161],
 [157, 158, 162],
 [158, 159, 163],
 [159, 160, 164],
 [160, 161, 165],
 [161, 162, 166],
 [162, 163, 167],
 [163, 164, 168],
 [164, 165, 169],
 [165, 166, 170],
 [166, 167, 171],
 [167, 168, 172],
 [168, 169, 173],
 [169, 170, 174],
 [170, 171, 175],
 [171, 172, 176],
 [172, 173, 177],
 [173, 174, 178],
 [174, 175, 179],
 [175, 176, 180],
 [176, 177, 181],
 [177, 178, 182],
 [178, 179, 183],
 [179, 180, 184],
 [180, 181, 185],
 [181, 182, 186],
 [182, 183, 187],
 [183, 184, 188],
 [184, 185, 189],
 [185, 186, 190],
 [186, 187, 191],
 [187, 188, 192],
 [188, 189, 193],
 [189, 190, 194],
 [190, 191, 195],
 [191, 192, 196],
 [192, 193, 197],
 [193, 194, 198],
 [194, 195, 199],
 [195, 196, 200],
 [196, 197, 201],
 [197, 198, 202],
 [198, 199, 203],
 [199, 200, 204],
 [200, 201, 205],
 [201, 202, 206],
 [202, 203, 207],
 [203, 204, 208],
 [204, 205, 209],
 [205, 206, 210],
 [206, 207, 211],
 [207, 208, 212],
 [208, 209, 213],
 [209, 210, 214],
 [210, 211, 215],
 [211, 212, 216],
 [212, 213, 217],
 [213, 214, 218],
 [214, 215, 219],
 [215, 216, 220],
 [216, 217, 221],
 [217, 218, 222],
 [218, 219, 223],
 [219, 220, 224],
 [220, 221, 225],
 [221, 222, 226],
 [222, 223, 227],
 [223, 224, 228],
 [224, 225, 229],
 [225, 226, 230],
 [226, 227, 231],
 [227, 228, 232],
 [228, 229, 233],
 [229, 230, 234],
 [230, 231, 235],
 [231, 232, 236],
 [232, 233, 237],
 [233, 234, 238],
 [234, 235, 239],
 [235, 236, 240],
 [236, 237, 241],
 [237, 238, 242],
 [238, 239, 243],
 [239, 240, 244],
 [240, 241, 245],
 [241, 242, 246],
 [242, 243, 247],
 [243, 244, 248],
 [244, 245, 249],
 [245, 246, 250],
 [246, 247, 251],
 [247, 248, 252],
 [248, 249, 253],
 [249, 250, 254],
 [250, 251, 255],
 [251, 252, 256],
 [252, 253, 257],
 [253, 254, 258],
 [254, 255, 259],
 [255, 256, 260],
 [256, 257, 261],
 [257, 258, 262],
 [258, 259, 263],
 [259, 260, 264],
 [260, 261, 265],
 [261, 262, 266],
 [262, 263, 267],
 [263, 264, 268],
 [264, 265, 269],
 [265, 266, 270],
 [266, 267, 271],
 [267, 268, 272],
 [268, 269, 273],
 [269, 270, 274],
 [270, 271, 275],
 [271, 272, 276],
 [272, 273, 277],
 [273, 274, 278],
 [274, 275, 279],
 [275, 276, 280],
 [276, 277, 281],
 [277, 278, 282],
 [278, 279, 283],
 [279, 280, 284],
 [280, 281, 285],
 [281, 282, 286],
 [282, 283, 287],
 [283, 284, 288],
 [284, 285, 289],
 [285, 286, 290],
 [286, 287, 291],
 [287, 288, 292],
 [288, 289, 293],
 [289, 290, 294],
 [290, 291, 295],
 [291, 292, 296],
 [292, 293, 297],
 [293, 294, 298],
 [294, 295, 299],
 [295, 296, 300],
 [296, 297, 301],
 [297, 298, 302],
 [298, 299, 303],
 [299, 300, 304],
 [300, 301, 305],
 [301, 302, 306],
 [302, 303, 307],
 [303, 304, 308],
 [304, 305, 309],
 [305, 306, 310],
 [306, 307, 311],
 [307, 308, 312],
 [308, 309, 313],
 [309, 310, 314],
 [310, 311, 315],
 [311, 312, 316],
 [312, 313, 317],
 [313, 314, 318],
 [314, 315, 319],
 [315, 316, 320],
 [316, 317, 321],
 [317, 318, 322],
 [318, 319, 323],
 [319, 320, 324],
 [320, 321, 325],
 [321, 322, 326],
 [322, 323, 327],
 [323, 324, 328],
 [324, 325, 329],
 [325, 326, 330],
 [326, 327, 331],
 [327, 328, 332],
 [328, 329, 333],
 [329, 330, 334],
 [330, 331, 335],
 [331, 332, 336],
 [332, 333, 337],
 [333, 334, 338],
 [334, 335, 339],
 [335, 336, 340],
 [336, 337, 341],
 [337, 338, 342],
 [338, 339, 343],
 [339, 340, 344],
 [340, 341, 345],
 [341, 342, 346],
 [342, 343, 347],
 [343, 344, 348],
 [344, 345, 349],
 [345, 346, 350],
 [346, 347, 351],
 [347, 348, 352],
 [348, 349, 353],
 [349, 350, 354],
 [350, 351, 355],
 [351, 352, 356],
 [352, 353, 357],
 [353, 354, 358],
 [354, 355, 359],
 [355, 356, 360],
 [356, 357, 361],
 [357, 358, 362],
 [358, 359, 363],
 [359, 360, 364],
 [360, 361, 365],
 [361, 362, 366],
 [362, 363, 367],
 [363, 364, 368],
 [364, 365, 369],
 [365, 366, 370],
 [366, 367, 371],
 [367, 368, 372],
 [368, 369, 373],
 [369, 370, 374],
 [370, 371, 375],
 [371, 372, 376],
 [372, 373, 377],
 [373, 374, 378],
 [374, 375, 379],
 [375, 376, 380],
 [376, 377, 381],
 [377, 378, 382],
 [378, 379, 383],
 [379, 380, 384],
 [380, 381, 385],
 [381, 382, 386],
 [382, 383, 387],
 [383, 384, 388],
 [384, 385, 389],
 [385, 386, 390],
 [386, 387, 391],
 [387, 388, 392],
 [388, 389, 393],
 [389, 390, 394],
 [390, 391, 395],
 [391, 392, 396],
 [392, 393, 397],
 [393, 394, 398],
 [394, 395, 399],
 [395, 396, 400],
 [396, 397, 401],
 [397, 398, 402],
 [398, 399, 403],
 [399, 400, 404],
 [400, 401, 405],
 [401, 402, 406],
 [402, 403, 407],
 [403, 404, 408],
 [404, 405, 409],
 [405, 406, 410],
 [406, 407, 411],
 [407, 408, 412],
 [408, 409, 413],
 [409, 410, 414],
 [410, 411, 415],
 [411, 412, 416],
 [412, 413, 417],
 [413, 414, 418],
 [414, 415, 419],
 [415, 416, 420],
 [416, 417, 421],
 [417, 418, 422],
 [418, 419, 423],
 [419, 420, 424],
 [420, 421, 425],
 [421, 422, 426],
 [422, 423, 427],
 [423, 424, 428],
 [424, 425, 429],
 [425, 426, 430],
 [426, 427, 431],
 [427, 428, 432],
 [428, 429, 433],
 [429, 430, 434],
 [430, 431, 435],
 [431, 432, 436],
 [432, 433, 437],
 [433, 434, 438],
 [434, 435, 439],
 [435, 436, 440],
 [436, 437, 441],
 [437, 438, 442],
 [438, 439, 443],
 [439, 440, 444],
 [440, 441, 445],
 [441, 442, 446],
 [442, 443, 447],
 [443, 444, 448],
 [444, 445, 449],
 [445, 446, 450],
 [446, 447, 451],
 [447, 448, 452],
 [448, 449, 453],
 [449, 450, 454],
 [450, 451, 455],
 [451, 452, 456],
 [452, 453, 457],
 [453, 454, 458],
 [454, 455, 459],
 [455, 456, 460],
 [456, 457, 461],
 [457, 458, 462],
 [458, 459, 463],
 [459, 460, 464],
 [460, 461, 465],
 [461, 462, 466],
 [462, 463, 467],
 [463, 464, 468],
 [464, 465, 469],
 [465, 466, 470],
 [466, 467, 471],
 [467, 468, 472],
 [468, 469, 473],
 [469, 470, 474],
 [470, 471, 475],
 [471, 472, 476],
 [472, 473, 477],
 [473, 474, 478],
 [474, 475, 479],
 [475, 476, 480],
 [476, 477, 481],
 [477, 478, 482],
 [478, 479, 483],
 [479, 480, 484],
 [480, 481, 485],
 [481, 482, 486],
 [482, 483, 487],
 [483, 484, 488],
 [484, 485, 489],
 [485, 486, 490],
 [486, 487, 491],
 [487, 488, 492],
 [488, 489, 493],
 [489, 490, 494],
 [490, 491, 495],
 [491, 492, 496],
 [492, 493, 497],
 [493, 494, 498],
 [494, 495, 499],
 [495, 496, 500],
 [496, 497, 501],
 [497, 498, 502],
 [498, 499, 503],
 [499, 500, 504],
 [500, 501, 505],
 [501, 502, 506],
 [502, 503, 507],
 [503, 504, 508],
 [504, 505, 509],
 [505, 506, 510],
 [506, 507, 511],
 [507, 508, 512],
 [508, 509, 513],
 [509, 510, 514],
 [510, 511, 515],
 [511, 512, 516],
 [512, 513, 517],
 [513, 514, 518],
 [514, 515, 519],
 [515, 516, 520],
 [516, 517, 521],
 [517, 518, 522],
 [518, 519, 523],
 [519, 520, 524],
 [520, 521, 525],
 [521, 522, 526],
 [522, 523, 527],
 [523, 524, 528],
 [524, 525, 529],
 [525, 526, 530],
 [526, 527, 531],
 [527, 528, 532],
 [528, 529, 533],
 [529, 530, 534],
 [530, 531, 535],
 [531, 532, 536],
 [532, 533, 537],
 [533, 534, 538],
 [534, 535, 539],
 [535, 536, 540],
 [536, 537, 541],
 [537, 538, 542],
 [538, 539, 543],
 [539, 540, 544],
 [540, 541, 545],
 [541, 542, 546],
 [542, 543, 547],
 [543, 544, 548],
 [544, 545, 549],
 [545, 546, 550],
 [546, 547, 551],
 [547, 548, 552],
 [548, 549, 553],
 [549, 550, 554],
 [550, 551, 555],
 [551, 552, 556],
 [552, 553, 557],
 [553, 554, 558],
 [554, 555, 559],
 [555, 556, 560],
 [556, 557, 561],
 [557, 558, 562],
 [558, 559, 563],
 [559, 560, 564],
 [560, 561, 565],
 [561, 562, 566],
 [562, 563, 567],
 [563, 564, 568],
 [564, 565, 569],
 [565, 566, 570],
 [566, 567, 571],
 [567, 568, 572],
 [568, 569, 573],
 [569, 570, 574],
 [570, 571, 575],
 [571, 572, 576],
 [572, 573, 577],
 [573, 574, 578],
 [574, 575, 579],
 [575, 576, 580],
 [576, 577, 581],
 [577, 578, 582],
 [578, 579, 583],
 [579, 580, 584],
 [580, 581, 585],
 [581, 582, 586],
 [582, 583, 587],
 [583, 584, 588],
 [584, 585, 589],
 [585, 586, 590],
 [586, 587, 591],
 [587, 588, 592],
 [588, 589, 593],
 [589, 590, 594],
 [590, 591, 595],
 [591, 592, 596],
 [592, 593, 597],
 [593, 594, 598],
 [594, 595, 599],
 [595, 596, 600],
 [596, 597, 601],
 [597, 598, 602],
 [598, 599, 603],
 [599, 600, 604],
 [600, 601, 605],
 [601, 602, 606],
 [602, 603, 607],
 [603, 604, 608],
 [604, 605, 609],
 [605, 606, 610],
 [606, 607, 611],
 [607, 608, 612],
 [608, 609, 613],
 [609, 610, 614],
 [610, 611, 615],
 [611, 612, 616],
 [612, 613, 617],
 [613, 614, 618],
 [614, 615, 619],
 [615, 616, 620],
 [616, 617, 621],
 [61
```

```

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 1;
useEffect(() => {
 setStep(0);
 setPlaying(false);
}, [tab]);

useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNumber(0)) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveCls = vars ? vizString(vars.cls) : null;
const liveK = vars ? vizNumber(vars.k) : null;
const liveCert = vars ? vizArray(vars.cert) ?? vizArray(0) : null;
const liveOk = vars ? (vars.ok === undefined ? null : vizBoolean(true)) : null;
const liveN = vars ? vizNumber(vars.n) : null;
const liveOps = vars ? (vizNumber(vars.ops) ?? vizNumber(0)) : null;

```

```

 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 return dd === 0 ? slope : nn / dd;
 })();
const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e0b",
 : slope <= 1.25
 ? { name: "≈ линейный, $O(n)$ ", color: "#10b981",
 : slope <= 1.8
 ? { name: "≈ $O(n \log n)$ ", color: "#06b6d4", hint: "log"
 : slope <= 2.4
 ? { name: "≈ квадратичный, $O(n^2)$ ", color: "#f59e0b",
 : slope <= 3.4
 ? { name: "≈ кубический, $O(n^3)$ ", color: "#fb9238",
 : { name: "полином высокой степени", color: "#fb9238"
 }
 return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-1" >
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-purple-600 text-white" >
 <Boxes className="w-5 h-5" />
 </div>
 </div>
 <h3 className="text-lg sm:text-xl font-bold" >
 <p className="text-xs text-purple-300">P · I
 </p>
 </div>
 </div>
)

```

```

<svg viewBox="0 0 500 380" className="w-full h-380">
 {CLASSES.map((c) => {
 const on = liveCls ? liveCls.toUpperCase() : ''
 return (
 <g key={c.id}>
 <ellipse cx={c.x} cy={c.y} rx={c.rx} ry={c.ry} strokeDasharray={on ? 3 : 1.5} stroke={c.stroke} />
 <text x={c.x} y={c.y - c.ry + 18} textAnchor="middle">{c.name}</text>
 <text x={c.x} y={c.y - c.ry + 32} textAnchor="middle">{c.desc}</text>
 </g>
)
 })}
 {/* примеры задач – внутри своих овалов */}
 {CLASSES.map((c) => {
 c.tasks.map((t, i) => {
 <text key={t} x={c.x} y={c.y + (c.id % 2) * 10} textAnchor="middle" fill="#e2e8f0" fontSize="8">
 {t}
 </text>
 })
 })}
 {/* NP-полные = пересечение */}
 <text x={250} y={352} textAnchor="middle">NP-полные = пересечение</text>
 <text x={250} y={368} textAnchor="middle">рхии времени</text>
</svg>
</div>
<p className="text-xs text-slate-400 leading-4">
 Подсказка: выполни в компиляторе <code>class NP-complete {
 </code>
</p>
</div>
)}

```

```

{/* _____ СВЕДЕНИЕ _____ */}
{tab === "reduce" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b

```

```

 {CERT.map((v, i) => {
 <span key={i} className={"font-mono text-
ld-600/50 text-emerald-300" : "bg-slate-900
 x{i + 1} = {liveCert && typeof liveCe

 }}}
 {liveCert && <span className="font-mono t
le-950/40">cert из кода}
</div>
<div className="space-y-2">
 {CLAUSES.map((cl, i) => {
 const checked = liveK !== null ? i < li
 const current = liveK !== null ? i ===
 const st = VERIFY_STEPS[i];
 return {
 <div
 key={i}
 className={"flex items-center gap-3
amber-950/20" : checked ? {st.ok ? "border-
slate-800 bg-slate-900/40"}}
 >
 <span className="text-[10px] font-b
 <span className="font-mono text-xs
span>
 <span className="font-mono text-[10
 ")}
 <span className={"text-[11px] font-l
0"))}>
 {checked ? (st.ok ? "✓ истинно" :

 </div>
);
 }}}
</div>
{(verifyDone || liveOk !== null) && (
 <div className={"mt-3 rounded-lg border p
/30 text-emerald-300" : "border-rose-600/40
 {(liveOk ?? allOk) ? "Сертификат принят

```

```

 pts.push(`${x.toFixed(1)}${Math.max(
 }
 return <polyline key={g.id} points={pts
acity={0.55} />;
)))
 {samples.map((s) => {
 const x = 44 + (Math.log(Math.max(1, s.n
 const y = 200 - (Math.log(Math.max(1, s
 return <circle key={`${s.n}`} cx={Math.m
strokeWidth={2} />;
)))
 {GROWTH_LINES.map((g, i) => {
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + 1
 <text x={343 + (i % 3) * 58} y={24 + 1
 </g>
)))
</svg>
{samples.length === 0 && {
 <p className="text-xs text-slate-500 text
 Пусто. Запусти в компиляторе код с пере
t-mono text-purple-300">ops</code> – напри
 </p>
})
</div>
<div className={"rounded-xl border p-3.5 " +
 /60"}>
 <p className="text-sm font-bold text-white
 {verdict ? `Похоже на ${verdict.name}` :
 </p>
 <p className="text-xs text-slate-400 leadin
 {verdict
 ? `Наклон в лог-лог координатах ≈ ${ver
dict.slopeTail.toFixed(2)} – кривая загибае
 : "Как только в коде появятся n и ops,
e с n)."}
 </p>
</div>

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: "}", highlight: "normal" },
];

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
}

```

```

 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
 activeEdge: [1,6], backEdge: null,
 log: "Из В идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме В). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из В идём в С. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа А (уже посещён)! Обратное ребро",
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
 activeEdge: [2,3], backEdge: null,
 log: "Идём С→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],

```



```

 activeCodeLine: [13,14,15,16,17]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:2,2:3,3:4,4:5,5:6},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) - мост! DFS завершён! Мосты: (B,G) и (C,D). Алгоритм готов!",
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:2,2:3,3:4,4:5,5:6},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм готов!",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:2,2:3,3:4,4:5,5:6},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм готов!",
 activeCodeLine: []
 },
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
 const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ? vizNumber(vars.v) : null);
 const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ? vizNumber(vars.to) : null);
 const toId = (value: string | null) => {
 if (value === null) return null;
 const byLabel = GRAPH_NODES.find((node) => node.label === value);
 return byLabel ?? (Number.isFinite(Number(value)) ? Number(value) : null);
 };
 const v = toId(rawV);
 const to = toId(rawTo);
 if (v === null) return null;

```

```

 </button>
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 aria-label="Шаг вперёд" title="Шаг вперёд"
 className="p-1.5 hover:bg-slate-700 rounded
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 aria-label="Сбросить DFS" title="Сбросить D
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
 {/* Graph visualization */}
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.ke
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.activ
));
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdg
 (step.backEdge[0]===e.v && step.backEdg
));
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw
 else if (step.visitedIndices.includes(e.u

```

```

 </div>
 </div>
 </div>

 {/* Code panel + stack */}
 <div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border" style={{borderColor: 'black', borderStyle: 'solid', borderWidth: 1px}}>
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400">
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed">
 {CODE_LINES.map((cl, i) => {
 const hl = step.activeCodeLine.includes(cl.line)
 return <div key={i}
 className={`px-1 transition-all duration-200 ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}>
 {i + 1}
 {cl.line || ' '}
 </div>
)}}
 </pre>
 </div>
 </div>
 </div>

 {/* Stack */}
 <div className="bg-slate-900/50 rounded-xl border" style={{borderColor: 'black', borderStyle: 'solid', borderWidth: 1px}}>
 <div className="text-[10px] font-bold text-slate-400">
 Стек DFS
 {step.stack.length}
 </div>
 <div className="flex flex-col-reverse gap-1">
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-400">Стек пуст</div>
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0.5 text-[10px] transition-all duration-200 ${i === step.stack.length - 1 ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}>
 {step.stack[id]}
 </div>
)}
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

```

interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}

if (!u) break;

visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: `  Выбираем вершину ${u} с мин. непосещённой вершиной`,
 activeCodeLine: 5,
});

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: `  Смотрим соседа ${v}. Текущее расстояние: ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({

```

```

const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
const compilerLinked = liveU !== null || liveV !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
 distances: Object.fromEntries(nodes.map((node) => [node, 0])),
 previous: Object.fromEntries(nodes.map((node) => [node, null])),
 visited: Object.fromEntries(nodes.map((node) => [node, false])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Полный
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap items-center justify-between" >
 <button
 onClick={() => { setIsPlaying(false); setCurrentStep(currentStep); }}
 disabled={!!compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Вернуть к началу"}
 className="flex items-center gap-1 bg-slate-700/80 p-3 px-3 py-2 rounded"
 >
 Сброс
 </button>
 </div>
 </div>
);

```

```

 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

```

```

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
 </marker>
 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 {/* Рёбра */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u)
 const vNode = nodes.find((n) => n.id === edge.v)
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v)

 const isOptimal = currentStep.previous[edge.u] === edge.v

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 />
 </g>
)
 })}

```

```

 "middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-center
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u
 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60
 t-slate-300'
 }`>
 <div className="flex items-center gap-2
 <span className={`px-2 py-0.5 rounded
 te-300'}`>
 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.to
 return (

```



```

 return (
 <tr key={n.id} className={`border
 isCurr ? 'bg-indigo-950/60 fo
 }`}>
 <td className="py-2.5 px-3 flex
 <span className={`w-2 h-2 rou
 {n.name}
 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 }}}
 </tbody>
 </table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg
 {codeLines.map((item) => {
 const isActive = currentStep.activeCode
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-

```

```
];
```

```
export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }
 }
 });
}
```

```

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: ` Расчет завершен! fibMemo(${targetN}) = $
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
 ? Object.fromEntries(Object.entries(liveMemo).flatMap(
 (entry) => {
 const number = vizNumber(entry.value);
 return number === null ? [] : [[Number(entry.key), number];
 }
))
 : baseStep?.memoState || {};

```

```

 </select>
 </div>

 <div className="flex items-center bg-slate-950" >
 <button
 onClick={() => { setIsPlaying(false); set
 className="p-2 text-slate-400 hover:text-
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:
 : 'bg-emerald-600 text-white hover:
 }`
 >
 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.leng
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>

```

```

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-500" />
 Текущий N: <strong className="text-emerald-400" /> {currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b" />
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 ${activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-slate-500'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Eye className="w-4 h-4" />
 Шаг-бай-шаг состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 ${activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg-slate-500'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify" />
 {activeTab === 'table' && {

```



```

const [c1Msg, setC1Msg] = useState("Кликни на вершину");
const [c1Done, setC1Done] = useState(false);

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "walk");
}, [chapterId]);

const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кликни на вершину");
};

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы пройти все рёбра");
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери другую вершину");
 return;
 }
 const key = ekey(last, vId);
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает повторений");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 if (nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, феникс!");
 } else {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, феникс!");
 }
 }
};

```

```

 path,
 keys: Array.from(new Set(keys)),
 deg,
 odd,
 last: last !== null && last >= 0 && last < C1_NODES.length,
 step,
 complete: keys.length >= C1_EDGES.length,
 };
}, [vars]));

```

```

const path = live && live.path.length ? live.path : c1Path;
const visitedKeys = live ? live.keys : c1VisitedEdges;
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[path.length - 1] : null;
const done = live ? live.complete : c1Done && visitedKeys.length === C1_EDGES.length;
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёбра. Число нечётных вершин ${odd.length} → это ${odd.length === 0 ? "цикл" : "маршрут"} из кода: ${live.path.length ? live.path.join(" ") : "начало"} / ${C1_EDGES.length}`
 : c1Msg;

```

```

const verdict =
 odd.length === 0
 ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт и финиш в одной вершине)" }
 : odd.length === 2
 ? { text: `2 нечётные (${odd.map((id) => C1_NODES[id].label).join(" и ")}) в другой`, cls: "bg-amber-950/40 border-amber-950/40" }
 : { text: `${odd.length} нечётных → эйлерова маршрута нет` };

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 Проверка существования эйлерова маршрута: все рёбра ровно по одному разу</h4>

```



```

 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
 stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
 strokeWidth={2.5}
 className="transition-all duration-200"
 <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="middle">deg {deg[n.id]}{isOdd ? " · нечёт" : ""}</text>
 </g>
 </svg>
 <div className="absolute top-2 left-2 bg-slate-500 text-white p-2">
 Ребёр: {visitedKeys.length}/{C1_EDGES.length}
 </div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done && !live
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"
 }`}>
 {msg}
 </div>

 {/* Критерий Эйлера */}
 <div className={`p-3 rounded-xl border text-xs font-mono ${
 verdict ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50" : "bg-rose-950/40 border-rose-500/30 text-rose-50"
 }`}>
 Критерий Эйлера (для связного графа): {verdict}

 Степени: {C1_NODES.map((n) => `${n.label}=${deg[n.id]}`).join(" ")}
 две нечётны. Алгоритм Иера строит маршрут за

 </div>

 <div className="flex flex-wrap items-center gap-2">


```

```

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 ва листать вручную.";

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольн
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

 const adjList: { [key: number]: { v: number; w: numbe
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],

```

```

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), u: 0,
 log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
 activeCodeLine: 7,
 });
 }
 }
}

```

```

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), u: 0,
 log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
 activeCodeLine: 8,
});

```

```

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

```

```

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1));
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

```

```
 : baseStep;
```

```
return (
```

```
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
```

```
 <div className="flex flex-wrap items-center justify-center" >
```

```
 <div className="flex items-center gap-3" >
```

```
 <div className="p-2.5 bg-emerald-600 text-white" >
```

```

```

```
 </div>
```

```
 <div>
```

```
 <h3 className="text-2xl font-bold text-white" >
```

```
 <p className="text-sm text-emerald-300" >Полн
```

```
 </div>
```

```
 </div>
```

```
 <div className="flex items-center gap-2" >
```

```
 <button onClick={() => { setIsPlaying(false);
```

```
 TEXT : "Вернуться к первому шагу" } className="
```

```
 disabled:cursor-not-allowed">Сброс</button>
```

```
 <button onClick={() => setIsPlaying(!isPlaying
```

```
 нную демонстрацию" } className={`px-4 py-2 rounded
```

```
 bg-white disabled ${isPlaying ? 'bg-amber-600' : 'bg-emera
```

```
 <button onClick={() => { setIsPlaying(false);
```

```
 ; }} disabled={compilerLinked} title={compiler
```

```
 id-600 text-white rounded-lg text-sm disabled
```

```
 </div>
```

```
</div>
```

```
<div className="flex flex-col lg:flex-row gap-6 ml-6" >
```

```
 <div className="flex flex-col" >
```

```
 <div className="flex flex-col" >
```

```
 <div className="w-full lg:w-2/3 bg-indigo-950/20" >
```

```
 <div className="flex flex-col justify-center min-h-[400px]" >
```

```
 <div className="absolute top-3 left-3 bg-indigo-950/20" >
```

```
 <div className="font-bold shadow-sm" >
```

```
 {currentStep.k >= 0 && currentStep.k < 7 ?
```

```
 </div>
```

```
 <div>
```

```
 <svg className="w-full h-auto max-h-[500px]" >
```

```

 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
s] ? 4 : 2} className="transition-colors du
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y + 32} fill=
 </text>
 </g>
);
 }}}}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
 </div>
 </div>

 {/* Список смежности */}
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 }`>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
rentStep.j === edge.v);
 return (

```

```

 {labelOf(i)} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== -
 tep.j === j});
 let bg = 'bg-slate-900/40 text-slate-50';
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50';
 else if (isVia) bg = 'bg-indigo-900/80 text-indigo-50';
 else if (currentStep.i === -1 && currentStep.k === -1)
 bg = 'bg-indigo-950/60 text-indigo-50';
 else if (i === j) bg = 'bg-slate-900/40 text-slate-50';
 const shown = val === null ? '.' : val;
 return (<td key={j} className={bg} style={textStyle}
 >{shown}</td>
)
)
 </tr>
 </tbody>
</table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-50">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-4
 e === item.line ? 'bg-blue-900/80 text-amber-400' :
 'bg-slate-950/80 text-slate-50'}>
 {item.line}
 {item.code}
 </div>
))}
 </div>
 </div>
</div>
</div>
</div>

```

```

 y: number;
}

/** Граф из 10 вершин и трёх компонент — ровно как в те
const NODES: Node[] = [
 { id: 0, label: "0", x: 90, y: 80 },
 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

const ADJ: number[][] = NODES.map(() => []);
EDGES.forEach(([u, v]) => {
 ADJ[u].push(v);
 ADJ[v].push(u);
});
ADJ.forEach((a) => a.sort((x, y) => x - y));

/** Палитра компонент. */
const COLORS = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];

interface DfsFrame {
 title: string;
 note: string;
 comp: number[]; // 0 = не покрашена, иначе н
 cur: number | null; // текущая вершина

```

```

 } вершин. Ищем следующую серую вершину.`});
 }
 push(null, [], с, "Готово: компонент = " + с, `Внешние
 вязности.`);
 return frames;
}

```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v ? v : find(parent[v]));
 let comps = NODES.length;
 const push = (title: string, note: string, edge: [number, number] | null, comps: number) => {
 frames.push({ title, note, parent: [...parent], comps });
 push("Старт: каждая вершина сама по себе", `parent[v] = v`, null, false);
 };
 EDGES.forEach(([u, v], i) => {
 const ru = find(u);
 const rv = find(v);
 if (ru === rv) {
 push(`Ребро ${i}: (${u}, ${v}) – корни совпали`, `parent[v] = ru`, null, comps);
 return;
 }
 parent[rv] = ru;
 comps -= 1;
 push(`Ребро ${i}: union(${u}, ${v})`, `find(${u}) = find(${v})`, [u, v], true);
 });
}

```



```

 if (!playing) return;
 timer.current = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 900);
 return () => {
 if (timer.current) window.clearInterval(timer.current);
 };
 }, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.floor(n / 1000)));

/* — переменные компилятора, которые рисуются НАПРЯМУЮ
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р-ция
const liveComp = useMemo(() => {
 const arr = vars ? (vizArray(vars.comp) ?? vizArray(vars.colors)) : [];
 if (!arr) return null;
 return arr.slice(0, NODES.length).map((v) => (typeof v === 'string' ? v : v.comp)), [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
 const arr = vars ? (vizArray(vars.parent) ?? vizArray(vars.colors)) : [];
 if (!arr || arr.length < NODES.length) return null;
 return arr.slice(0, NODES.length).map((v) => vizNumber(v.parent)), [vars]);
const liveCur = vars ? (vizNumber(vars.v) ?? vizNumber(vars.colors)) : null;
const liveComps = vars ? (vizNumber(vars.comps) ?? vizNumber(vars.colors)) : null;
const liveFrontier = useMemo(() => {
 if (!vars) return null;
 const arr = vizArray(vars.stack) ?? vizArray(vars.colors);
 if (!arr) return null;
 return arr.map((v) => vizNumber(v)).filter((v): v is number => v !== liveCur);
}, [vars]);

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-3xl">
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-3xl">
 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">DFS
 <p className="text-xs text-emerald-300">10 мс
 </div>
 </div>
 <div className="flex items-center bg-slate-950 p-5 rounded-3xl">
 <button
 onClick={() => switchMode("dfs")}
 className={
 "flex items-center gap-1.5 px-3 py-2 text-sm font-medium" +
 "bg-slate-800 text-white shadow" +
 "hover:bg-slate-700 hover:text-slate-400 hover:shadow"
 }
 >
 <Layers className="w-3.5 h-3.5" /> DFS (Dijkstra)
 </button>
 <button
 onClick={() => switchMode("dsu")}
 className={
 "flex items-center gap-1.5 px-3 py-2 text-sm font-medium" +
 "bg-slate-800 text-white shadow" +
 "hover:bg-slate-700 hover:text-slate-400 hover:shadow"
 }
 >
 <Network className="w-3.5 h-3.5" /> DSU (path)
 </button>
 </div>
 </div>

 {/* панель управления */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={() => { setStep(0); setPlaying(true); }}
 className="p-2 text-sm font-medium bg-slate-700 text-slate-300 hover:text-white transition"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setPlaying(false); goToStep(0); }}

```



```
 </div>
 </div>
);
 };
};

export default GraphComponentsViz;
```

---

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
```

```

 }

 dfs(start);
 return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}`,
 ray.from(vis)} });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Добавляем узел ${v} в очередь`,
 q, visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Визит узла ${u}`,
 tack: [...q], visitedNodes: [...Array.from(vis)] });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Очищаем очередь`,
 om(vis)} });

 return steps;
}

```

```

 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-700 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>

 <div className="flex flex-col md:flex-row gap-4">
 <div>
 <div className="bg-slate-900 border border-slate-700 p-4 min-h-[300px]">
 <svg className="w-full h-full max-w-[600px] max-h-[300px]">
 <g>
 <!-- Edges -->
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);
 // Check if edge is part of current step
 const edgeTraversed = isVisited(e.v) || isCurrent(e.v);
 const strokeDash = edgeTraversed ? [5, 5] : [2, 2];

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke="white" strokeDash={strokeDash} />
);
 })}
 </g>
 </svg>
 </div>
 </div>
 </div>
 </div>
);

```

```

 <circle cx={n.r}
 className="stroke"
 r={n.r}
 stroke="black"
 stroke-width={n.w}
 style={n.s}
 data-kind="parent" data-rs="2"/>
 </g>
)
 }}}}
 </svg>
 </div>

 {/* Structure View (Stack / Queue) */}
 <div className="bg-slate-900 border border-
r">
 <div className={`absolute -top-3 left-
 ${mode === 'dfs' ? 'border-
ld-300 border border-emerald-500'}`} >
 {mode === 'dfs' ? 'Стек вызовов' : ''}
 </div>

 <div className="flex flex-col gap-2 w-
 {step.queueOrStack && step.queueOrStack.length > 0}
 <div key={idx} className={`w-
 ${mode === 'dfs' ? 'bg-slate-900' : 'bg-white'}
 text-emerald-200`} >
 ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
 ? `0_10px_rgba(245,158,11,0.2)}` : ''}
 <div>
 Узел: {item}
 {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
 ? `<span className="ml-2"> ${item.value}</span>` : ''}
 }
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

---

ФАЙЛ 46/111: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../utils';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
```



```

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болят сердце', emoji: '👤', priority: 1, animState: 'idle' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤', priority: 2, animState: 'idle' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤', priority: 3, animState: 'idle' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈', priority: 4, animState: 'idle' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤', priority: 5, animState: 'idle' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤', priority: 6, animState: 'idle' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animState: 'idle' });
 });
 return arr;
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();
 const liveHeap = vizArray(runtime?.variables?.h) ?? vizArray([]);
 const liveI = vizNumber(runtime?.variables?.i);
 const visualHeap: Patient[] = liveHeap
 ? liveHeap.map((value, index) => {
 const tuple = vizArray(value);
 const priority = vizNumber(tuple?.[0]) ?? vizNumber(0);
 return {
 id: `python-${index}`,
 name: tuple ? String(tuple[1] ?? value) : String(index),
 emoji: " ",
 symptom: `индекс ${index}`,
 priority,
 animState: "idle",
 };
 })
 : heap;
 const [customName, setCustomName] = useState('');
```

```

 addLog(` Поступил пациент: ${finalName} с тяжестью

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() *
 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.priority));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name}

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x,

```

```

const res: React.ReactElement[] = [];
const p = nodePos(idx);
const l = 2 * idx + 1;
const r = 2 * idx + 2;
if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
}
if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
}
return res;
});

return (
 <div className="flex flex-col gap-6">
 /* МНЕМОНИКА / ПРАВИЛО */
 <div className="bg-emerald-950/40 border border-e
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 te
 ty Queue)</h3>
 <p className="text-xs text-slate-300 mt-1 lea
 В реанимации не важно, кто пришел раньше, а
 остояние (максимальный приоритет).

```

```

 onChange={e => setCustomPriority(Number(e.target.value), {
 className="w-20 accent-emerald-500 cursor-pointer"
 })}
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer"
 >
 Вести
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
<div className="grid grid-cols-1 lg:grid-cols-12 gap-4">

 {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}

```

```

 { /* Tooltip */ }
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

{ /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
<div className="lg:col-span-4 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-
 {healingPatient ? (
 <div className="flex flex-col items-cente
 <div className="relative">
 <span className="text-6xl block anim-l
 <span className="absolute -top-4 -right
 <span className="absolute -bottom-2 -
 </div>
 <div>
 <h4 className="text-white font-black
 Пациент: {healingPatient.name
 </h4>
 <p className="text-emerald-400 text-x
 Состояние: Стабилизация ({healingPa
 </p>

```

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — мож
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
 const [step, setStep] = useState(0);
```

```
 const paused = useContext(DemoPauseContext);
```

```
 const period = Math.round(ms * SPEED);
```

```
 useEffect(() => {
```

```
 if (steps <= 1 || paused) return;
```

```
 const t = setInterval(() => setStep((s) => (s + 1) %
```

```
 return () => clearInterval(t);
```

```
 }, [steps, period, paused]);
```

```
 return step;
```

```
}
```

```

stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

ФАЙЛ 48/111: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React from "react";
```

```

};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"]],
 ["D", "E"]],
 ["F", "G"], ["G", "H"], ["F", "H"]],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
))
)}
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={
 }}}
 </Svg>
);
};

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 40],
 const edges: [string, string, number][] = [["A", "B",

```



/\*\* Сжатие координат: разрежённые числа → плотные индексы

```
export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={35} />
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={20} />
 <text x={53 + i * 52} y={77} textAnchor="middle" />
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
```

```

 />
)})
 {Object.entries(f.pos).map(([id, [x, y]]) => {
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="middle">
 {id}
 </text>
 </g>
 })}
 </Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: p → x"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "p", "x": "x"}]},
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "p", "p": "x"}]}
]}
 />
};

/** Zig-Zig – x и p с одной стороны, крутим сначала верхнюю. */
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В: g → p → x"]}
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }, edges: [{"g": "g", "p": "p", "x": "x"}]},
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }, edges: [{"p": "p", "x": "x", "g": "g"}]},
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }, edges: [{"x": "p", "p": "g", "g": "x"}]}
]}
 />
};

/** Zig-Zag – x и p с разных сторон, крутим сначала нижнюю. */

```

```

 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}}
 </Svg>
);
 };
};

```

```

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k ===
 }}}
 </Svg>
);
};

```

```

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["носки", "боты"],
 ["штаны", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, {

```

```

 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color="black" />))}
 </Svg>
);
};

```

14

```
export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
)))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={step
)))}
 </Svg>
);
};
```

ФАЙЛ 50/111: `src/components/hints/demosStruct.tsx`  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68],
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
```

```

 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={18} fill={C.dim} />
 <text x={53 + i * 54} y={71} textAnchor="middle" />
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" />
 {p.map((par, i) => (
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]} Z` fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
))}
 {pos.map(([x, y], i) => (<Node key={i} x={x} y={y} />))}
 </Svg>
);
};

const TRIE_NODES = [

```

```

 });
 });
};

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3, 0]];
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться, чтобы

 {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
 })}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x + b.x) / 2} ${(a.y + b.y) / 2} ${b.x} ${b.y}`
 fill="none" stroke={C.warn} strokeWidth={1.5} />
);
 })}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={C.warn} />
))}
 </Svg>
);
};
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
};

```

```

 >
 </g>
)))
 </Svg>
);
};

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы дл
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={18} />
 <text x={25 + i * 30} y={29} textAnchor="middle">
 {i}
 </text>
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18} width={26} height={18} />
 <text key={i} x={25 + i * 30} y={52 + (i % 4) * 18} textAnchor="middle">
 {i}
 </text>
 })}
 </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18} />
 <text key={i} x={25 + i * 30} y={104 - c * 10} textAnchor="middle">
 {i}
 </text>
 })}
 </Svg>
);
};

```



```

 fill={idx <= i && step > 0 ? C.warn : C.idle}
 </g>
)))
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={10}
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : filled ? C.warn : C.idle}
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
 fontSize={10} fontWeight="700" fill={C.dim}>
 {π[idx]}
)}
 </g>
 </g>
);
 })
)}
 </Svg>
);
};

```

```

 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};

```

```

/** Список работающих мини-демо — его проверяет `npm run`
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

```

```

export const MiniDemo: React.FC<{ kind: DemoKind }> = ({
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);

```

```

const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end" >
 <div className="absolute inset-0 bg-slate-950/80" >
 <div className="relative w-full sm:max-w-5xl max-h-
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4" >
 <div className="min-w-0 flex-1" >
 <h3 className="font-bold text-white text-sm" >
 {viz.hint && <p className="text-[11px] text-white" >
 <VizLiveStatus demoTitle={viz.title} className="font-normal" />
 </p>
 </h3>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain" >
 {/* Тот же id главы: развёрнутая демонстрация */}
 <VizChapterContext.Provider value={vizId}>
 <Component />
 </VizChapterContext.Provider>
 </div>
 </div>
 </div>,
 document.body
 >
);
};

```

```

 let left = anchor.rect.left + anchor.rect.width / 2
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor

 setPos({ top: Math.max(8, Math.min(top, vh - height
 }, [anchor]));

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD W, typeof window !== "unde

```

```

 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 })
 </div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: 4, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 5, labelDx: 0, labelDy: 0 },
 { id: 'CS', u: 'C', v: 'S', weight: 0, labelDx: 0, labelDy: 0 },
];
}

```

```

 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
 };

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-slate-dim)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-slate-dim)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0})`;
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;

```

```

 Это наша база для расчета потенциалов
 </div>
);
case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгоритм
 для всех вершин:

 ...

 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый вес.
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра
 </div>
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.
 Старый вес:
 ...

 Вычисляем: $-2 + 0 - (-2) =$
 ...

 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра $B \rightarrow C$.
 Старый вес: 1.
 Вычисляем: $1 + (-2) - (-1) =$
 ...

 </div>
);
case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра $C \rightarrow A$.
 Старый вес: 2.
 Вычисляем: $2 + (-1) - 0 =$
 ...

 </div>
);

```





```

 </g>
 }}
 </g>
)
 }}}}
 </svg>
 </div>

 <div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size={24}
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bold text-white
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "В процессе"}
 {step !== MAX_STEP && <Skip
 </button>
 </div>

 {/* Progress Dots */}

```

```
const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];
```

```
const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];
```

```
interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}
```

```
export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
```

```

 null,
 false,
 -1,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 initialEdges.forEach((e) => adj[e.u].push(e.v));

 // DFS 1: post-order list
 const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
 u,
 false,
 -1,
);
 };

 // Run DFS1 on all unvisited
 for (let i = 0; i < 5; i++) {

```

```

 }
};

// Run DFS2 using the order (reversed from back to
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} комп
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

```

```

<div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

```

```

<div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

```

```

<button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>

```

```

</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12
 { /* Playfield SVG */ }
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]"
 { /* Markers for directed arrows */ }

```

```

const u = initialNodes.find((n) => n.id === edge.u)
const v = initialNodes.find((n) => n.id === edge.v)

// If transposed, draw reversed coordinates
const x1 = step.transposed ? v.x : u.x;
const y1 = step.transposed ? v.y : u.y;
const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 />

```

```

 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
 }}}
</svg>

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

```

```


 ЛОГ ОПЕРАЦИЙ:

<div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
 })}
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 № {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 px-2 py-1"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 px-2 py-1"
 >
 Вперед
 </button>
 </div>
</div>

```



```
interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
```

```

 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветные вершины D и E',
 type: 'info'
 }, ...prev]));
 },
 // Step 4: Include D-E
 () => {
 setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {color: 'blue'} : v),
 setEdges(prev => prev.map(e => e.id === '3-4' ? {color: 'blue'} : e),
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в остоле',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
 },
 // Step 5: Inspect B-C (4)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {color: 'red'} : e),
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и бесцветную вершину C',
 type: 'info'
 }, ...prev]));
 },
 // Step 6: Include B-C
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {color: 'red'} : v),
 setEdges(prev => prev.map(e => e.id === '1-2' ? {color: 'red'} : e),
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в остоле',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
 },
 // Step 7: Inspect A-D (5)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {color: 'red'} : e),

```

```

 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
 },
 // Step 12: Include E-F
 () => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
 },
 // Step 13: Inspect D-G (8)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
 },
 // Step 14: Include D-G
 () => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
 },
 // Step 15: Inspect C-F (9) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {

```

```

 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
},
// Step 22: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
},
]);

// --- PRIM STEPS ---
const primSteps = [

```

```

 description: 'Плесень расширяется в сторону верш
 type: 'info'
 }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к верши
 type: 'info'
 }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-
 type: 'success'
 }, ...prev]);
},
// Step 8: Pop B-C (4)

```

```

 },
 // Step 12: Pop E-F (7)
 () => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет',
 type: 'info'
 }, ...prev]));
 },
 // Step 13: Include E-F, add F's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]));
 },
 // Step 14: Pop D-G (8)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
 },
 // Step 15: Include D-G, add G's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{

```

```

 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)']);
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
 },
 // Step 20: Pop E-H (11) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 id: '4-7',
 weight: 11,
 type: 'cycle'
 }, ...e.edges)),
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]));
 },
 // Step 21: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 id: '4-7',
 weight: 11,
 type: 'cycle'
 }, ...e.edges)),
 setHeapQueue(['H-I (12)', 'F-I (13)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
 },
 // Step 22: Pop H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 id: '7-8',
 weight: 12,
 type: 'cycle'
 }, ...e.edges)),
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой вершине F.',
 type: 'info'
 }, ...prev]));
 },
 // Step 23: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 id: 8,
 weight: 12,
 type: 'included'
 }, ...v.vertices)),
 setLogs(prev => [{
 title: 'Шаг 13: Вершина H-I включена',
 description: 'Вершина H-I включена в плесень.',
 type: 'success'
 }, ...prev]));
 }
}

```

```

// Concurrently build chosen roads. Merge into:
// Component 1: {A, B, C} (colored red)
// Component 2: {D, E, F, G, H, I} (colored gold)
setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' }
));
setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
}));
setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
 type: 'success'
}, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}:
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 status: 'included', color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост',
 description: 'Теперь Красный и Синий колхозы одобрят мост A-D с весом 5. Остальные мосты B-E(6) и C-F(9) остаются в планах.',
 type: 'warning'
 }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {

```



```

 description: 'Представь, что все 9 вершин графа
 type: 'info',
 }];
}
if (algo === 'prim') {
 return [{
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем старто
 ь (Heap) и захватываем соседей!',
 type: 'info',
 }];
}
return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 дерев
 ы объединиться в колхоз.',
 type: 'info',
}];
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 setLogs(introLog(algo));
};

/* ————— связь с Python-компилятором —————
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

// активная вкладка = подрежим страницы: код в панели
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, activeAlgo);
}, [chapterId, activeAlgo]);

```

```

const nums = item.match(/-?\d+(?:\.\d+)?/g);
const letters = item.match(/[A-Za-z]/g);
let u: number | null = null;
let v: number | null = null;
if (letters && letters.length >= 2) { u = idOf(letters); }
else if (nums && nums.length >= 2) { u = idOf(nums); }
if (u === null || v === null) return null;
return { u, v, w: nums && nums.length ? Number(nums[0]) : null };
}
if (Array.isArray(item)) {
const letters = item.filter((x) => typeof x === 'string');
const nums = item.filter((x) => typeof x === 'number');
if (letters.length >= 2) {
const u = idOf(letters[0]); const v = idOf(letters[1]);
if (u === null || v === null) return null;
return { u, v, w: nums.length ? nums[0] : null };
}
if (nums.length >= 2) {
const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
if (u === null || v === null) return null;
return { u, v, w: nums.length > 2 ? nums[0] : null };
}
}
return null;
};

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → один цвет */
const colorOfComp = (root: string | undefined): string | undefined {
if (!root) return 'none';
let h = 0;
for (let i = 0; i < root.length; i++) h = (h * 31 + root.charCodeAt(i)) % PALETTE.length;
return PALETTE[h];
};

```

```

/** Всё, что код пользователя рассказал про граф. */
const live = useMemo(() => {
if (!vars) return null;

```

```

 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentId.get(cur);
 if (nxt === undefined || nxt === cur || seen.has(nxt)) continue;
 seen.add(nxt);
 cur = nxt;
 }
 compOfVertex.set(id, String.fromCharCode(65 + cur));
 });
 const takenRaw = vizArray(vars.mst) ?? vizArray(vars.heap);
 const takenEdges = (takenRaw ?? []).map(parseEdge);
 const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.heap))
 .map((x) => idOf(typeof x === 'string' ? x : vizNumber(x)))
 .filter((x): x is number => x !== null);
 const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
 const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
 const curW = vizNumber(vars.w);
 const heapRaw = vizArray(vars.heap) ?? vizArray(vars.mst);
 const heapEdges = (heapRaw ?? []).map(parseEdge);
 const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.mst);
 const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw)
 .filter((e): e is Edge => e !== null) : [];
 const total = vizNumber(vars.total) ?? vizNumber(vars.mst);
 const any = compOfVertex.size > 0 || takenEdges.length > 0 ||
 (curU !== null && curV !== null) || heapEdges.length > 0;
 return any ? { compOfVertex, takenEdges, takenVertices, total,
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [vars]};

```

/\*\* Рёбра, которые рисуем: код пользователя важнее ка

```

const effEdges: Edge[] = useMemo(() => {
 if (!live) return edges;
 const key = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;
 const takenKey = new Set(live.takenEdges.map((e) => key(e.u, e.v)));
 const heapKey = new Set(live.heapEdges.map((e) => key(e.u, e.v)));
 const cheapKey = new Set(live.cheapestEdges.map((e) => key(e.u, e.v)));
 return edges.map((e) => {
 const k = key(e.u, e.v);
 const isCur = (live.curU === e.u && live.curV === e.v) ||

```



```

 className="flex items-center justify-center gap-2 font-semibold text-xs rounded-xl border-200"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2 initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border-2"
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500"
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500"
 : "bg-rose-600 hover:bg-rose-500 text-white")
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex + 1}
 </button>
 </div>
 </div>

 /* Что компилятор сообщил графу прямо сейчас */
 <div className="mb-6 flex flex-wrap items-center" >

 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {live ? (
 <>
 {live.curU !== null && live.curV !== null}

 ребро {String.fromCharCode(65 + live.curU)} - {String.fromCharCode(65 + live.curV)}

 </>
) : null}
 </div>

```

```

 { ' ' }<code className="font-mono text-purple-300" style={{
 e className="font-mono text-purple-300">u</code>
 mono text-purple-300">w</code> перекрасят э

 }}
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-3" style={{
 /* Graph Canvas */
 <div className="lg:col-span-2 bg-slate-950 rounded-3xl relative min-h-[480px] overflow-hidden">

```

```

 /* Legend */
 <div className="absolute top-4 left-4 bg-slate-950 p-4 rounded-3xl text-center gap-3 text-xs">

```

```

 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">
 </div>
 {activeAlgo === 'kruskal' ? (
 <div

```

```

 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">
 </div>

```

```

 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">
 </div>

```

```

 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">
 </div>

```

```

 </>

```

```

) : activeAlgo === 'prim' ? (

```

```

 <div

```

```

 <div className="flex items-center gap-1.5">
 <div className="w-3 h-1 bg-sky-600">
 </div>

```

```

 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">
 </div>

```

```

 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {effVertices.map(vertex => (
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%",
 className={"absolute -translate-x-1/2 -translate-y-1/2"}
 font-weight: 'bold' text-base border-2 transition-all duration-300"}
 id)}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'path' ? ' (Source)' : ''}
 </div>
)

```

```

 <span key={idx} className={"px-2 py-2 shadow-md animate-pulse" : "bg-slate-800"}
 {item}

))
 })
 </div>
</div>
})

<div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all"
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500'
 : 'bg-slate-900/60 border-slate-700'}
 }
 >
 <div className="flex items-center gap-2"
 {log.type === 'success' && <CheckCircle className="text-emerald-500" />
 {log.type === 'warning' && <HelpCircle className="text-amber-500" />
 {log.type === 'error' && <XCircle className="text-rose-500" />
 {log.type === 'info' && <Play className="text-slate-500" />
 <h5 className="font-bold text-sm leading-tight"
 {log.message}
 </h5>
 </div>
 <p className="text-xs text-slate-300"
 {log.timestamp}
 </p>
 </div>
)}
 </div>
</div>
</div>

```



```

<div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Clock</p>
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Award</p>
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
</div>
<div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Award</p>
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Clock</p>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">

```

```
{`# 1. Инициализируем DSU`}
```

```
def find(i):
```

```
 if parent[i] == i: return i
```

```
 parent[i] = find(parent[i])
```

```
 return parent[i]
```

```
def union(i, j):
```

```
 r_i, r_j = find(i), find(j)
```

```
 if r_i != r_j:
```

```
 parent[r_i] = r_j
```

```

 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
180 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
 </pre>
 </div>
 </div>
 </div>
 </div>
}}

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-rose-400">
 </p>

```

```
 if bridge != -1 and union(bridge[0], bridge
 mst.append(bridge)`}
 </pre>
 </div>
</div>
</div>
 })
</div>

</div>
);
};
```

---

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
```

```
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
```

```
 * панели компилятора — только сам текст тем, чтобы:
```

```
 * 1) любую страницу можно было распечатать/сохранить
```

```
 * 2) внешний парсер (ридер, «версия для слабовидящих»
```

```
 * LLM-индексатор) получал чистый текст без служебных
```

```
 *
```

```
 * Все спойлеры <details> раскрыты, визуализация описания
```

```
 * показывает, какие у неё вкладки и какие переменные к
```

```

/** Раскрываем все спойлеры и вычищаем служебную разметку
function toPlainTextHtml(html: string): string {
 return html
 .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
 // кнопки/иконки внутри текста не нужны в текстовой версии
 .replace(/<button[\s\S]*?</button>/gi, "")
 .replace(/<svg[\s\S]*?</svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/\\sdata-term-id="[^\"]*" /g, "")
 .replace(/class="term-hint"[^\"]*" /g, "")
 // ссылки на другие билеты должны вести в текстовую версию
 .replace(/href="\\?topic=/g, 'href="?lite=1&topic=')
 .replace(/class="ticket-link"/g, "");
}

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

export const LiteGuide: React.FC<Props> = ({ topicId }) => {
 const list = useMemo(() => {
 if (!topicId) return chapters;
 const one = chapters.filter((c) => c.id === topicId);
 return one.length ? one : chapters;
 }, [topicId]);

 return (
 <div className="lite-wrap">
 <style dangerouslySetInnerHTML={{ __html: LIGHT_GUIDE_CSS }} />

 <h1>Универсальное пособие: алгоритмы и структуры данных</h1>
 <p className="lite-meta">
 Билеты 1–24 • текстовая версия (без навигации, без картинок)
 {topicId ? `одна тема: ${list[0]?.title ?? topicId}` : ""}
 полная интерактивная версия
 </p>
 </div>
);
}

```

```

 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {
 const sync = PAGE_SYNC[k];
 const demo = k.includes("#") ? k.split("#")[1] : k;
 return {
 <div key={k}>
 <p>
 {demo ? `Вкладка «${demo}»` : ''}
 {sync.vizTitle ? ` – ${sync.vizTitle}` : ''}
 {sync.stepNote ? <> Шаг демонстрации </> : ''}
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => {
 <li key={v.name}>
 <code>{v.name}</code> – {v.value}
 {v.range ? ` (${v.range})` : ''}

 })}

)}
 </div>
);
 })}
 <p className="lite-meta" style={{ margin: 0; padding: 0; }}>
 В интерактивной версии эта демонстрация переменных перерисовывают её напрямую
 </p>
</div>
)}

{
 ((quizzes[chapter.id] ?? []).length > 0 &&
 <div className="lite-quiz">
 <h3>Блиц: вопросы по теме</h3>

 {
 ((quizzes[chapter.id] ?? []).map((q, i) => {
 <li key={i}>
 <p>
 {q.question}
 {q.answer ? `<b>Ответ: ${q.answer}</b>` : ''}
 </p>
 }
))
 }

 </div>
)
}

```

```

done?: DebugValue[] | null;
/** Идентификаторы вершин, нарисованных на схеме демо
demoIds: string[];
/** Подпись таблицы (по умолчанию «Ваши расстояния»).
title?: string;
}

const fmt = (value: DebugValue | undefined): string =>
 if (value === undefined || value === null) return "—";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "−∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "−∞";
 return value;
 }
 return String(value);
};

/**
 * Таблица «ваших» расстояний, когда имена вершин в коде
 *
 * Схема демонстрации нарисована на фиксированном наборе
 * S...F у Форда–Беллмана). Если в коде пользователя верши
 * {0...6, v1...v9}, картинка физически не может их показать
 * просто пропадали, и казалось, что визуализация живёт
 * выводятся рядом, а подсказка объясняет, как связать
 */
export function LiveDistTable({ dist, prev, done, demoId
 if (!dist) return null;

 const keys = Object.keys(dist);
 if (keys.length === 0) return null;

```

```
 {parent && parent !== "-" ? <span className={
 {fixed ? <span className="text-emerald-400"

 };
 }}}
 {keys.length > shown.length ? (
 <span className="inline-flex items-center rounded-
 e-400">
 +{keys.length - shown.length} ещё

) : null}
 </div>
</div>
);
}
```

---

ФАЙЛ 59/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Фриказоид (Декстер Дуглас / Дейкстра)',
 title: '✂ Фриказоид (Декстер Дуглас)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'неотрицательными',
 highlightColor: 'text-emerald-400',
 rest: 'весами ($w \geq 0$). Подсунь отрицательное ребро -',
 complexity: ' $O((V+E) \log V)$ ',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
```





```
 <div className="absolute inset-y-0 left-0 w-[100%] h-[100%]
 te-[tocIn_.18s_ease-out]">
 <div className="flex items-center justify-between p-2">

 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 />
 <X className="w-5 h-5" />
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
)
</>
);
};
```

---

ФАЙЛ 61/111: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Sparkles, FileDown, FileText, Code2, Loader2, } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
```

```

 </div>
</div>

<div className="flex items-center flex-wrap sm:
late-700">
 <span className="hidden sm:flex items-center
shadow-indigo-600/30 whitespace-nowrap">
 <BookOpen className="w-4 h-4" /> Учебное по

 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 aria-label={compilerOpen ? "Скрыть панель P
 className={`shrink-0 flex items-center just
 duration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white bg-s
 }`}
 >
 <Terminal className="w-4 h-4" /> Python
 </button>

<div id="download-menu" className="flex items
 <a
 id="download-guide-pdf-btn"
 href={`${import.meta.env.BASE_URL}export/
 download="guide_lite.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex-1 sm:flex-none flex items
 bg-indigo-600 hover:bg-indigo-500 text-whit
 n-200 whitespace-nowrap"
 title="Скачать PDF пособия: компактный ко
 >

```

```
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center gap-2 w-
te hover:bg-amber-600/20 border border-amber
ointer"
 title="Скачать всё пособие одним авто
 >
 {busy ? <Loader2 className="w-3.5 h-3
 </button>
 </div>
 </details>
</div>
</div>
</div>
</header>
);
};
```

---

ФАЙЛ 62/111: src/components/PlanarColoringViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useCallback, useContext, useEffect, use
import { ChevronLeft, ChevronRight, Play, Pause, Rotate
import { PlanarityDemo } from "./PlanarityDemo";
import { VizChapterContext, emitVizDemo, useVizRuntime,

/**
 * Билет 7 – «Графы. Планарные. Покраска».
 *
 * Четыре вкладки:
 * 1. «Планарность» – K5 и K3,3 с подсвеченными пересеч
 * 2. «Жадная покраска» – $\chi \leq \Delta + 1$: вершины красятся
 * достаётся наименьший цвет, не занятый соседями;
 * 3. «Двудольность» – проверка 2-раскрашиваемости BFS
```

```

 g.adj = buildAdj(g);
 return g;
}
const edges: [number, number][] = [[0, 1], [0, 2], [1, 3], [1, 4], [2, 5], [2, 6], [3, 7], [4, 7], [5, 6], [6, 7]];
const pos = [
 { x: 70, y: 60 }, { x: 175, y: 40 }, { x: 150, y: 140 },
 { x: 250, y: 185 }, { x: 365, y: 110 }, { x: 350, y: 240 }
];
const g: Graph = { key, label: "Основной (8 вершин, Δ = 3)", "5", "6", "7" };
g.adj = buildAdj(g);
return g;
}

function buildAdj(g: Graph): number[][] {
 const adj: number[][] = g.edges.reduce((acc: number[][], [u, v]) => {
 acc[u].push(v);
 acc[v].push(u);
 return acc;
 }, Array.from({ length: g.n }, () => [] as number[]));
 adj.forEach(a => a.sort((x, y) => x - y));
 return adj;
}

interface Frame {
 title: string;
 note: string;
 color: number[]; // 0 = не покрашена
 ecolord?: number[]; // 0 = не покрашено
 v: number | null;
 u: number | null;
 e: number | null;
 used?: number[];
 bad?: [number, number] | null;
}

/** Жадная покраска вершин: $\chi \leq \Delta + 1$. */
function greedyFrames(g: Graph): Frame[] {

```

```

const queue: number[] = [];
let conflict: [number, number] | null = null;
frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",
 note: "BFS красит вершину в цвет, противоположный р
 двудольный ($\chi > 2$).",
 color: [...color], v: null, u: null, e: null,
});
for (let start = 0; start < g.n && !conflict; start++)
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.`,
 color: [...color], v: start, u: null, e: null,
 });
while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {
 if (color[u] === 0) {
 color[u] = 3 - color[v];
 queue.push(u);
 frames.push({
 title: `${g.names[v]} → ${g.names[u]}: цвет
 note: `Сосед не покрашен: color[${g.names[u]
 x}).join(", ")} || "пуста"`.`,
 color: [...color], v, u, e: null,
 });
 } else if (color[u] === color[v]) {
 conflict = [v, u];
 frames.push({
 title: `Конфликт: ${g.names[v]} и ${g.names
 note: `Ребро (${g.names[v]}, ${g.names[u]})
 покрасить: $\chi \geq 3$.`,
 color: [...color], v, u, e: null, bad: conf
 });
 }
 }
}

```

```

 title: `Ребро (${g.names[a]}, ${g.names[b]}): цве
 note: `В вершине ${g.names[a]} заняты цвета ${us
 y) => x - y).join(", ") || "—"}}. Наименьший св
 да.`
 color: new Array(g.n).fill(0), ecolord: [...ecolor
 });
});
const chiE = Math.max(...ecolor);
frames.push({
 title: `Готово:  $\chi' = \chiE$  при  $\Delta = \delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф: $\chi' = \Delta = \delta$ – это теор
 сс 1 по Визингу.`
 : ` $\chi' = \chiE$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$
 $\chi' = \Delta + 1$ – классом 2 (например, K3 и любой
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React
 { id: "planar", label: "1 • Планарность: K5 и K3,3",
 { id: "greedy", label: "2 • Покраска вершин ($\chi \leq \Delta + 1$)",
 { id: "bipartite", label: "3 • Двудольность (2 краски)",
 { id: "edges", label: "4 • Покраска рёбер (Визинг, Кё
];

```

```

export const PlanarColoringViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("greedy");
 const [graphKey, setGraphKey] = useState<GraphKey>("m
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);

 useEffect(() => {

```

```

const liveColor = vars ? vizArray(vars.color) ?? vizA
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vi
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumb
const linked = Boolean(liveColor || liveEcolor || (li

const frame = frames[Math.min(step, maxStep)] ?? ({ t
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number"
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV)
const curU = liveU !== null && Number.isFinite(liveU)
const curE = liveE !== null && Number.isFinite(liveE)
const delta = Math.max(...graph.adj.map((a) => a.leng
const usedColors = Array.from(new Set(graph.pos.map((
const usedEcolors = Array.from(new Set(graph.edges.ma

const go = useCallback((d: number) => {
 setPlaying(false);
 setStep((s) => Math.max(0, Math.min(maxStep, s + d)
}, [maxStep]);

if (tab === "planar") {
 return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded
 <Header />
 <Tabs tab={tab} setTab={setTab} />
 <PlanarityDemo />
 <p className="text-[11px] text-slate-500 mt-2">
 Формула Эйлера $V - E + F = 2$ и оценка $E \leq 3V$
 </p>

```

```

 {tab === "edges"
 ? <span className="bg-slate-950 border border-
rs}
 : <span className="bg-slate-950 border border
 }}

</div>

<div className="grid grid-cols-1 lg:grid-cols-5 g
 <div className="lg:col-span-3 bg-slate-950 round
 <svg viewBox="0 0 480 300" className="w-full l
 {graph.edges.map(([a, b], i) => {
 const ec = edgeColor(i);
 const isCur = i === curE;
 const isBad = Boolean(frame.bad && ((fram
)))
 return (
 <g key={i}>
 <line
 x1={graph.pos[a].x} y1={graph.pos[a
 stroke={isBad ? "#f43f5e" : ec > 0
 strokeWidth={isCur ? 6 : isBad ? 5
 strokeDasharray={ec === 0 && !isBad
 className="transition-all duration-
 opacity={isCur || isBad ? 1 : 0.85}
 />
 {tab === "edges" && ec > 0 && (
 <text x={{graph.pos[a].x + graph.po
 ={{colorOf(ec)}} fontSize="10" fontWeight="bo
 }}
 </g>
);
 })}
 {graph.pos.map((p, id) => {
 const c = nodeColor(id);
 const isCur = id === curV;
 const isNbr = id === curU;
 return (

```



```

 {tab === "edges" && <>Рёберная покраска:
 Δ (Кёниг). Задача « $\chi' = \Delta?$ » NP-полна даже ,
 </div>
 </div>
 </div>

 <div className="mt-3 flex flex-wrap items-center" >
 <span className="text-[10px] uppercase tracking-
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
) : (

 открой панель Python: массив <code classNam
 igo-300">ecolor</code> для рёбер) и вершина

)}
 </div>
 </div>
);
};

```

```

const Header: React.FC = () => {
 <div className="flex flex-wrap items-center justify-b
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-fuchsia-600 text-white r
 <Palette className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold tex
 <p className="text-xs text-fuchsia-300">K5/K3,3

```

```

00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
<svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f[id]; }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 if (!crosses.includes(i)) lines.push([p1.x,p1.y,p2.x,p2.y]);
 else circles.push([p1.x,p1.y,p2.x,p2.y]);
 }
 })}

```

```

const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d)
if (!p1||!p2||!p3||!p4) return false;
function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number)
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax)
}
return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)>0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)>0
}
const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i)
 if (!crosses.includes(j)) crosses.push(j)
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width=2 />);
}
const labels = ["\u04141","\u04142","\u04143","\u04144"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
}
return <>{lines}{circles}</>;
})();
</svg>
<div className="absolute bottom-2 left-2 right-2 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

```

 Terminal,
 Unlink,
 Variable,
 X,
 } from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохран
 width?: number;
 height?: number | null;
 onWidthChange?: (w: number) => void;
 onHeightChange?: (h: number) => void;
 /** Перейти к странице пособия (посмотреть визуализац
 /** Кнопка «открыть страницу в пособии» – не нужна, е
 onOpenGuide?: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

```

Универсальное пособие • полный исходный код

```

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но
 * напечатанной строки: print("x" * 10_000_000) или print(
 * – это один шаг, после которого весь текст уезжал в <
 * Обрезаем по границе строки и честно пишем, сколько з
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
 if (text.length <= OUTPUT_CHAR_CAP) return text;
 const cut = text.slice(0, OUTPUT_CHAR_CAP);
 const lastBreak = cut.lastIndexOf("\n");
 const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.sl
 return `${head}\n... вывод обрезан: показаны первые ${h
}

/** -----
/** Шаблон синхронизации: короткая шапка + минимальный
/** скелет, шаг-в-шаг с подсветкой визуализации (см. da
/** -----

/** -----
/** Шпаргалка Python: команды сортировки, циклы, структ
/** «Вставить» кладёт сниппет в редактор под курсор.
/** -----

interface Snippet {
 label: string;
 tip: string;
 code: string;
}
```

```

 },
 {
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = pop",
 code: 'st = []\nst.append("A") # push\nst.pop() # pop',
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n).",
 code: 'from collections import deque\nq = deque()\nq.append(1)\nq.popleft()',
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: 'import heapq\nh = []\nheapq.heappush(h, (1, "A"))\nheapq.heappop(h)',
 },
 {
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечность float('inf')",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m for _ in range(n)]',
 },
 {
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)',
 },
],
 },
 {
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 },
],
 },

```

```

const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 if (saved.code === saved.base || saved.code.trim()
 return saved.code;
});
/** «Глаз» – переключатель: эталонный код демо ↔ иници
const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится .
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);
/**
 * Панель жива? Заккрытие (крестик, «глаз» в шапке, см
 * компонент, но `runPythonAsync` к тому моменту може
 * этого флага доехавший до конца прогон опубликовал с
 * после закрытия: панель исчезла, а визуализация про
 * кода, которого на экране больше нет.
 */
const aliveRef = useRef(true);
useEffect(() => {
 aliveRef.current = true;
 return () => {
 aliveRef.current = false;
 };
}, []);

```

```

 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старого
 setRefOn(false);
 if (code === prev.base || code === prev.full || code === prev.template) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [initTemplate, template]);

```

```

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template };
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

```

```

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать — буфер переполнен");
 }
}, [code]);

```

/\* — Пошаговый отладчик —————

```

/** Связь больше не зависит от эталонных строк: совпадают
const hasViz = sync.variables.length > 0;

```



```

 }, []));

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!aliveRef.current) return; // панель закрыта
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d

```

```

 setOutputOpen(false);

// Пустой редактор: Pyodide не гоняем, панель в реж
// связь с визуализацией снимаем — демо показывает
if (source.trim() === "") {
 setDebug({ result: { steps: [], error: "", trunca
 setOutput("В редакторе пусто — выполнять нечего.\n
 «глаз».");
 setRuntimeMessage("Пустой код: связь с визуализаци
 setOutputOpen(true);
 setRunning(false);
 return;
}

setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.p
 runtime.setStderr({ batched: (message) => stderr.p
 runtime.setStdin({ stdin: () => (inputLines.length

 const raw = await runtime.runPythonAsync(buildDeb
 // Панель могли закрыть, пока Python считал: выхо
 if (!aliveRef.current) return;
 const result = JSON.parse(String(raw)) as DebugRe
 const printed = clipOutput([...stdout, ...stderr]

 if (result.steps.length === 0) {
 // Шагов нет (только комментарии, синтаксическа
 // заменяется пустым листингом, а визуализация
 // показывает собственный встроенный сценарий {
 setDebug({ result, idx: 0, src: source, input,

```

```

 setRunning(false);
 }
}, [chapterId, code, stdin, traceRequest, running, run

// Код выполняется сам: debounce даёт спокойно допечать
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не ст
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) retu
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(-1);
 }
 if (e.key === " ") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
 if (e.key === "Escape") stopDebug();
 }

```

```

/** Одна и та же кнопка запуска всегда видна: из редактора
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) stepToNext();
 setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

```

```

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
 const start = ta.selectionStart ?? current.length;
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
 const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + padded.length;
 });
 return next;
 });
}, []);

```

```

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
 -[520px] 2xl:w-[600px] flex flex-col bg-slate-900"
 >

```

```

 { /* Левый край меняет ширину на десктопе. */ }
 { onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину п
 title="Потяните, чтобы изменить ширину панели
 className="hidden lg:block absolute top-0 bot
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 })
 >
 <div className="absolute left-1/2 top-1/2 -tr
 digo-500 transition-colors" />
 </div>
)}
 { /* — Заголовок панели
 <div className="shrink-0 flex flex-wrap items-cen
 <span className={`w-2 h-2 rounded-full shrink-0
 <Terminal className="w-4 h-4 text-emerald-400 s
 <span className="text-[13px] font-bold text-whi
 <span className="hidden sm:inline text-[10px] t

 <div className="ml-auto flex flex-wrap items-ce

```

```

 >
 <RotateCcw className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip
 side="bottom"
 content={
 refOn
 ? "Убрать эталонный код: редактор стане
обственном встроенному демо. Кнопка ↶ верн
 : "Вставить полный эталонный код этого
удет заменён."
 }
>
 <button
 type="button"
 onClick={() => {
 const next = !refOn;
 setRefOn(next);
 // «Убрать эталон» – значит убрать код
 // выполняется, поэтому демонстрация бо
 // значений и честно показывает встроенн
 // странице скажет об этом явно).
 setCode(next ? template : "");
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 }}
 >

```

```

<div className="shrink-0 border-b border-slate-800"
 <div className="flex items-center gap-1 px-2 pt-2"
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-white" : ""
 }`
 >
 <Variable className="w-3.5 h-3.5" /> Переменная
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text-white" : ""
 }`
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказка
 </button>
 <div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реальный узел"
 : "На выбранной странице нет интерактивных элементов"
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1
 hasViz ? "text-emerald-400" : "text-slate-400"
 }`
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визуализации"}

 </div>
 </div>
</div>

```

```

 <Unlink className="w-3.5 h-3.5 shrink-0">
 Страница сменилась – подставить её пер
 </button>
 </Tooltip>
 })
 {hasViz && {
 <div className="flex flex-wrap items-center">
 {sync.variables.map((v) => {
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald
 >
 {v.name}
 </code>
 </Tooltip>
 })}
 {sync.stepNote && {
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-fle
text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3

```



```

<div className="shrink-0 flex items-center gap-1"
 <Pencil className="w-3.5 h-3.5 text-emerald-400"
 <span className="text-[10px] font-bold uppercase"
 Редактор • выполняется этот текст

 {state && (
 <span
 className="inline-flex items-center gap-1
 font-bold text-amber-300 shrink-0"
 title="Трасса ниже построена по предыдущему"
 >
 <Loader2 className="w-2.5 h-2.5 animate-spin"

)}
 <span className="ml-auto text-[10px] text-slate-500"
 {code.trim() === "" ? "пусто – ничего не выведено" : ""}

 </div>

```

```

<textarea
 ref={editorRef}
 value={code}
 onChange={(event) => {
 setEditHold(null);
 setRefOn(false);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
 event.preventDefault();
 void debugRun();
 }
 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-[2] min-h-[110px] w-full resize-none
 line-none focus:ring-2 focus:ring-inset focus:ring-amber-300"

```

```

e hover:bg-slate-800 disabled:opacity-40 transition
 <ChevronRight className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="В конец трассы">
 <button type="button" aria-label="В конец трассы"
 result.steps.length - 1); }} disabled={!currentStep}
 <span text-slate-400 hover:text-white hover:bg-slate-800
 <SkipForward className="w-4 h-4" />
 </button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-slate-400"
 {curDebug?.step ? (

 шаг {debug.idx + 1}/{debug.result.steps.length}
 {curDebug.step.func !== "<module>" ? " " : ""}

) : (
 "шагов нет — исправьте ошибку и начните заново"
)}

{debug.result.truncated && (
 <Tooltip content={`Выполнение остановлено: ${debug.result.truncated}
 <span tabIndex={0} className="cursor-help rounded-full px-1.5 py-0.5"
 ЛИМИТ

 </Tooltip>
)}
<Tooltip content={hasViz ? "Визуализация: старт... Эталонный текст не обязателен — пишите что хотите, но не
нет имён, связанных с демонстрацией; Python не нужен" : ""}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1 border-emerald-500/40 bg-emerald-500/10 padding-2
 >
 {hasViz ? <Link2 className="w-3 h-3" href="#">

```

```

const ln = i + 1;
const active = ln === curDebug?.step?.line;
return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-1
 active ? "bg-emerald-500/15 text-emerald-300" : ""
 }`
 >
 <span
 className={`w-7 shrink-0 select-none
 active ? "text-emerald-300" : "text-emerald-500"
 }`
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
}}
</div>

```

```

/* панель переменных — «Variables», как в VS Code
<div className="shrink-0 max-h-[34%] overflow-hidden"
 <div className="flex items-center gap-1.5"
 <Variable className="w-3 h-3 text-indigo-500"
 Переменные на этом шаге
 <span className="normal-case tracking-normal"
 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap"
)}
 {debugOrdered.length === 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500"
 </div>
)}
 <div className="mt-1 flex flex-wrap gap-1

```

```

 <input
 id="python-stdin"
 value={stdin}
 onChange={({event}) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono"
 />
 </div>
 </div>

 /* — Вывод —————
 <div className="shrink-0 border-t border-slate-800"
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between gap-2"
 >
 <span className="inline-flex items-center gap-2"
 <Terminal className="w-3.5 h-3.5 text-emerald-500" />

 <span className="inline-flex items-center gap-2"
 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />}
 {outputOpen ? "Закрыть" : "Открыть"}

 </button>
 {outputOpen && (
 <pre
 data-testid="compiler-output"
 className="flex-1 min-h-[70px] overflow-auto text-slate-300"
 >
 {output}
 </pre>
)}
 </div>
)}
 </div>
</div>

```

```

 тартапы.' }},
];

 let counter = 3;

 export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'red', bubbleText: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍷', color: 'blue', bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green', bubbleText: 'без сметаны, плиз.', animState: 'idle' },
]));

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: "🐍",
 color: "from-emerald-500/20 to-emerald-600/20",
 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

 // ENQUEUE: Встать в конец очереди (хвост / Tail)
 const enqueue = useCallback(() => {

```

```

addLog(` DEQUEUE: Повар наливает суп первому – ${headGuest.name}

// Step 1: Head guest gets eating animation
setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c, animState: 'in' }, ...prev.slice(1)));

setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue sh...
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c, animState: 'out' }, ...prev.slice(1)));

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`  ${headGuest.name} поел и ушел сытым.`, 400);
 }, 900);
}, [queue, isServing]);

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', col: 1, animState: 'idle', animSpeed: 1000, animDelay: 0, animText: 'Я голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'blue', col: 2, animState: 'idle', animSpeed: 1000, animDelay: 0, animText: 'Напишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐱', color: 'green', col: 3, animState: 'idle', animSpeed: 1000, animDelay: 0, animText: 'Дайте котика без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'idle' })), 500);
 });
};

return {

```

```

 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
 <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && {liveQ.length}}
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 <div className="flex items-end justify-between" >

 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
 <div className="flex flex-col items-center" >
 <div className="relative" >
 <div className="absolute -top-10 left-1/2 -translate-x-1/2" >
 <div className="w-1.5 h-6 bg-slate-800 rounded" >
 <div className="w-2 h-4 bg-slate-400 rounded" >
 </div>
 {liveQ.length}
 </div>
 <div className="text-center" >
 {liveQ.length}

 ПОВАР

 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```





```
 <div className="text-[10px] font-mono text-slate-500">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`}
 >
 {idx === 0 ?
 {entry}
 </div>
)}}
 </div>
 </div>
);
 };
```

---

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect, useContext, useMemo } from 'react';
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizTheme } from 'viz';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}
```

```

const parsed = (JSON.parse(liveWordsSig) as unknown
 .map((w) => (typeof w === "string" ? w : typeof w
 .filter((w) => Boolean(w))
 .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g,
 .filter((w) => w.length > 0)
 .slice(0, 8);
if (!parsed.length) return;
if (parsed.join("|") === words.join("|")) return;
setWords(parsed);
buildInitialTrie(parsed);
// eslint-disable-next-line react-hooks/exhaustive-
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[l
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail
 setSimulationStep(liveStep <= 0 ? 0 : failReady ?
 }
 if (liveQueue) {
 const q = liveQueue.map((x) => (typeof x === "num
 efined);
 if (q.length) setBfsQueue(q);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
}, [vars]);

const compilerLinked = Boolean(liveWordsSig || liveSta

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);

```

```

 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (Б

```

```

 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на гл
 очке суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полны
 ь полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.ch

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== null) {
 v = updatedNodes[v].fail;
 }
 }
};

```

```

 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>

 {/* Верхняя панель: Говорящий лосось и диалоговое
 <div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45"
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 40 85"
 fill="#be123c"
 className="origin-[35px_100px] animate-
 />

 {/* Плавники */}

```

```
</div>
```

```
{/* Пузырь с текстом (Баббл) */}
```

```
<div className="md:col-span-2 bg-slate-800 border-2 border-t-transparent min-h-[140px]">
```

```
 {/* Треугольник баббла */}
```

```
 <div className="absolute -left-3 top-1/2 -transform rotate(-45deg) bg-slate-800 border-b-[12px] border-b-transparent" />
```

```
 <div className="flex items-center space-x-2 text-white">
```

```
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
```

```
 Прямое вещание из аквариума:
```

```
 </div>
```

```
<p className="text-slate-200 text-base leading-relaxed">{speechText}</p>
```

```
</div>
```

```
<div className="mt-4 pt-3 border-t border-slate-700">
```

```
 <div className="text-xs text-slate-400 flex justify-between">
```

```
 <Info className="w-3.5 h-3.5 text-blue-400" />
```

```
 Статус: {simulationStep === 0 ? 'Борется с вирусом' : 'Восстановлен!'}</div>
```

```
 </div>
```

```
<div className="flex gap-2">
```

```
 {simulationStep === 0 && (
```

```
 <button
```

```
 onClick={startBFS}
```

```
 className="bg-emerald-600 hover:bg-emerald-500" />
```

```
 <button className="bg-slate-700 hover:bg-slate-600 shadow-lg shadow-emerald-900/40 transition" />
```

```
)
```

```
 <Play className="w-4 h-4" /> Запустить симуляцию
```

```
 </button>
```

```
 </div>
```

```
 {simulationStep === 1 && (
```

```
 <button
```

```
 onClick={stepBFS}
```

```

<svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#c0c0c0'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
)
 })}

```

```

let fill = '#1e293b';
let stroke = '#475569';

if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
} else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
} else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
}

return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}
 y={node.y - 12}

```



```

 </div>
 </div>

 <form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>
 <tr className="border-b border-slate-700" >

```

```

 {node.id === 0 &&
 </td>
 <td className="py-2.5 px-3">

 {node.char}

 </td>
 <td className="py-2.5 px-3 text-slate-500">
 {node.id === 0 ? '-' : `#${node.id}`}
 </td>
 <td className="py-2.5 px-3">
 {node.id === 0 ? (

 {node.term}

) : (

 #{node.fail}

)}
 </td>
 <td className="py-2.5 px-3">
 {node.id === 0 || node.term_link ? (

 {node.term}

) : (

 {node.term}

)}
 </td>
 <td className="py-2.5 px-3 text-slate-500">
 {Object.keys(node.children).length === 0
 ? 'Ø'
 : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (

```

```


 })
 </div>
 </div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizStore';

```

```

// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index to value
 highlightNodes: number[]; // current highlighted nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range in array
 result?: number; // partial result
}

```

```

// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 }
 go(1, 0, n - 1);
 return tree;
}

```

```

 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}
 { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build(${v}
 lightNodes: [v], arrayHighlight: [l, r] }));
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 tate: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: ` Дерево построено! Корень
 Highlight: [0, n - 1] }));
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 4, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 return left + right;
 }
}

```

```

 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`,
 , treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}`,
 , arrayHighlight: [qL, qR], result: res });
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
 , highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }
```

```
function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m),
 right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 "}"
]

```

```

 return () => clearTimeout(t);
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
 border-indigo-500 bg-indigo-600', childRing: 'ring-
 -900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
 30 border-emerald-500 bg-emerald-600', childRing: '
 -slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
 der-amber-500 bg-amber-600', childRing: 'ring-rose-
 [color];

 return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
 | 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree
 const renderNode = useCallback((nd: VNode): React.Rea
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate
 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (

```







```


 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">

 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 </div>
 <div className="text-[10px] text-slate-500">
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((sum, val) => sum + val, 0))\}$
 </div>
 </div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
 <button onClick={() => setView('build')} className="text-sm font-medium transition-colors duration-200" >
 Построить
 </button>

```



```

 <p className="text-sm text-slate-300 leading-4">
 <strong className="text-white">Итеративный способ проверки
 корректности. Но если нужны <strong className="text-white">рекурсивный (сверху-вниз)
 </p>
 </div>
 </div>
)}
</div>
);
}

```

ФАЙЛ 68/111: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters, SECTIONS, sectionOf } from "../data/chapters";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
 /** Узкий режим: номер билета и название в одну строку */
 compact?: boolean;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate, compact }) => {
 const [query, setQuery] = useState("");

 /**
 * Группы – это разделы пособия (SECTIONS), а не случайные темы.
 * связанные темы стоят рядом, порядок разделов не важен.
 */

```

```

 >
 <X className="w-3.5 h-3.5" />
 </button>
 })
 </div>
})
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4" >
 {groups.length === 0 && <p className="text-sm text-slate-500">
 Нет данных
 </p>}

 {groups.map(({ section, items }) => {
 <div key={section.id}>
 <div className="flex items-baseline gap-1.5">

 {section.from > 0 && {

 {section.from}

 }}

 </div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 // «18–20. Графы. Основное дерево...» → 3
 const num = chapter.title.match(/^(\d+)/)
 const label = num ? chapter.title.slice(0, num.length) : chapter.title
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 title={chapter.title}
 aria-current={isSelected ? "page" : undefined}
 className={`w-full text-left rounded-md px-2 py-1.5`} ${

```

```
 номерам билетов 1 → 24, каждая опирается на предыдущую.
 Следующая тема» продолжают этот же порядок.
 </p>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по клику.
 </p>
 <p className="text-slate-400 italic">Стрелки ← и → указывают на
 </div>
</aside>
);
};
```

---

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';

interface Plate {
 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;
 name: string;
 icon: string;
}

interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}
```

```

 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} ${newPlate.icon} `);
 };

 const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} ${topPlate.icon} `);
 };

 const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
 };

 // Queue handlers
 const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🍲' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🧑‍💻' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setAddMessage(` Добавили в очередь: ${newPerson.name} ${newPerson.icon} `);
 };

 const removePerson = () => {
 if (queue.length === 0) {
 setAddMessage("🚫 Очередь пуста!");
 return;
 }
 const frontPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setRemoveMessage(` Удален из очереди: ${frontPerson.name} ${frontPerson.icon} `);
 };

 const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Любитель борща', icon: '🍲' },
 { id: '2', name: 'Курьер доставки', icon: '📦' },
 { id: '3', name: 'Усталый админ', icon: '🧑‍💻' },
 { id: '4', name: 'Турист с рюкзаком', icon: '🎒' },
]);
 setAddMessage(" Очередь восстановлена.");
 };

```

```

 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
 `);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 `);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability > a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">

```

```

>
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'fifo' ? 'bg-indigo-500' : 'bg-slate-800'} rounded-x`}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </div>
</button>

<button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between gap-3 p-5 rounded-x
 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-white'}
 `}
>
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'bg-emerald-500' : 'bg-slate-800'} rounded-x`}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">
 <div className="text-xs opacity-80">Приоритетная
 </div>
 </div>
 </div>

 Beam Search

 </div>
</button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x">

```



```

 imate-fade-in">

 </div>
)}

 { /* Visualization */ }
 <div className="bg-slate-950/60 p-6 rounded-2xl" style={popMessage} >
 >
 {stack.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Сторона
 <p className="text-xs mt-1">Добавьте граф
 </div>
) : (
 <div className="w-full max-w-sm flex flex-direction-column align-items-center">
 <div className="absolute top-0 text-xs text-slate-400">
 full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 animate-pulse" />
 </div>

 { /* Reversed map so top of stack is visible */ }
 {stack.slice().reverse().map((plate, index) => {
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-center justify-between p-2
 ${isTop
 ? 'bg-gradient-to-r from-blue-500 to-blue-400'
 : 'bg-slate-900/90 border-slate-800'}
 `}
 >
 <div className="flex items-center" style={plate} >
 {plate.name}
 </div>
 </div>
)
 })}
)}
 </div>

```

```

 onClick={addPerson}
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

{dequeueMessage && {
 <div className="bg-indigo-950/60 border bor
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 } : {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 }}

```

```

 isFirst ? 'bg-indigo-500/30 text-white' : 'bg-white text-indigo-500'
 }`}>
 {isFirst ? 'Получает сун' : `Взрывает`}

</div>
});
}}

<div className="flex flex-col items-center justify-center gap-3 w-full h-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ НЕ ПОЛУЧИЛ?
</div>
</div>
)}

{ /* HEAP TAB */
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-xl">
 <div>
 <h3 className="text-lg font-bold text-white">
 Приоритетная очередь / Куча

 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Здесь не важно, кто пришел первым. Важно, кто пришел вторым.
 </p>
 </div>
 <div className="flex items-center gap-3 w-full">

```

```

min="0.01"
max="0.99"
step="0.01"
value={newCustomProb}
onChange={e => setNewCustomProb(Number(e.target.value))}
className="w-full accent-emerald-500 cursor-pointer"
/>
</div>
<div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 text-sm font-medium text-white border border-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в избранное
 </button>
</div>
</form>

```

```
{heapMessage && {
 <div className="bg-emerald-950/60 border bo
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {heapMessage}
 </div>
 }}
}
```

```

{ /* Visualization */ }
<div className="bg-slate-950/60 p-6 rounded-2
>
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
) : (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
)}

```

```
<div className="flex items-center justify-between gap-8">
 </>
 {progress} %
 </div>

 <div className="mt-6 text-xs text-slate-500">
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
 </div>

```

```

 {[
 { id: "cases" as Tab, label: "1 · Механика: Z" },
 { id: "big" as Tab, label: "2 · Большое дерево" },
 { id: "play" as Tab, label: "3 · Собери сам" },
]}.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 ${tab === t.id ? "bg-indigo-600 text-white"}
 `}
 >
 {t.label}
 </button>
)})
 </div>

 {tab === "cases" && <SplayRotationsViz />}
 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с правым
 </p>
 }}
</div>
);
};

export default SplayPlayground;

```

```

 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf
 const n4: TNode = { key: 4, left: leaf(3), right:
 return { key: 8, left: n4, right: leaf(9) };
 },
},
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number) => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
}

```

```

 if (xLeft === pLeft) return { node: p.key, kind: "Zig"
 return { node: x.key, kind: "Zig-Zag (сначала нижняя
}

```

```

/* ----- раскладка -----

```

```

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

```

```

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

```

```

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

```

```

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
 const colW = 46;
 const rowH = 62;
 return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 };
}

```



```

 setMoves((m) => [...m, { node: key, correct: want?.correct }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
 });

```

```

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

```

```

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build()))

```

```

return (
 <div className="w-full bg-slate-950 rounded-2xl border">
 <div className="flex items-start justify-between">
 <div>
 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 leading-4">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к

 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1 text-sm font-medium text-slate-400 transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>

```

```

 <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
 {PRESETS.map((p, i) => (
 <button
 key={p.name}

```

```

const clickable = !solved && n.depth > 0;
return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6666ff" stroke-width={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6666ff" : path.hasBranch ? "#a5b4fc" : "#475568"}
 stroke={isTarget ? "#a5b4fc" : "#475568"}
 stroke-width={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
}})
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>

```

```

>
 <div className="flex items-center gap-2 font-l
 {wrong === 0 ? <CheckCircle2 className="w-4
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.leng
 : `Узел ${target} в корне за ${moves.leng
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили <span classN
 {m.expected}

)
)
 }

)}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
)}
 </div>
)}
</div>
);
};

```

---

ФАЙЛ 72/111: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```

import React, { useEffect, useMemo, useRef, useState }
import { Gauge, Pause, Play, RotateCcw, Terminal } from
import { useVizRuntime, vizNumber, vizString } from "..

```

```

keys: Partial<Record<NodeId, number>>;
/** Обход слева направо – одинаковый на всех кадрах.
inorder: string[];
/** Что означают поддеревья A..D. */
ranges: string;
frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",

```

```

{
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый сл
 зглаживает.",
 ms: RESULT_MS,
},
{
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крути
 focus: ["g", "p"],
 ms: AIM_MS,
},
{
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему п
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
},
{
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
},
{
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, ветка x подтянулась",

```

```

 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

```

```

 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill={color}
 {len > pad * 2 + 6 && {
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
 }}
 </g>
);
}
}

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a) && focus.includes(b));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
 />
);
})
}
}

```

```
/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-zigzig": 1,
};
```

```
export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);
```

```
/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizString("zig-zig") : null;
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(0) : null;
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(0) : null;
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(0) : null;
const lastSig = useRef<string | null>(null);
```

```
useEffect(() => {
 if (!liveCase && liveStep === null) return;
 const norm = (liveCase ?? "").trim().toLowerCase();
 const idx = CASE_BY_NAME[norm] ?? caseIdx;
 const sig = `${idx}:${liveStep ?? ""}`;
 if (sig === lastSig.current) return;
 const first = lastSig.current === null;
 lastSig.current = sig;
 setPlaying(false);
 if (first || idx !== caseIdx) {
 setCaseIdx(idx); // новый случай — начинаем его
 setFrame(0);
 } else {
 // тот же случай, следующий шаг кода — пролистывать
 setFrame((f) => Math.min(CASES[idx].frames.length, f + 1));
 }
}, [liveCase, liveStep, caseIdx]);
```



```

 Кор,
 ({active.rotat
</div>

```

```

{/* шаг + пояснение стоят НАД картинкой: сначала
<div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`
 >
 {current.kind === "aim" ? "прицел" : current

 <span className="text-sm font-bold text-white
 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leadin
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
 <Tree frame={current} prev={prevPos} keys={acti

{/* обход слева направо – одинаковый на всех ка
<div className="mt-2 pt-2 border-t border-slate
 <div className="flex items-center gap-1.5 fle
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-m
 /\d/.test(t) ? "bg-emerald-500/15 tex
 }`
)
)
}

```

```

 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1x" : "1x"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto

```

```
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не
 </p>
 </div>
);
};
```

---

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useMemo, useRef, useState } from 'react';

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот:
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 * rot – поворот: зелёным подсвечены рёбра, которых не было.
 * Рядом с логом всегда висит врезка «Механика одного поворота».
 * На каждом кадре – живая автопроверка in-order (сортировка).
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents).
 */

export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | null {
```

```

 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в виде "p→c" – для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];

```

```

let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск упёрся в ${last.key}
 rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 note: aimNote,
 });
 // ...

```

```

 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
}
const pIsLeft = g.left === p;
const xIsLeft = p.left === target;
if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key
);
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "в
 emitRotation(
 target,
 "Zig-Zag",
 [p.key, target.key],
 `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: п
);
}

```

```

const heightOf = (t: SN | null): number => (t ? 1 + Mat
const inorderKeys = (t: SN | null): number[] => (t ? [.

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: #
 { line: 6, text: " rotate(p over g) # B
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z
 { line: 9, text: " rotate(x over p) # H
 { line: 10, text: " rotate(x over g)" },
];

/* ----- врезка «механика одного по

const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border b
 <h4 className="text-xs font-bold text-slate-300 m
 <p className="text-[10px] text-slate-500 mb-2">
 Как вправление вывиха: одна кость ходит вокруг
 </p>
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fon

```

```

 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669" />
 <text x="70" y="50" textAnchor="middle" fill="black" />
 <circle cx="45" cy="86" r="15" fill="#1e293b" />
 <text x="45" y="90" textAnchor="middle" fill="black" />
 <circle cx="27" cy="126" r="13" fill="#0f172a" />
 <text x="27" y="130" textAnchor="middle" fill="black" />
 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill="black" />
 <circle cx="95" cy="126" r="13" fill="#0f172a" />
 <text x="95" y="130" textAnchor="middle" fill="black" />
 <text x="112" y="92" fill={isRot ? "#10b981" : "black"}
 {isRot ? "↓ β теперь при р" : "β при р"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize=14 />
 </g>
 </svg>
 <p className="text-[10px] text-slate-500 leading-5" style={textStyle}>
 Обход слева-направо α x β p γ не меняется никог
 числа поворотов. Единственный, кто меняет хозяи
 </p>
</div>
);
};

/* ----- рендер -----

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

```



```

 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 });
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(({[
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py

const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
const hBefore = useMemo(() => (steps ? heightOf(steps
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
 if (!steps || !step || idx <= 0) return null;
 return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {

```

```

 };

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Кликн
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3">
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current)
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 placeholder="ключ + Enter"
 className="w-28 bg-slate-950 border border-slate-800
 s:border-indigo-500"
 />
 <button
 onClick={() => {
 baseRef.current = buildInitial();
 setSteps(null);
 }}
 />
 </div>
 </div>
 </div>
);
 }
}

```

```

 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); s
 className="px-3 py-1.5 rounded-lg text-xs fo
 Шаг →
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (
 <span className="px-2.5 py-1 rounded-full t
 00/60">
 {step.caseLabel}

)}
</div>
)}

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
 {!steps && (
 <div className="absolute top-3 left-3 bg-indi
 ont-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} classNam
 {edges.map((e) => {
 const a = pos.get(e.a);
 const b = pos.get(e.b);
 if (!a || !b) return null;
 const isAim = step?.aimEdge && step.aimEdge
 const isCutEdge =
 step?.kind === "cut" && step.cutEdge && s
 .cutEdge[1];
 const isMoved = !isAim && !isCutEdge && pre
 return {

```

```

 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row"
 <div className="flex-1 bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "× Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Клики по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "
 {checkOk ? " " : " "} проверка кадра: o
 {inorderKeys(viewRoot).length}
 {movedCount > 0 ? ` · переехало рёбер:
 </p>
 }}
 </div>
 <div className="w-full lg:w-[300px] shrink-0"
 <MechanismInset step={step} />
 </div>

```

```
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 || za2
 return (
 <div key={l.line} className={`py-1 px-2 border-400` : "text-slate-400"}`}>

 {l.text}

 </div>
);
 }
 }
 </div>
</div>
</div>
);
};

export default SplayWalkViz;
```

---

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```

const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')

```

```

 });
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

 const topIndex = displayStack.length - 1;

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-4xl">
 Ты складываешь грязные тарелки друг на друга.
 </h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 А когда приходит время мыть, ты берёшь именно эту.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}

```

```

"></div>
 <div className="absolute w-4 h-4 rounded-
"></div>
 <div className="absolute w-5 h-5 rounded-
"></div>
 <div className="absolute text-4xl anim-cr
</div>
})

{displayStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {displayStack.slice().reverse().map(({plate
 const idx = displayStack.length - 1 - r
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}

```



```

<div className="flex-1 flex flex-col justify-center"
 {cleanRack.length === 0 ? {
 <div className="flex-1 flex flex-col item-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
 } : {
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-0.5"
 >
 {plate.name}

 </div>
 })}
 </div>
 })}
</div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
 <div className="text-[10px] font-mono text-slate-400"
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ?
 {entry}
 } : {

```

```

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }
}

```

```

const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPa
 if (mode === "kmp") return generateKmpSteps(safe
 else return generateZSteps(safePattern, safeText
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

```

```

<div className="flex items-center gap-2"
 <div className="bg-slate-800 p-1 flex items-center"

 <input value={pattern} onChange={onPatternChange} />
 </div>
 <div className="bg-slate-800 p-1 flex items-center"

 <input value={text} onChange={onTextChange} />
 </div>
 </div>
 </div>

<div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start"
 <div className="flex gap-1.5 px-4 h-full"
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-900";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-400";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(index)) {
 borderColor = "border-emerald-500";
 bgColor = step.match === index ? "bg-emerald-500" : "bg-slate-900/40";
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-500";
 }
 }
 })}
 </div>
</div>

```

```

 <span className="abs"
 </div>
 }}
 {mode === "kmp" && step.i
 <div className="abs"
 <span className="abs"
 </div>
 }}
 {mode === "kmp" && step.i
 <div className="abs"
 <span className="abs"
 </div>
 }}

 {/* Z fingers */}
 {mode === "z" && step.i
 <div className="abs"
 <span className="abs"
 </div>
 }}
 {mode === "z" && step.z
 <div className="abs"
 <span className="abs"
 </div>
 }}
 {mode === "z" && step.z
 <div className="abs"
 <span className="abs"
 </div>
 }}

 {/* Found flash effect */}
 {step.found && mode ===
 <div className="abs"
 ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_
)}
 }
}

```

```

 {steps.length}}</div>
 <div className="text-sm text-slate-500">
 {step.desc}
 </div>
 </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400 leading-relaxed">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значение {mode === 'kmp' ? "применяется" : "не применяется"}
 о было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z).
 </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 px-4 py-2 border border-slate-700">
 Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {

```

```

 <span
 className={`relative inline-flex ${className} ?? ""
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 l duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full
 } ${open ? "visible translate-y-0 opacity-100"
 >
 {content}
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "-bottom-[5px] border-b bo
 }`}
 />

);
 };
};

```

ФАЙЛ 77/111: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
 Play,
 Pause,

```

```

 topoList: number[];
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(

```



```
// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
} else {
 setIsPlaying(false);
}
return () => clearInterval(timer);
```

```

 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 { /* Playfield SVG */ }
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4" />
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8" />
 </marker>
 </defs>
 </svg>
 </div>
</div>

```

```

 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text

```

```

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
</div>

{/* Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Следующий шаг
 </button>
 </div>

```

```
* search – «отсортирую бинарным поиском»: поиск не с
* сортировка платит $\geq n \cdot \log n$ сравнений, сор
*
* Канонический набор: все x и все y различны $\Rightarrow$ дерево
* Узел идентифицируется парой id = "x:y".
*/
```

```
export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>
```

```
interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];
```

```
/** Порядок ввода (как в черновике) – намеренно НЕ отсор
```

```
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
```

```

 trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
};
const res = go(root);
return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; note: string; }[];
}

export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null => {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: `Объединено целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `Выбрано ${a.key} с его правым поддеревом с (${b.key}, ${b.grade})` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `Выбрано ${b.key} с его левым поддеревом с (${a.key}, ${a.grade})` });
 b.left = go(a, b.left);
 return b;
 };
};

```

```

 if (parentGrade !== null && n.grade > parentGrade) {
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
 }

```

/\* ----- раскладка сцены дерева -----

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 44 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id }); }
 if (n.right) { es.push({ from: n.id, to: n.right.id }); }
 };
 walk(root);
 return es;
};

```

```

interface TreeViewProps {

```

```

 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate
 <circle r={pulse ? 24 : 20} fill={nodeColo
 } strokeWidth={pulse ? 4 : 2} className="tr
 <text y={-2} fill="#fff" fontSize="13" fo
 <text y={13} fill="#cbd5e1" fontSize="10"
 </g>
);
 })}
 </svg>
 {subtitle && <p className="text-xs text-slate-400
 </div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;

```



```

 2} strokeDasharray={isBad ? "4,3" : "none"}
 }}}

 {/* временная линия игры */}
 {interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDash=
 />
)}

 {points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key ===
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ?
 onPointerUp={interactive && onPointUp ? () :
 style={interactive ? { cursor: "pointer" }
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={
 troke={isSel ? "#a5b4fc" : done ? "#38bdf8"
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
 ">{p.key}</text>
 </g>
);
)}
)}

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}
)}

```

```

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа
export function forestIsBst(edges: GEdge[], all: Pair[]) {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || [])]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
 const ok = (n: Pair | undefined, lo: number, hi: number) => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 const oneParent = new Set(edges.map((e) => gpid(e.b)));
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.key));
 const complete = edges.length === all.length - 1 && roots.length === 1;
 return { heap, oneParent, acyclic, bst, complete, win: true };
}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
 if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" };
 if (a.grade === b.grade) {
 return { ok: false, reason: "«!» равны – «кто выше?»" };
 }
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpid(child))) {
 return { ok: false, reason: `У точки ${child.key} уже есть родитель` };
 }
 return { ok: true, reason: "ОК" };
}

```

```

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
 a: Pair;
 b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
 edges: GameEdge[],
 x: Pair,
 y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
 if (x.key === y.key && x.grade === y.grade) return { ok: false, reason: "такая линия уже проведена" };
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 (pid(e.a) === pid(y) && pid(e.b) === pid(x))) {
 return { ok: false, reason: "такая линия уже проведена" };
 }
 }
 if (x.grade === y.grade) return { ok: false, reason: "родитель и ребёнок должны быть в разных классах" };
 const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
 if (edges.some((e) => pid(e.b) === pid(child))) {
 return { ok: false, reason: `у точки (${child.key}; ${child.grade}) уже есть родитель` };
 }
 // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a))) {
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 }
}

```

```

const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяе
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
 };
 walk(roots[0]);

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return;
 setRaw(live.pairsText);
 setPhase("connect");
 setOrder("y-desc");
 setSortIdx(3);
 setConnIdx(10000); // все рёбра проведены – ср
 setPlaying(false);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [live.pairsText]);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | nul

 const parsed = useMemo(() => parsePairs(raw), [raw]);
 const userPairs = parsed.errors.length ? [] : parsed.p

 const n = userPairs.length;

 const userTree = useMemo(() => (userPairs.length >= 1
 const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, gr
 left}); }
 if (node.right) { es.push({ a: { key: node.key, g
 de.right}); }
 }
 }, [userPairs]);

```

```
const [gameMsg, setGameMsg] = useState<string | null>
const [badEdge, setBadEdge] = useState<GameEdge | null>
const [hintUsed, setHintUsed] = useState(false);
```

```
const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};
```

```
const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};
```

```
const verdict = useMemo(() => checkGameState(userPair)
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree
```

```
const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};
```

```
const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 }
};
```

```

 <h4 className="text-sm font-bold text-slate-300">
 <button onClick={() => { setRaw(DEMO_RAW); setRaw(
 py-1 rounded-lg text-xs font-bold bg-slate-500
 демо-набор
 </button>
 </div>
 <textarea
 value={raw}
 onChange={(e) => { setRaw(e.target.value); setRaw(
 rows={2}
 className="w-full bg-slate-950 border border-
 s:border-emerald-500"
 placeholder="3 5, 45 8, 1 -4 ..."
 />
 {parsed.errors.length > 0 && (
 <div className="mt-2 text-xs text-rose-300 bg-
 {parsed.errors.map((e, i) => <div key={i}>{e}
 </div>
)}
 {parsed.errors.length === 0 && (parsed.equalX |
 <div className="mt-2 text-xs text-amber-300 b
 {parsed.equalY && <div>⚡ Есть равные y
 кто раньше в списке (сортировка ста
 {parsed.equalX && <div>⚡ Есть равные x
 de.key → left).</div>}
 </div>
)}
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4"
 <div className="flex flex-wrap items-center jus
 <h4 className="text-base font-bold text-emera
 <div className="flex items-center gap-2">
 {phase === "sort" ? (
 <div>
 <span className="text-xs text-slate-400"
 {["quicksort", "counting"] as const).m

```

```

 })
 </div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} пар
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: cl
 {comparisons} ($n \cdot \log_2 n$).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не
 Игру </>
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказан
 ево: линии вниз по y, обход по x отсортиров
) : gameMsg ? (
 <>⚠ {gameMsg}</>

```



```

 ate-700 text-white disabled:opacity-30"> C
 <button onClick={showSolution} disabled={gameOver}
 te disabled:opacity-30"> Показать решение<
 </div>
)}
 </div>

 {/* Победное дерево в игре */}
 {phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-4">
 <h4 className="text-sm font-bold text-emerald-300">
 Победное дерево
 </h4>
 <TreeView root={gameTree} subtitle="Слева направо существует." />
 </div>
)}

 {/* Дерево */}
 {order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">
 Дерево
 </h4>
 <TreeView root={userTree} subtitle="Слева направо существует." />
 </div>
)}
</div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
 const split = useMemo(() => splitTree(buildTree(POINTS), x0), [fullTree, x0]);
 const steps = split.trace;

```

```

<div className="bg-slate-900/40 rounded-xl p-4 border" style={borderStyle} >
 <h4 className="text-sm font-bold text-slate-300">Розовые узлы</h4>
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={({id}) => (inL.has(id) ? "#1e3a8a" : "#f43f00")}
 nodeStroke={({id}) => (inL.has(id) ? "#60a5fa" : "#f43f00")}
 subtitle={done ? "Каждый узел покрашен ровно 1 раз" : ""}
 />
</div>

<div className="text-xs text-slate-300 bg-slate-900/40 rounded-xl p-4 border" style={borderStyle} >
 {steps[idx]?.note}
</div>

{done && (
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4" style={borderStyle} >
 <div className="bg-sky-950/20 rounded-xl p-3 border" style={borderStyle} >
 <h5 className="text-sm font-bold text-sky-300">Левое дерево</h5>
 <TreeView root={lTree} width={480} height={300} />
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 border" style={borderStyle} >
 <h5 className="text-sm font-bold text-rose-300">Правое дерево</h5>
 <TreeView root={rTree} width={480} height={300} />
 </div>
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-900/40 rounded-xl p-4 border" style={borderStyle} >
 Как это работает на самом деле: идём с корнем (или с веткой, или с
 деревом), отдаётся вместе с веткой – даём её в левое или в правое
</div>
</div>
);
};

/* ===== РЕЖИМ MERGE ===== */

```

```

<div className="text-xs text-slate-300 bg-slate-900" style={cur?.chosenId ? {
 <div> {cur.note}</div>
} : {
 <div>Готово: {done ? "деревья склеены обратно в исходное состояние" : ""}</div>
}}>
</div>

{done && (
 <div className="bg-emerald-950/20 rounded-xl p-4" style={cur?.chosenId ? {
 <h5 className="text-sm font-bold text-emerald-400">Готово:</h5>
 <TreeView root={merged.root} subtitle="merge({left, right})">
 </div>
)}>

 <div className="text-xs text-slate-400 bg-slate-900" style={cur?.chosenId ? {
 Грамотно про «слияние»: у корней сравнения
 в (то, что сохраняет порядок x) отправляется, а в (то, что сохраняет порядок y)
 </div>
 </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
 const res = useMemo(() => eraseNode(buildTree(POINTS), key), [key]);
 const steps = res.trace;

 // key из кода выбирает удаляемую точку, step листает
 useEffect(() => {
 if (live.key === null || !POINTS.some((p) => p.key === key)) {
 setKey(live.key);
 }
 }, [live.key]);
};

```

```

 nodeStroke={(id) => (pathSet.has(id) ? "#f59e02" : "#f59e02")}
 subtitle="Куча сама умеет удалять только вершину"
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-300" >Валидность
 <TreeView root={res.root} subtitle="Валидность" >
 <ul className="mt-2 space-y-1 text-sm" >
 обход слева-направо по-прежнему сортирует
 каждое ребро вниз по y (куча цела) –

 </div>
 </div>
)}

 <div className="text-xs text-slate-300 bg-slate-900/40 p-4" >
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-900/40 p-4" >
 Ответ на «убрать можно только самый верхний узел» – нет.
 Ягивает merge двух детей. Удаляется любая вершина.
 </div>
</div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("heap");
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null);
 const walk = (n: TNode | null, i: number) => {

```

```

 >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 ro
 der-slate-700" : "bg-rose-950/40 border-rose
 <div className="text-slate-500 text-[
 {id ? <div className="text-white font
 </div>
 </div>
);
 }}}
</div>
<p className="text-xs text-rose-300 mt-2">Смо
то левая ветка ушла на 4 уровня и заняла да
казателями (left/right), а не арифметикой.<
</div>
)}
 <div className="text-xs text-slate-400 bg-slate-9
 Грабля «дети в массиве по $2k/2k+1$ »: фор
 ивым: слои дырявые, и «адрес» ребёнка посчита
 </div>
 </div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====

const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО п
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },

```

```

 })
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSteps
 - 1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
 })
 <button onClick={() => { setStage(0); setBidx(0)
 C6poc</button>
</div>

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s :
 const isMid = stage === 2 && (() => { const s :
 const isFound = stage === 2 && bidx >= bsSteps
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} class=
 nsition-all ${highlight ? "border-rose-500 b
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px]
 </div>
);
 }}}
</div>

<div className="text-xs text-slate-300 bg-slate-9
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарн

```

```

 setMsg(`  Линия (${parent.key}; ${parent.grade})`);
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
 };
 };

```

```
const win = v.win;
```

```

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0, 3)); }}>
 rounded-lg text-xs font-bold bg-slate-700 text-white
 <button onClick={() => { setEdges([]); setSel(null); }}>
 3 py-1.5 rounded-lg text-xs font-bold bg-slate-700 text-white
 <button onClick={() => { setEdges(CANON_GAME.map(e => {
 }); }} className="px-3 py-1.5 rounded-lg text-white">
 линий:
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4">
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} className="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="white" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16}/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)}/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(1)}/>

 {edges.map((e, i) => (
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}

```

```

 <p className="mt-3 text-sm font-bold text-emerald-500">
 ения просто не существует.</p>
 </div>

 <div className="text-xs text-slate-300 bg-slate-900 p-2">
 </div>
 </div>
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ ===== */

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИВАЕМ ДЕРЕВО
 * mid – разрез для вкладки Split;
 * key – удаляемая точка для вкладки Erase;
 * step / i – номер шага (листаем кадры Split / Erase);
 * height, n – высота дерева и число узлов (показываем прогресс);
 */
const useTreapLive = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const pairsText = useMemo(() => {
 if (!vars) return null;
 const rawPts = vizArray(vars.points) ?? vizArray(vars.pairs);
 if (!rawPts || rawPts.length === 0) return null;
 const parts: string[] = [];
 rawPts.forEach((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const x = vizNumber(it[0]);
 const y = vizNumber(it[1]);
 if (x !== null && y !== null) parts.push(`${x} ${y}`);
 } else if (typeof it === "string") {
 const m = it.match(/-?\d+(?:\.\d+)?/g);
 if (m && m.length >= 2) parts.push(`${m[0]} ${m[1]}`);
 }
 });
 }, [vars]);

 return {
 pairsText,
 };
};

```



```

<div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пар
 </p>
 </div>
 </div>
</div>

<div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
 </div>
 </div>

 /* что сейчас сообщил компилятор */
 <div className="mb-4 flex flex-wrap items-center">

 <Terminal className="w-3.5 h-3.5" /> из компи

 {live.vars ? (
 <div>
 {live.pairsText &&
 d px-2 py-1">points = {live.pairsText}
 {live.n !== null &&
 py-1">n = {live.n}
 {live.height !== null &&
 px-2 py-1">height = {live.height}
 </div>
)}
 </div>

```

```

 demoTitle?: string;
 className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**
 * Строка состояния связи «компилятор → демонстрация» д
 *
 * Раньше компонент молча проигрывал собственный встроен
 * редакторе Python был пустой код, код с другими именам
 * размерности, — выглядело так, будто компилятор испол
 * картинка живёт отдельной жизнью. Теперь источник дан
 * «данные из компилятора {i, j, dist 5x5}», «компилятор
 * пока нет» (отладчик стоит до первой строки) или «встр
 */
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name) :
 const linked = names.length > 0;

 const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
 const matrix = liveMatrix(matrixSource?.value);

 const detail = matrix
 ? `${matrixSource?.name} ${matrix.sizeLabel}`
 : names.length
 ? names.slice(0, 6).join(", ") + (names.length >
 : undefined;

 return (
 <VizSourceBadge
 linked={linked}
 awaiting={!linked && !!runtime}
 line={runtime?.line}
 detail={detail}

```

```

import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки — это ровно то, что нужно для DFS.",
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую.",
 Component: HeapViz,
 },
}
```

```

"euler-path-vs-cycle": { title: "Эйлеров путь и цикл"
"planarity-euler-formula": {
 title: "Планарность и покраска: $K_5/K_{3,3}$, $\chi \leq \Delta+1$, д
 hint: "Четыре вкладки: непланарные K_5 и $K_{3,3}$, жадна
 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
},
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируе
 Component: DijkstraViz,
},
"bellman-ford": {
 title: "Форд–Беллман",
 Component: BellmanFordViz,
},
floyd: {
 title: "Флойд–Уоршелл",
 Component: FloydViz,
},
"johnson-algo": { title: "Алгоритм Джонсона", Component
mst: {
 title: "Остовное дерево: Краскал, Прима, Борувка",
 hint: "Три алгоритма (билеты 18–20) на одном графе
 еременные parent/mst/taken/u/v/w перекрашивают гр
 Component: KruskalSimulator,
},
"graph-components": {
 title: "Компоненты связности: обход и DSU",
 hint: "Две вкладки: покраска обходом (каждый новый
 nt из вашего кода красят вершины.",
 Component: GraphComponentsViz,
},
"complexity-classes": {
 title: "Классы сложности и сведения",
 hint: "Иерархия $P \subseteq NP \subseteq PSPACE \subseteq EXP$, сведение $A \leq$
 Component: ComplexityViz,
},

```

```

 * false – компонент показывает собственную встроенную
 */
 linked: boolean;
 /**
 * Компилятор уже выполнил код и прислал шаг, но пере
 * (отладчик стоит ДО первой строки или в коде другие
 * подпись должна объяснять, почему картинка ещё не и
 */
 awaiting?: boolean;
 /** Строка пользовательского кода, на которой стоит о
 line?: number;
 /** Короткая подпись данных компилятора: «матрица 5x5
 detail?: string;
 /** Как называется встроенная демонстрация (для подпи
 demoTitle?: string;
 /** Размеры демо-сетки и данных пользователя не совпа
 mismatch?: string | null;
 className?: string;
}

/**
* Явный индикатор источника данных визуализации.
*
* Без него демонстрация молча проигрывала собственный
* когда редактор Python пуст или в нём другие имена/ра
* так, будто компилятор «исполняет какой-то свой код».
* «из компилятора» или «встроенное демо».
*/
export function VizSourceBadge({ linked, awaiting = false,
 const tip = linked
 ? `Значения берутся из выполненного Python-кода${de
 : awaiting
 ? `Панель Python выполнила код и стоит на строке
 ажмите «шаг вперёд» (→) или «в конец трассы» –
 лизация их не увидит: список имён этой темы есть
 : "Сейчас показана встроенная демонстрация компонент
 визуализация переключится на ваши переменные."`;

```

```
 });
 }
}
```

---

ФАЙЛ 82/111: src/components/WaterfallAnimationWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';
```

```
// Структура заранее подготовленного красивого бора для визуализации
```

```
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4, fail: 0, term_link: 0, is_terminal: true, words: ['HERS'] },
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['HIS'] },
};
```

```

const liveI = vars ? vizNumber(vars.i) ?? vizNumber(v
const liveMode = vars ? vizString(vars.mode) : null;
const liveMatchesRaw = vars ? vizArray(vars.matches)
const liveMatchesSig = useMemo(() => (liveMatchesRaw
const liveFail = vars ? vars.fail : undefined;

useEffect(() => {
 if (!vars) return;
 if (liveMode && /search|поиск/i.test(liveMode)) set
 else if (liveMode && /train|обуч|bfs/i.test(liveMod
 if (liveText) {
 const clean = liveText.toUpperCase().replace(/[^A
 if (clean && clean !== textToScan) {
 setTextToScan(clean);
 setCurrentIndex(0);
 setCurrentNode(0);
 setFoundMatches([]);
 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
 }
 if (liveState !== null && liveState >= 0 && WATERFA
 if (liveI !== null && liveI >= 0) setCurrentIndex(M
 ToScan).length));
 if (liveMatchesSig) {
 const parsed = (JSON.parse(liveMatchesSig) as unk
 .map((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const idx = vizNumber(it[0]);
 const word = vizString(it[1]);
 if (idx !== null && word) return { index: i
 }
 if (typeof it === "string") {
 const m = it.match(/(\d+)\D+([A-Za-zA-Яa-я]
 if (m) return { index: Number(m[1]), word:
 }
 })
 return null;
 }

```

```

 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен!');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лосось доволен!`;
 setJumpPath(null);
 } else {

```



```

 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'Ø',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на Dep
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,

```

```

 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
};

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?

```

```

 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
</div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? (
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 аг 8 (SHE → HE), где наглядно виден
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white fon
 : 'bg-slate-800 text-slate-400
 }`}
 >
 {step
 {idx === trainStep && <span classNa
 </button>
 >>>
 </div>
)
 </div>

```

```

 {currentTrainInfo.codeLine === 1 && <
тут}
 </div>
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-l-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold m
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-:
w-inner min-h-[72px] flex items-center">

```

```

 : 'bg-slate-800 text-slate-500'
 }` }
 >
 {char}

)))
 {currentIndex >= textToScan.length && (
 <span className="bg-emerald-600 text-
 ГОТОВО ✓

)}
</div>

<button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium
 isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'
 }` }
 >
 {currentIndex === 0 ? 'Начать план' : ''}
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4" >
 <div>
 <h5 className="text-white font-bold mb-3">
 ⚡ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-lg">
 <div className={`p-1.5 rounded flex items-center justify-between
 border-blue-500 text-blue-300 font-bold` : ''}>
 1. curr = state; char = text[i]

```

```

<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none"></div>
<div className="absolute top-0 left-1/2 -translate-x-1/2 -hidden shadow-2xl">
 <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>

 </div>

 <div className="w-full overflow-x-auto custom-scrollbar">
 <svg viewBox="0 0 800 520" className="w-full h-full">
 <g>
 <!-- Горизонтальные линии и подписи уровней -->
 <g>
 <g>
 {
 { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
 { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
 { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
 { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
 { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' },
 }.map((level) => {
 <g key={`depth-line-${level.depth}`}>

```



```
}}}
```

```

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetId;
 const hasWords = node.words.length > 0;

```

```

return (
 <g key={`node-${node.id}`} className="node" style={{position: 'relative'}}>
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r="24" fill="none" stroke="black" stroke-width="2"/>
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r="24" fill="none" stroke="red" stroke-width="2"/>
)}

```

```

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24' : '20'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? 'red' : 'white'}
 stroke={isCurrent ? 'black' : isTargetChild ? 'red' : 'black'}
 stroke-width={isCurrent || isTargetChild ? 2 : 1}
 className="transition-all duration-300"
 />

```

```

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 font-size={isCurrent || isTargetChild ? 14 : 12}
 font-weight="bold"
 text-anchor="middle"
 />

```

```
>
```



```

 />
 { /* Сама рыба */ }
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezier(0.25, 0.8, 0.25, 0.75)' }}
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' : ' ♀ '}
 </text>
 </g>

</svg>
</div>
</div>

{ /* Список найденных совпадений (только в режиме поиска) */ }
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded-lg" >
 <h5 className="text-white font-bold mb-3 text-sm" >
 Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic" >
 Пока ничего не поймали. Запустите шаги симуляции
 </p>
) : (
 <div className="flex flex-wrap gap-2" >
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-500 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-400" />
 {match.word}
 </div>
))}
 </div>
)}
 </div>
)}

```

```
 </div>
 </div>
);
};
```

---

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

---

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 10.5 КБ

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
} from '../../../data/vizStepBus';

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```

)))
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]],
 []
);
};

/** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "a" }>({ kind: "a" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
 : Math.max(0, Math.min(A1.length, (liveP?.length ?? 0)))
 : null;

const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

```

```

 >
 {v}
 </div>
);
 }}}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === pref.length - 1;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue === null || liveValue === "");
 const masked = !hasLiveP && shownDriven !== null;
 const isDrivenCur = shownDriven !== null && i === shownDriven;
 const shownValue = hasLiveP ? liveValue : null;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref", value: v })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help ${
 isDrivenCur
 ? "bg-emerald-600 text-white ring-1 ring-emerald-600"
 : blank
 ? "bg-slate-950 text-slate-600"
 : masked
 ? "bg-slate-800/50 text-slate-700"
 : active
 ? "bg-emerald-500 text-white"
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-500"
 }`}
 >
 {shownValue}
 </div>
);
 })}
 </div>
 {liveP}
</div>

```

```

) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)
 </div>

 { /* Запрос */ }
 <div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[range
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ п
 </p>
 </div>
</div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {

```

```

 Array.from({ length: prefCols }, (_, j) => liveA[0][j]
)
 : P;

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
 ? Array.from({ length: srcRows }, (_, i) => Array.from(
 { length: srcCols }, (_, j) => liveA[i][j]
))
 : A2;

const cellAt = (i: number, j: number): number | null {
 return i >= 0 && j >= 0 && i < prefRows && j < prefCols ? srcGrid[i][j] : null;
}

const sizeMismatch =
 hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
 ? `Демо построено на матрице ${n}x${m} (сетка преобразована из массива). Сетка имеет размер ${liveS.sizeLabel}. Ниже нарисована сетка с границами запроса`
 : null;

useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);
 if (r1 === null || c1 === null || r2 === null || c2 === null) return;
 setRect({
 r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(r1))),
 c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(c1))),
 r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(r2))),
 c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(c2)))
 });
}, [vars, srcRows, srcCols]);

// Границы запроса держим внутри реальной сетки (у пользователя может быть сетка 10x10)
const r1 = Math.max(0, Math.min(rect.r1, prefRows - 1));
const c1 = Math.max(0, Math.min(rect.c1, prefCols - 1));
const r2 = Math.max(r1, Math.min(rect.r2, prefRows - 1));
const c2 = Math.max(c1, Math.min(rect.c2, prefCols - 1));

```

```

 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1
 ? { r1: i, c1: j, r2: Math.max(i, j), c2: Math.max(i, j) }
 : { ...r, r2: i, c2: j }
)
)
 className={` ${cellBase} cursor-pointer
 inRect ? "bg-indigo-500 text-white" : ""
 }`}
 >
 {v === null ? "." : v}
 </div>
);
 }}}
</div>
)}}
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Префиксные суммы {liveS ? `${livePrefSource}` : ""}
 </div>
 <div className="inline-block">
 {grid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const blank = v === null || v === undefined;
 const shownValue = v;
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hover?.j === j;
 const isCompilerCell = liveI === i && liveJ === j;
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"

```

```

 <p className="text-slate-300">
 <b className="text-amber-400">
 {liveS ? livePrefSource?.name ?? "S" : "P"}
 {" "}
 – сумма всего прямоугольника от левого верхнего
 </p>
) : (
 <p className="text-slate-500">Наведите курсор
 </p>
)}
 </div>

 <div className="bg-slate-900 border border-indigo-500">
 <p className="font-mono text-sm sm:text-base text-slate-100">
 сумма = A +
 C + D
 = {" "}
 {sum}
 </p>
 <p className="font-mono text-[11px] text-slate-100">
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · сумма
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
</div>
);
};

```

ФАЙЛ 84/111: src/components/visualizers/SparseTableViz.  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 899 | РАЗМЕР: 1.5 КБ

```

import React, { useContext, useEffect, useRef, useState } from 'react';
import {
 emitVizDemo,

```



```

 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

```

```

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
);
 };

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>(null);
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);
 const liveRows = vizArray(runtimeVars?.st);
 const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
 const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
 const compilerLinked = liveI !== null || liveJ !== null;
 const shownSource = hasLiveSource
 ? liveSource?.length
 ? liveSource
 : Array.from({ length: N }, () => null)
 : arr1D;
 const liveTarget =
 liveI !== null && liveJ !== null && Number.isInteger(liveI) && Number.isInteger(liveJ)
 ? { i: liveI, j: liveJ }
 : null;

 /** Размерность таблицы пользователя: сетка демо 8x4,
 const liveTable = liveMatrix(runtimeVars?.st);
 const dimMismatch = liveTable

```

```

 } : null}
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1)}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <div className="text-[11px] uppercase tracking-wider">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
 {shownSource.map((v, idx) => {
 const inCover = !!covered && idx >= covered - 1;
 const display = v === undefined || v === null
 ? "."
 : typeof v === "object"
 ? JSON.stringify(v)
 : String(v);
 return (
 <div key={idx} className="flex flex-col items-center justify-center flex-grow-1">

```

```

<div className="inline-block min-w-full bg-slate-500" style={{width: 100%; height: 100%;}}>
 <div className="grid grid-cols-[auto_repeat(4,1fr)] gap-4" style={{width: 100%; height: 100%;}}>
 {/* Headers */}
 <div className="text-slate-500 font-mono text-sm" style={{width: 100%; height: 100%;}}>
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500" style={{width: 100%; height: 100%;}}>
 rap">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500" style={{width: 100%; height: 100%;}}>
 rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500" style={{width: 100%; height: 100%;}}>
 rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500" style={{width: 100%; height: 100%;}}>
 rap">
 2^3 (L=8)
 </div>

 {/* Matrix */}
 {Array.from({ length: N }).map((_, i) => (
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono text-sm" style={{width: 100%; height: 100%;}}>
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => (
 const isValid = i + (1 << j) <= N;
 const liveRow = liveRows?.[i];
 const liveValue = liveRow?.[j];
 // Уровень j=0 – это исходный a[i], от которого берется значение
 // очищаемого рабочего массива и не должно быть вычислено
 const sourceValue = j === 0 ? liveSourceValue : liveValue;
 const runtimeValue = liveValue ?? sourceValue;
 const runtimeHasValue = runtimeValue !== null;
 <div className="text-center font-mono text-sm" style={{width: 100%; height: 100%;}}>
 {runtimeHasValue ? runtimeValue : ''}
 </div>
))}
 </React.Fragment>
))}
 </div>
</div>

```

```

 onMouseLeave={() => setHover(false)}
 onClick={() => {compilerLinked
 }}}

 initial={{ opacity: 0, scale: 1 }}
 animate={{
 // При st = [] каркас уже с
 opacity: compilerLinked ? 1 : 0,
 scale: compilerLinked || isSrc1 ? 1 : 0,
 }}
 className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors
 hover && hover.i === i && h
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : compilerLinked
 ? "bg-slate-950 text-white"
 : "bg-transparent"
 }`}
 title={compilerLinked ? `st[${
 >
 {displayValue}
 </motion.div>
)}

```

```

/* Arrows visualization */
{(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{

```

```

 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(

 ST[{c.i}][{c.j - 1}]

 ,

 ST[{c.i + half}][{c.j - 1}]

) → min(
 {stlD[

 {stlD[c.i + half][c.j - 1]}

) ={" "}

 {stlD[c.i][c.j]}

 </p>
);
))()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)
 </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num

```

```

 не её не отрисовываются, значения вашей матрицы
 : null
 : null;

```

```

// Уровень и клетка берутся прямо из k/r/c; номер строки
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

```

```

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

```

```

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 sm:p-4">
 <h3 className="text-xl font-bold text-indigo-400">
 <p className="text-sm text-slate-400 mb-6 text-indigo-400">
 Для наглядности показываем только <b className="text-rose-300">

 иц!
 </p>
 {dimMismatch2D ? (
 <p className="mb-4 max-w-[520px] rounded-lg bg-slate-800 p-2">
 {dimMismatch2D}
 </p>
) : null}

```

```

 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setPinnedK(step); }}
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-800`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
 </button>
)]}
 </div>

```

```

// показываем из A сразу, как A
// цикла построения st2. Уровни
const sourceValue = step === 0
const shownValue = compilerLink
const isBlank = shownValue === 0
let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-600'
 : 'bg-slate-800/50 text-slate-600'
let zIndex = 'z-0';

const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

if (isActive) {
 if (isCurr) {
 // Значения в ячейке ещё не
 // контур «сюда смотрит код»
 // залитая пустая клетка вы
 highlightClass = isBlank
 ? `bg-indigo-500/10 text-white`
 : `bg-indigo-500 text-white`
 == c ? 'ring-[3px] ring-white ring-offset-2'
 && locked.c === c ? 'ring-[3px] ring-white'
 && locked.c === c ? 'ring-[3px] ring-white'
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (shownK
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
 md:scale-[1.08]';
 } else if (isTop && !isLeft)
 highlightClass = 'bg-blur
] md:scale-[1.08]';
 } else if (!isTop && isLeft

```



```

<div className="flex-1 min-w-0 flex flex-col gap-4" style={{flex: 1}}>
 <div className="bg-slate-900 p-4 sm:p-6 rounded" style={{background: 'black', color: 'white'}}>
 <h3 className="text-lg font-bold text-slate-300">Таблица
 {shownK === 0 ? (
 <div className="text-slate-400 text-sm leading-5">
 {liveTableEmpty ? (
 <div className="rounded-xl border border-slate-700 p-4">
 <p className="font-bold text-amber-300">Таблица
 <p className="mt-2 text-xs text-slate-400">
 Слева на вкладке «Оригинал (1x1)» :
 блоки 1x1 равны её элементам. Пусты
 циклам:
 </p>
 <div className="mt-3 rounded-lg bg-[#0a0f1e] p-4">

 <div className="flex flex-wrap gap-2">
 <div>
 </div>
 </div>
 </div>
) : (
 <div>
 <p className="mb-4">При
 драты имеют размер
 <div className="bg-[#0a0f1e] rounded" style={{background: 'black', color: 'white'}}>
 //

 </div>
 <p className="mt-6 italic text-indigo-400">
 <Play size={14} className="fill-indigo-400">
 </p>
 </div>
) : (
 <div>
 <p>
 а.
 </p>
 </div>
) : (
 <div className="flex flex-col gap-4 animate-fade-in">
 <div className="bg-[#0a0f1e] rounded-xl p-4" style={{background: 'black', color: 'white'}}>
 // Те

```

```

 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c][shownK-1][shownK-
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c + prevL][shownK-1]
 </div>
 <div className="flex item
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}())}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim

```

```

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6"
 <div className="flex-1 min-w-0 bg-slate-900"
 <h3 className="text-xl font-bold text-rose-500"
 <p className="text-slate-400 text-[13px] font-normal"
 еpx, r2, c2 - правый низ).</p>
 {queryMismatch ? (
 <p className="mb-3 rounded-lg border border-rose-500"
 {queryMismatch}
 </p>
) : null}

 <div className="flex flex-col items-center"
 <div className="grid grid-cols-[repeat(8,1fr)] gap-2"
 <div className="relative shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_, idx) => {
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;

 return (
 <div key={idx} className={`flex flex-col items-center rounded text-[clamp(9px,2.4vw,12px)] font-normal border-rose-500/50 border scale-105 z-10`
 {isInQuery ? "bg-rose-500 text-white" : "bg-slate-900 text-slate-400"}
 {val2D[r][c]}
 </div>
)
 })}
 </div>
 </div>
 </div>
 </div>
 </div>
)
)

```

```

 </div>
 </label>
 <label className="flex items-center" >

 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-2 px-2 rounded" >
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-2 px-2 rounded" >
 </div>
 </label>
 </div>
 </div>
 </div>

```

```

<div className="flex-1 bg-slate-900 p-6 rounded" >
 <h3 className="text-xl font-bold text-emerald-400" >
 <p className="text-sm font-sans text-slate-400" >
 Мы знаем, что максимальная степень дв
 rounded text-white">{Lx}. Аналогично
 te">{Ly}.
 </p>
 <p className="text-sm font-sans text-slate-400" >
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded border
 роса. (Цветные прямоугольники слева).
 </p>

```

```

 <div className="bg-[#0a0f1e] rounded-xl p-6" >
 text-slate-300 shadow-inner overflow-x-auto" >
 <div className="absolute right-3 top-3" >
 // 1.
 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}


```

```

 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky].text;

 let badge = null;
 if (isTL) { color = "bg-rose-400 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-blue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
 transition-all duration-300 relative shrink-0 ${cx} ${cy}
 {txt}
 {badge && <div className="text-white font-bold rounded"
 border="2" padding="2px 5px" style={{position: 'absolute', top: -10px, left: -10px;}}
 </div>
 </div>
)
)}
 </div>
 </div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white" style={{padding: 10px}}>
 <div className="text-center font-bold text-white" style={{border: 1px solid #0a0f1e, padding: 3px 2px, border-radius: 10px, width: fit-content, margin: 0 auto;}}>


```

```
 : 'px-3 py-1 rounded text-sm font-bold'

const aside = document.querySelector('aside')
const headerMain = document.querySelector('h1')
const questionsContainer = document.getElementById('questions')
const wtfContainer = document.getElementById('wtf')

if (mode === 'wtf') {
 document.body.classList.add('wtf-mode')
 if (aside) aside.style.display = 'none'
 if (headerMain) headerMain.style.display = 'none'
 if (questionsContainer) questionsContainer.style.display = 'none'
 if (wtfContainer) wtfContainer.classList.remove('hidden')

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname + window.search)
 window.scrollTo(0, 0);
} else {
 document.body.classList.remove('wtf-mode')
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden')
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]').forEach(btn => {
 btn.classList.remove('outline', 'outline-1');
 });

 if (theme === 'dark') document.getElementById('dark-button').classList.add('active')
 if (theme === 'light') document.getElementById('light-button').classList.add('active')
```

```

 setTimeout(() => drawerOverlay.
 }
 document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.c
 toggleDrawer();
 }
 }));
});

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href

```

```

 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; color: white;
 overflow: hidden; flex-shrink: 0; text-shadow: 2px 2px 0 #000,
 -1px -1px 0 #000; font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute; text-shadow: 1px 1px 0 #000,
 -1px -1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after {
 font-size: 0.8em;
 }

 /* Global formula scroll settings */
 .formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
 }
 .formula-scroll::-webkit-scrollbar {
 display: none;
 }

 mjx-container {
 max-width: 100% !important;
 }

```



```

 .card-blue { background-color: #004dcf; } .card
 .card-green { background-color: #339e35; } .card
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
 background-color: #fcf8eb !important; /* li
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Se

```

```

</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Режим
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal" class="text-slate-400
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf" class="text-white
 bg-slate-700 hover:bg-slate-700 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark" class="text-slate-400
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light" class="text-slate-400
 bg-slate-700 hover:bg-slate-700 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook" class="text-slate-400
 bg-slate-700 hover:bg-slate-700 transition-colors">Notebook</button>
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" markerHeight="6" viewBox="0 0 6 6">
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d" />
 </marker>
 <marker id="arrow-orange" markerWidth="6" markerHeight="6" viewBox="0 0 6 6">
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d" />
 </marker>
 <marker id="arrow-pink" markerWidth="6" markerHeight="6" viewBox="0 0 6 6">
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b2" />
 </marker>
 </defs>
 </svg>

```

```

 <path d="M 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="15"
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin
 </svg>
 </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0
 background-color: #000; opacity: 0.5; z-index: 1000;

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-

 <!-- Левое меню (Оглавление) -->
 <aside id="mobile-drawer" class="fixed inset-y-0 left-0
 translate-x-full transition-transform duration-300
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
 <div class="h-full flex flex-col p-6 overflow-
```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">

 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-teal-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">

 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">

```

```
<div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class:
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
 <a href="#q8-5" class="bloc
 <a href="#q8-6" class="bloc
 <a href="#q8-7" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-9" class:
er-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q9-1" class="bloc
 <a href="#q9-2" class="bloc
 <a href="#q9-3" class="bloc
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые многочлены)

 </summary>
 <div class="pl-6 space-y-2 text-blue-400">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-emerald-400">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
 14. Кратность корня. Коэффициент

 </summary>
 <div class="pl-6 mt-2 space-y-2 text-fuchsia-400">
```

```

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-blur

 <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
 <span class="font-bold text-eme

 <a href="#question-20" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-ros

 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m

 ▲ Ликбез: Алгоритмы выживания

 </nav>
 <script>
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail

```

```

 const corresponding
 if (correspondingLi
 const details =
 if (details &&
 details.open
 document.qu
 if (othe
 othe
 }
 }));
 }
}
});
}, {
 root: null,
 rootMargin: '-10% 0px -70%
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw,
 const modal = document.createEl
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = '
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'co
 modal.style.alignItems = 'cente
 modal.style.justifyContent = 'c

```



```

copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

```



```

 } else if (theme === 'terminal') {
 body.classList.add('bg-black');
 document.getElementById('theme-selector').style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: transparent;
 }
 body.bg-black section h1,
 body.bg-black button {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path,
 body.bg-black .text-slate-950 {
 stroke: #22c55e !important;
 fill: transparent !important;
 }
 `;
 }

 document.head.appendChild(style);
 }
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border-1px border-slate-950 p-4">
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-950 font-medium">
 <button onclick="downloadWord()" class="text-slate-950 font-medium">
 Word
 </button>

 </div>
 </div>
</div>

```

## ВЫСШАЯ АЛГЕБРА

</h1>

<p class="text-slate-400 italic text-lg">

<!-- Print Table of Contents -->

<div class="hidden print:block mt-8 text-center: always;">

<h2 class="font-bold text-xl mb-4">

<ul class="text-base space-y-2">

<li><a href="#question-1">1. Ал

<li><a href="#question-2">2. Ко

<li><a href="#question-3">3. Тр

<li><a href="#question-4">4. Ст

<li><a href="#question-5">5. Гр

<li><a href="#question-6">6. Де

<li><a href="#question-7">7. Ал

<li><a href="#question-8">8. Кл

<li><a href="#question-9">9. Ко

<li><a href="#question-10">10. ,

<li><a href="#question-11">11. <

<li><a href="#question-12">12. l

<li><a href="#question-13">13. l

<li><a href="#question-14">14. l

<li><a href="#question-15">15. <

<li><a href="#question-17">17. l

<li><a href="#question-18">18. >

<li><a href="#question-19">19. l

<li><a href="#question-20">20. l

<li><a href="#question-21">21. l

<li><a href="#question-22">22. l

</ul>

</div>

</header>

<!-- ===== WTF CONTAINER =====>

<div id="wtf-container" class="hidden">

<h2 class="text-3xl font-black text-cen

```

<a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

<!-- Row 3: НОД и НОК / Алгоритм Ев
<a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow-500"
 <div class="font-bold text-slate-700"

<a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

<!-- Empty 3rd row cell -->
<div class="hidden md:block"></div>

<!-- Row 4: Факторизация / Структур
<a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow-500"
 <div class="font-bold text-slate-700"

<a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

<a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

<!-- Row 5: Корни / Уравнения -->

```

```


 <a href="#question-18" onclick="setf
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-50
 <div class="font-bold text-slate-700"

 <a href="#question-19" onclick="setf
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emerald-500
 <div class="font-bold text-slate-700"

 <a href="#question-20" onclick="setf
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-500
 <div class="font-bold text-slate-700"

 <a href="#question-21" onclick="setf
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo-500
 <div class="font-bold text-slate-700"

 <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia-500
 <div class="font-bold text-slate-700"

 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
 <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

 <a href="#proof-algorithms" onclick="setf
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500
 <div class="font-bold text-slate-700"

 </div>

```

```

<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
 <h3 class="text-xl text-fuchsia" style="color: #ff00ff; margin: 0;">
 <p class="text-sm text-slate-400" style="color: #a0b0c0; margin: 0;">
 <ul class="list-disc pl-5" style="list-style-type: none;">

i>

 т.д.).

что так и задумано: "Ой, противоречие, теорема".

 </div>

```

```

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
 <h3 class="text-xl text-emerald" style="color: #00ff00; margin: 0;">
 <p class="text-sm text-slate-400" style="color: #a0b0c0; margin: 0;">
 <ul class="list-disc pl-5" style="list-style-type: none;">

, r_2).

счет ограничений (размер остатка) докажи, что это не так.

 </div>

```

```

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
 <h3 class="text-xl text-yellow" style="color: #ffff00; margin: 0;">
 <p class="text-sm text-slate-400" style="color: #a0b0c0; margin: 0;">
 <ul class="list-disc pl-5" style="list-style-type: none;">

валось.

Просто перепиши формулу с буквой k .

усок k , замени его на жареного карася из списка.


```

```
 <p class="text-sm text-slate-400">
 <p class="text-xs font-sans text-slate-400">
ициентов (они делятся на p), кроме старшего,
зложить.</p>
```

```
</div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400 font-sans">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-sans text-slate-400">
. Чаще всего после этого Фейс-контроль p .
```

```
</div>
```

```
</div>
```

```
 <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400 font-sans">
 <p class="text-sm text-slate-400">
 <code class="text-x"
 </div>

```

```
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400 font-sans">
 <p class="text-sm text-slate-400">
х двух, потом последних.

 <code class="text-x
```



Универсальное пособие • полный исходный код

---

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, инт
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся —
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
```

```
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75em; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
};
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
 * Размечает термины и превращает их в якорные ссылки на главы
 * Возвращает готовый HTML главы и список найденных терминов
 */
```

```
function prepareBody(html: string, opts: { selfId?: string;
 body: string;
 termIds: string[];
}) {
 const host = document.createElement("div");
 host.innerHTML = html;
```

```
 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку – выгружаем текст как есть */
 }
```

```
 /*
 * Упоминания «билет 15» становятся якорями на главу 15
 * только для книги, где все главы на месте: в файле book.html
 * вёл бы в никуда, а битые ссылки – ровно то, от чего мы хотим избежать
 */
```

```
 if (opts.ticketAnchors) {
```

```
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
 <h2>${book ? "Словарик терминов пособия" : "Словарик терминов"}
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ны сюда.</p>
 ${items}
 </section>`;
}
```

```
function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string }) {
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">
 <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">
 <div style="font-size:.72rem;color:#475569;margin-top:10px">
 <div class="export-note">
 Это автономная копия: стили внутри файла, интернет-ресурсы в файлах.
 Интерактивные тренажёры и анимации в статичный HTML-код не встраиваются.
 Они живут в онлайн-версии: онлайн-версия
 </div>
 </div>
 </h1>
 </div>
 </header>
 <div class="export-content">
 <div class="export-note">
 Это автономная копия: стили внутри файла, интернет-ресурсы в файлах.
 Интерактивные тренажёры и анимации в статичный HTML-код не встраиваются.
 Они живут в онлайн-версии: онлайн-версия
 </div>
 </div>
 <div class="export-gloss">
 <h2>${book ? "Словарик терминов пособия" : "Словарик терминов"}
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ны сюда.</p>
 ${items}
 </div>
 <div class="export-footer">
 <p>© ${stamp} ${opts.title}
 онлайн-версия
 </p>
 </div>
 </div>
</body>
</html>`;
```

```

 } catch {
 /* картинка недоступна — глава останется со ссы
 }
 })
};
return doc.body.innerHTML;
}

/**
 * Кадры визуализаций: книга встраивает снимки демонстр
 * файл показывал, что крутилось в приложении. Снимки д
 * (capture-viz-shots.mjs) и публикует в /export/viz-sh
 * без них (офлайн-сборка, локальный дев) книга остаётс
 * нормально и молча пропускается.
 */
async function vizFigures(chapterId: string): Promise<string> {
 try {
 const base = import.meta.env.BASE_URL || "/";
 const manifestRes = await fetch(`${base}export/viz-
 if (!manifestRes.ok) return "";
 const manifest = (await manifestRes.json()) as { files: string[] };
 const mine = manifest.filter((m) => m.file === `${chapterId}.mjs`);
 const figures: string[] = [];
 for (const m of mine.slice(0, 4)) {
 const res = await fetch(`${base}export/viz-shots/${m.file}`);
 if (!res.ok) continue;
 const blob = await res.blob();
 if (blob.size > 3_000_000) continue;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 figures.push(
 `<figure class="export-fig">
 `<figcaption>${m.name ? `Режим «${escapeHtml(m.name)}»` : ""}
);
 }
 } catch {
 //
 }
}

```

```
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 triggerDownload(await buildChapterHtml(chapter), safe)
}

/** Собрать книгу: всё пособие одним файлом со сквозным
export async function buildBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#ch-${c.id}">${c.title}</a></li>`)
 .join("\n");

 for (const c of chapters) {
 const { body: rawBody, termIds } = prepareBody(c);
 // картинки глав встраиваются сразу: книга одна, а не по главам
 // снимки демонстраций подтягиваются из /export/viz
 const body = (await vizFigures(c.id)) + (await inlineImages(rawBody));
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="background-color:#f9f9f9; padding: 1.5rem 1.5rem 0 1.5rem; margin-bottom: 1.5rem;">
<p style="text-align:right;font-size:.72rem;margin:0 0 1.5rem 0">Глава ${c.title}</p>
<div style="margin-top:1.5rem">
<ol style="list-style-type: none; padding-left: 0">
1. ${c.body}

</div>
</div>`);
 }

 const body = `<nav id="toc" style="background:#0f172a; color:#fff; padding: 1.5rem 1.5rem 0 1.5rem;">
<h2 style="margin:0;font-size:1.05rem;font-weight:800">Содержание</h2>
<ol style="margin:0;padding-left:1.1rem;font-size:.85rem">
1. ${chapters[0].title}

</nav>
${bodies.join("\n")}`;
```

```
* * в PDF — остаётся текст (ссылки там всё равно не
*
* Номер билета → страница берётся из `chapterTopics`,
* «билет 19» корректно ведёт на общую страницу остовных
* (она закрывает 18, 19 и 20 сразу).
*/
```

```
const ticketToChapter = new Map<number, string>();
for (const [id, tickets] of Object.entries(chapterTopic
 for (const n of tickets) if (!ticketToChapter.has(n))
}
```

```
/** id страницы по номеру билета (undefined, если такого
export const chapterForTicket = (ticket: number): string
```

```
/** «билет», «билеты», «билетов», «билете», «билетам»...
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{
```

```
/**
* Константы TreeWalker числами: глобальный `NodeFilter
* во всех окружениях (в jsdom его нет) — с числами разн
*/
```

```
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;
```

```
/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, but
```

```
/** Одна страница на диапазон? «билеты 18–20» → mst, а
function chapterForRange(from: number, to: number): str
 let found: string | undefined;
 for (let n = from; n <= to; n += 1) {
 const id = ticketToChapter.get(n);
 if (!id) return undefined;
 if (found && found !== id) return undefined;
 found = id;
 }
```

```

 })
 .join("");
}

/**
 * DOM-версия — для интерактивной страницы. Проходит то
 * (как `annotateTerms`), поэтому атрибуты и разметка не
 * Возвращает число созданных ссылок — его проверяют DOM
 */
export function annotateTicketLinks(
 root: HTMLElement,
 opts: { selfId?: string; lite?: boolean } = {}
): number {
 if (typeof document === "undefined") return 0;
 const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (!/билет/i.test(text)) return FILTER_REJECT;
 return FILTER_ACCEPT;
 },
));

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 let created = 0;
 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 TICKET_RE.lastIndex = 0;
 const frag = document.createDocumentFragment();
 let last = 0;

```

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
```



```

 });
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.joi
}

```

ФАЙЛ 91/111: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
}));

```

```

 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
);
}
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !

```

ФАЙЛ 92/111: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 687 | РАЗМ

```

/**
 * Компактный PDF пособия: public/export/guide_lite.pdf
 *
 * текст всех тем (без служебной разметки)
 * + снимки визуализаций, если они отрендерены (tmp/viz
 * capture-viz-shots.mjs в CI через headless-браузер)
 * + иллюстрации страниц (jpg из src/assets)
 * + текстовое описание каждой демонстрации и переменны
 * Python-компилятор (работает и без браузера – локал
 * – БЕЗ кода панели компилятора и без навигации прилож
 *
 * Запуск: npm run export:guide
 * Цель: 40–60 страниц (жёсткий предел – 100).
 *
 * Вёрстка целиком на собственном курсоре (cy): pdfkit
 * сам, длинные таблицы/код/боксы режутся на фрагменты
 */
import fs from "node:fs";

```

```

],
 oblique: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique",
 "/usr/share/fonts/dejavu/DejaVuSans-Oblique.ttf",
],
 mono: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
 "/usr/local/share/fonts/DejaVuSansMono.ttf",
 "C:/Windows/Fonts/consola.ttf",
],
 };

const pick = (list) => list.find((f) => fs.existsSync(f));

/**
 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает — их оставляем.
 */
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}]/gu;

/** Русское склонение счётных существительных: 1 страница */
const plural = (n, one, few, many) => {
 const mod10 = Math.abs(n) % 10;
 const mod100 = Math.abs(n) % 100;
 if (mod10 === 1 && mod100 !== 11) return one;
 if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 10)) return many;
 return many;
};

const plain = (text) =>
 String(text ?? "")
 .replace(UNSUPPORTED, "")
 .replace(/\s{2,}/g, " ")
 .replace(/\u0000/g, "")
 .trim();

```

```

 if (files.length > 0) {
 return files.map((f) => ({
 file: path.join(dir, f),
 name: f.replace(/\. (png|jpe?g)$/i, "").split(
 }));
 }
 }
}
return [];
}

```

/\* \_\_\_\_\_ отрисовка PDF \_\_\_\_\_

```

class Renderer {
 constructor(doc, fonts) {
 this.doc = doc;
 this.fonts = fonts;
 this.footerTitle = "";
 this.pages = 0;
 this.cy = TOP;
 }

 /**
 * Колонтитул текущей страницы (печатаем до перехода к
 *
 * Важно: на время печати нижнее поле страницы уменьш
 * решает, что текст не влез, и молча добавляет пусты
 */
 stampFooter() {
 if (this.pages === 0 || !this.doc.page) return;
 const prevBottom = this.doc.page.margins.bottom;
 this.doc.page.margins.bottom = 16;
 const y = PAGE.height - PAGE.margin + 4;
 this.doc.font(this.fonts.regular).fontSize(7.6).fil
 this.doc.text("Универсальное пособие • алгоритмы и
 width: CONTENT_W * 0.62,
 align: "left",
 lineBreak: false,

```

---

```

* предложений, слишком длинное предложение – по слов
*
* Без этого pdfkit сам переносит остаток на новую стр
* создаётся в обход рендерера – без нижнего колонтит
* из-за чего в файле оказывалось на одну страницу бо
*/

```

```

splitToFit(text, opts, max) {
 const measure = (t) => this.height(t, opts);
 const source = String(text);
 if (measure(source) <= max) return [source];
 const chunks = [];
 let current = "";
 const flush = () => {
 if (current.trim()) chunks.push(current.trim());
 current = "";
 };
 for (const sentence of source.split(/(?<=[.!?…;:])\s/)) {
 const withSentence = current ? `${current} ${sentence}` : sentence;
 if (measure(withSentence) <= max) {
 current = withSentence;
 continue;
 }
 flush();
 if (measure(sentence) <= max) {
 current = sentence;
 continue;
 }
 for (const word of sentence.split(/\s+/)) {
 const withWord = current ? `${current} ${word}` : word;
 if (measure(withWord) <= max) current = withWord;
 else {
 flush();
 current = word;
 }
 }
 }
 flush();
 return chunks.length ? chunks : [source];
}

```

```

/** Абзац: режим по высоте страницы сами – автоперенос
para(rawText, { size = 10.0, indent = 0, color = "#1f
 const text = plain(rawText);
 if (!text) return;
 const font = mono ? this.fonts.mono : italic && thi
 const width = CONTENT_W - indent;
 const opts = { font, size, width, lineGap };
 const chunks = this.splitToFit(text, opts, BOTTOM -
 chunks.forEach((chunk, index) => {
 const last = index === chunks.length - 1;
 const h = this.height(chunk, opts);
 this.ensure(h + (last ? gapAfter : 0), 6);
 const y0 = this.cy;
 this.doc.font(font).fontSize(size).fillColor(color)
 this.cy = Math.max(y0 + h, this.doc.y) + (last ?
 if (this.cy > BOTTOM) this.newPage();
 });
}

```

```

bullet(rawText, { ordinal = null, depth = 0, size = 9
 const text = plain(rawText);
 if (!text) return;
 const indent = 16 + depth * 14;
 const marker = ordinal ? `${ordinal}.` : "*";
 const width = CONTENT_W - indent - 4;
 const opts = { font: this.fonts.regular, size, width
 const chunks = this.splitToFit(text, opts, BOTTOM -
 chunks.forEach((chunk, index) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 2.5, 6);
 const y0 = this.cy;
 if (index === 0) {
 this.doc.font(this.fonts.regular).fontSize(size)
 .align("right", lineGap: 1 });
 }
 this.doc.font(this.fonts.regular).fontSize(size).
 this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
 });
}

```

```

/** Таблица: строки режутся по страницам, заголовок по
table(rawRows, { size = 8.5 } = {}) {
 const rows = rawRows.map((r) => r.map((c) => plain(c)))
 if (!rows.length) return;
 const cols = Math.max(...rows.map((r) => r.length))
 const widths = new Array(cols).fill(CONTENT_W / cols)
 const pad = 5;
 const header = rows[0];

 const rowHeight = (cells) => {
 let max = 0;
 for (let i = 0; i < cols; i++) {
 const h = this.height(cells[i] ?? "", { font: this.font })
 max = Math.max(max, h);
 }
 return max + pad * 2;
 };

 const drawRow = (cells, y, isHeader, zebra) => {
 const h = rowHeight(cells);
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill(
 for (let i = 0; i < cols; i++) {
 const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0)
 this.doc
 .font(isHeader ? this.fonts.bold : this.fonts.regular)
 .fontSize(size)
 .fillColor("#1f2937")
 .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] })
 }
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1)
 return h;
 };

 this.ensure(rowHeight(header) + 8, 14);
 let y = this.cy;
 rows.forEach((cells, idx) => {
 const row = [];

```

```

 } catch {
 return false;
 }
 const capH = caption ? 14 : 0;
 const scale = Math.min(maxWidth / img.width, (maxHe
 const w = img.width * scale;
 const h = img.height * scale;
 this.ensure(h + capH + 10, 20);
 const y0 = this.cy;
 const x0 = PAGE.margin + (CONTENT_W - w) / 2;
 this.doc.image(file, x0, y0, { width: w, height: h
 this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
 this.cy = y0 + h + 4;
 if (caption) {
 this.doc.font(this.fonts.regular).fontSize(8).fil
 enter" });
 this.cy = Math.max(this.cy, this.doc.y) + 6;
 }
 return true;
 }
}

```

```

/** Бокс с заголовком и строками; длинные режутся на
box(rawTitle, rawLines, { color = "#eef2ff", border =
 const title = plain(rawTitle);
 const lines = rawLines.map((l) => plain(l)).filter(
 const inner = CONTENT_W - 20;
 let rest = lines.slice();
 let first = true;
 while (rest.length || first) {
 first = false;
 const headH = this.height(title, { font: this.fon
 this.ensure(headH + 24, 16);
 const y0 = this.cy;
 // сколько строк влезет на эту страницу
 const avail = BOTTOM - y0 - headH - 16;
 const taken = [];
 let h = 0;
 while (rest.length) {

```



```

 margin: PAGE.margin,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – алгоритмы и струк
 Author: "algv0 • автоэкспорт",
 Subject: "Текст всех тем + визуализации, без кода
 CreationDate: new Date(),
 },
 });
const stream = fs.createWriteStream(GUIDE_PDF_PATH);
doc.pipe(stream);

const R = new Renderer(doc, fonts);
const today = new Date().toISOString().slice(0, 10);
const totalTopics = new Set(Object.values(chapterTopic
const vizCount = chapters.filter((c) => VIZ_REGISTRY[

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false
R.h1("Алгоритмы и структуры данных", { size: 17, colo
R.para(
 `Билеты 1-${totalTopics} • ${chapters.length} ${plu
 Count, "интерактивная демонстрация", "интерактивн
 { size: 10.5, color: "#4b5563" }
);
R.para(
 "Автоматический компактный сборник: весь текст тем
 уализации. Навигация приложения и код панели Pytho
 b Pages.",
 { size: 9.6, color: "#374151" }
);
R.para(`Дата сборки: ${today}`, { size: 9, color: "#6

const missingViz = chapters.filter((c) => !VIZ_REGISTRY
R.box("Как устроено пособие", [
 "* Страница = один или несколько билетов: текст, ан

```

```

 if (b.type === "heading") {
 if (b.level <= 2) R.h2(b.text);
 else if (b.level === 3) R.h3(b.text);
 else R.para(b.text, { size: 10.2, color: "#33415F" });
 } else if (b.type === "summary") {
 R.para("► " + b.text, { size: 9.8, color: "#373737" });
 } else if (b.type === "li") {
 R.bullet(b.text, { ordinal: b.ordinal, depth: b.depth });
 } else if (b.type === "code") {
 R.code(b.text);
 } else if (b.type === "table") {
 R.table(b.rows);
 } else if (b.type === "quote") {
 R.quote(b.text);
 } else if (b.type === "caption") {
 R.para(b.text, { size: 8.3, color: "#6b7280", italic: true });
 } else {
 R.para(b.text);
 }
 }
}

```

```

/* иллюстрация страницы */
const img = CHAPTER_IMAGES[chapter.id];
if (img && fs.existsSync(path.join(ROOT, img))) {
 R.h2("Иллюстрация");
 R.image(path.join(ROOT, img), { maxHeight: 300, color: "#373737" });
}

```

```

/* демонстрация: снимки + описание синхронизации с компилятором */
const viz = VIZ_REGISTRY[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) => k !== "code");
if (viz || syncKeys.length) {
 R.h2("Демонстрация и связь с компилятором");
 if (viz) {
 R.para(viz.title, { size: 10.2, color: "#1e293b" });
 if (viz.hint) R.para(viz.hint, { size: 9.1, color: "#373737" });
 }
}

```

```

 });
 R.para(q.explanation, { size: 8.8, color: "#4b5"
 });
 }
}

R.stampFooter();
doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

const size = fs.statSync(GUIDE_PDF_PATH).size;
console.log("[guide] PDF пособия собран");
console.log(`        тем: ${chapters.length} • снимков: ${pages}`);
console.log(`        выход: ${path.relative(ROOT, GUIDE_PDF_PATH)}`);

// Самопроверка: страниц в файле должно быть ровно столько, сколько
// рендерер. Расхождение означает, что pdftk создал файл с лишними
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.readFileSync(GUIDE_PDF_PATH))).getPages().length;
console.log(`        страниц: ${R.pages} (рендер) / ${physical} (файл)`);
if (R.pages > 100) console.warn(`        ВНИМАНИЕ: ${R.pages} страниц - это много`);
if (physical !== R.pages) {
 console.error(`        ОШИБКА: pdftk добавил ${physical} страниц`);
 process.exitCode = 1;
}
}

main().catch((err) => {
 console.error("Ошибка сборки PDF пособия:", err.message);
 process.exit(1);
});

```

---

```
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 10 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```

const bundle = await build({
 stdin: {
 contents: `export { activeVizIds } from "../../src/...",
 resolveDir: path.join(ROOT, "scripts", "export"),
 loader: "ts",
 },
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 logLevel: "warning",
});
const mod = await import("data:text/javascript;base64...");
return mod.activeVizIds;
}

fs.mkdirSync(OUT_DIR, { recursive: true });

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: ${ids.length}`);

const browser = await puppeteer.launch({
 headless: true,
 args: ["--no-sandbox", "--disable-setuid-sandbox", "--disable-dev-shm-usage"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
for (const id of ids) {
 try {
 await page.goto(`${BASE_URL}/?topic=${id}`, { waitUntil: 'load' });
 await page.waitForSelector("#chapter-viz", { timeout: 10000 });
 // демо-анимации: даём кадру собраться до осмысленного состояния
 await new Promise((r) => setTimeout(r, 1600));
 const viz = await page.$("#chapter-viz");
 if (!viz) throw new Error("нет #chapter-viz");
 } catch (e) {
 failed++;
 }
 done++;
}

```

```

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков
// Пропущенные страницы не повод ронять пайплайн: PDF д
// текстовое описание демонстрации.
process.exit(0);

```

ФАЙЛ 95/111: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ

```

/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕХ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [

```

```

 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileType
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.s
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.start
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(r
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
 }

```

/\*\* Достаём заголовки глав из данных пособия (без запус

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"(.
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 }
 }

```

```
push();
```

```
push(HR);
```

```
push("
```

ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН

```
push(HR);
```

```
let current = null;
```

```
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
```

```
 if (cat !== current) {
```

```
 current = cat;
```

```
 push();
```

```
 push(`  # ${cat}`);
```

```
 }
```

```
 push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`)
```

```
});
```

```
push();
```

```
push();
```

```
current = null;
```

```
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
```

```
 if (cat !== current) {
```

```
 current = cat;
```

```
 push(HR);
```

```
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
```

```
 push(HR);
```

```
 push();
```

```
 }
```

```
 const content = fs.readFileSync(path.join(ROOT, rel)
```

```
 const lines = content.split("\n");
```

```
 push(SUBHR);
```

```
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
```

```
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |
```

```
 push(SUBHR);
```

```
 push();
```

```
 push(content.replace(/\n+$/, ""));
```

```
 push();
```

```
 push();
```



Универсальное пособие • полный исходный код

```

/** Куда кладём финальные артефакты (эта папка отдаётся
export const OUT_DIR = path.join(ROOT, "public", "export
/** Промежуточные PNG-страницы (в .gitignore, в CI уезж
export const PAGES_DIR = path.join(ROOT, "tmp", "export

export const TXT_PATH = path.join(OUT_DIR, "full_code_a
export const PDF_PATH = path.join(OUT_DIR, "full_code_a

/** Параметры вёрстки страницы (подобраны под A4 @150dp
export const PAGE = {
 /** DPI растра. Можно переопределить: EXPORT_DENSITY=
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
 pointsize: 8,
 /** Максимум символов в строке (дальше – жёсткий пере
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапк
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
```

```
});
```

```
/** Пустые элементы (без закрывающего тега). */
```

```
const VOID = new Set(["br", "hr", "img", "input", "meta
```

```
/** Блочные теги: их содержимое собирается в отдельный
```

```
const BLOCK = new Set([
 "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
 "summary", "figcaption", "table", "tr", "td", "th", "t
```

```
const HEADING = new Set(["h1", "h2", "h3", "h4", "h5",
```

```
/**
```

```
 * Разбор HTML в дерево: { tag, attrs, children }, текст
```

```
 * Намеренно простой: страницы пособия генерируются нами
```

```
 */
```

```
export function parseHtml(html) {
```

```
 const root = { tag: "#root", attrs: {}, children: []
```

```
 const stack = [root];
```

```
 const re = /<!--[\s\S]*?-->|<\s*\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)
```

```
 let m;
```

```
 while ((m = re.exec(html)) !== null) {
```

```
 const [full, closeTag, openTag, attrsStr, selfClose
```

```
 if (full.startsWith("<!--")) continue;
```

```
 if (closeTag) {
```

```
 const tag = closeTag.toLowerCase();
```

```
 // закрываем до совпадения (прощает незакрытые <pre>
```

```
 for (let i = stack.length - 1; i >= 1; i--) {
```

```
 if (stack[i].tag === tag) {
```

```
 stack.length = i;
```

```
 break;
```

```
 }
```

```
 }
```

```
 continue;
```

```
 }
```

```

 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+);/g, (_, d) => String.fromCharCode

const clean = (s) => decodeEntities(s).replace(/\s+/g,

const isEl = (n) => n.tag !== "#text";
const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) &&

/** Есть ли у узла блочные потомки (тогда его текст «ра
const hasBlockChild = (node) => node.children.some(isBl

/** Текст инлайнового поддерева в порядке документа. */
function inlineText(node) {
 let out = "";
 for (const c of node.children) {
 if (c.tag === "#text") {
 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {
 out += inlineText(c);
 }
 }
 return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей).
function collectText(node) {
 if (node.tag === "#text") return clean(node.text);
 if (SKIP.has(node.tag)) return "";
 if (node.tag === "table") return tableRows(node).map(
 return clean(node.children.map(collectText).join(" "))
 }
}

```

```

const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c,
 });
};

const emitList = (list, depth) => {
 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, depth);
 }
};

const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) return;

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 if (node.tag === "pre") {
 const text = decodeEntities(collectRawPre(node));
 if (text.trim()) push({ type: "code", text: text });
 return;
 }

 // есть блочные дети: идём по порядку, инлайн-пробе
 if (hasBlockChild(node)) {
 let buf = "";
 const flush = () => {
 const t = clean(buf);
 if (t) push({ type: "p", text: t });
 };
 }
};

```

```
/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat
 else if (b.type === "summary") lines.push("", "▶ "
 else if (b.type === "li") lines.push(" ".repeat(b.c
 else if (b.type === "code") lines.push("", ...b.tex
 else if (b.type === "table") {
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join("
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.
 else if (b.type === "caption") lines.push(" " + b
 else lines.push(b.text, "");
 }
 return lines.join("\n").replace(/\n{3,}/g, "\n\n").tr
}
```

---

ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child process";
```

```

const im = imageMagickCmd();
const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

fs.rmSync(PAGES_DIR, { recursive: true, force: true });
ensureDir(PAGES_DIR);

const total = pages.length;
const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`)
 `стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
});

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

-----  
\* визуализациям матрицы чужих размерностей и ловит пад  
\*/

```
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
import { chapters, chapterTopics, SECTIONS, sectionOf }
import { quizzes } from "../../src/data/quizzes";
import { VIZ_REGISTRY } from "../../src/components/vizR
import { VizLiveStatus } from "../../src/components/Viz
import { PAGE_SYNC, buildInitTemplate } from "../../src
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../
```

```
type Py = { runPythonAsync: (src: string) => Promise<un
```

```
export interface TestResult {
 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}
```

```
const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;
```

```
/* -----
/* утилиты
/* -----
```

```
const sleep = (ms: number) => new Promise((r) => setTim
```

```
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
```

```

 el.dispatchEvent(new window.MouseEvent("click", { bubble: true }));
 await sleep(20);
 });
}

/** Панель компилятора – все поиски внутри неё, чтобы не искать в документе
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="Compiler Panel"]');
}

function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, " ");
}

function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("text-area");
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window.HTMLTextAreaElement.prototype, "value");
 if (!setter) throw new Error("нет нативного setter'a для HTMLTextAreaElement");
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble: true }));
 });
}

async function key(k: string, init: KeyboardEventInit = { }) {
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", init));
 await sleep(20);
 });
}

/** Ждём появления признака в DOM (текст/предикат), перепроверяем каждые 100мс
async function waitFor(pred: () => boolean, timeoutMs = 1000) {
 const deadline = Date.now() + timeoutMs;
 while (Date.now() < deadline) {
 if (pred()) return true;
 await sleep(100);
 }
 return false;
}

```



```

 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
 }

 /** Смонтировать одну визуализацию так, как это делает
 async function mountViz(vizId: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 const entry = VIZ_REGISTRY[vizId];
 if (!entry) throw new Error(`в реестре нет визуализации`);
 const Comp = entry.Component;
 root = createRoot(host);
 await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 <Comp />
 </VizChapterContext.Provider>
);
 });
 await settle(60);
 return host;
 }

 const BAD_DOM = [
 { re: /\bNaN\b/, why: "NaN в разметке" },
 { re: /\bundefined\b/, why: "undefined в разметке" },
 { re: /\[object Object\]/, why: "[object Object] в разметке" },
];

```

```

 if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length})
 }
 results.push({ group, name, ok, ms, detail, consoleError});
 const mark = ok ? "PASS" : "FAIL";
 realConsoleError === console.error;
 process.stdout.write(`${mark} [${group}] ${name}${ok}
 if (!ok) process.stdout.write(detail.split("\n").map(
 return ok;
}

/* -----
/* сценарии
/* -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
 const chapterList = onlyChapters.length ? chapters.filter(chapter =>
 const want = (group: string) => !skipGroups.has(group);
 // перехват console.error приложения (React пишет сюда)
 console.error = (...args: unknown[]) => {
 consoleErrors.push(args.map(a => (a instanceof Error ? a.message : a)));
 };

 /* — 1. Панель Python: реальный запуск на каждой главе
 for (const chapter of chapterList) {
 const group = "A. страницы";
 await test(group, `${chapter.id}: монтируется, бейджик
 await mountApp(chapter.id);
 const txt = bodyText();
 if (!txt.includes(chapter.title.slice(0, 18))) throw new Error("не
 if (!VIZ_REGISTRY[chapter.id]) return "визуализация не найдена";
 }
}

```

```

/* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полн
const floydCases: { name: string; vars: Record<string
 { name: "5x5 (меньше демо-7x7)", vars: { n: 5, k: 1
 }, [9, 6, 9, 9, 0]] }, expect: "5x5" },
 { name: "10x10 (больше демо – старый краш nodeNames
 from({ length: 10 }, (_, c) => (r === c ? 0 : (r
 { name: "3x3 с бесконечностями", vars: { n: 3, k: 0
 "3x3" },
 { name: "не квадратная 4x6", vars: { n: 4, k: 1, i:
 , 17, 18, 19, 0, 20]] }, expect: "Демо-схема граф
 { name: "словарь словарей d[0][1]", vars: { n: 4, k
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9,
 { name: "ключи-пары \"(i, j)\"", vars: { n: 3, k: 0
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)":
 { name: "плоский список – это НЕ матрица", vars: {
 { name: "80x80 (обрезка до 64)", vars: { n: 80, k:
 (_, c) => (r === c ? 0 : 1 + ((r + c) % 50))) }
 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0,
 { name: "матрица под именем D и k/i/j вне диапазона
 [10, 11, 12, 0]] }, expect: "4x4" },
];

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 await act(async () => {
 emitVizState("floyd", { line: 3, func: "<module
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!txt.includes(c.expect)) throw new Error(`в р
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return `бейдж: ${txt.match(/данные из компилятор
 });
}

```

```

 await sleep(40);
 });
 const grids = () => Array.from((host ?? document).querySelectorAll('div.grid'));
 let g0 = grids()[0];
 if (!g0) throw new Error("нет ни одной матрицы блоков");
 const firstCell = () => (g0.children[0] as HTMLElement)?.textContent;
 // «Оригинал (1x1)» = сама матрица A: значения видны в консоли
 if (firstCell() !== "1") throw new Error(`оригинал не 1x1`);
 if ((g0.children[7] as HTMLElement)?.textContent ?? "").trim() !== ""
 // шаги кликаются даже под live-k из компилятора
 await click(findButton("Шаг 2 (4x4)"), "вкладка шаг 2");
 const txt = (host?.textContent ?? "").replace(/\s+/g, " ");
 if (!/Матрица блоков 4x4/.test(txt)) throw new Error("не 4x4");
 g0 = grids()[0];
 const cls = (g0.children[0] as HTMLElement)?.className;
 // пустая активная клетка (код смотрит на (0,0), стили не заданы)
 if (!/border-dashed/.test(cls) || !/indigo/.test(cls))
 throw new Error(`пустая активная клетка не нарисована`);
 if (domDefects(host).length) throw new Error(`артефакты`);
 return "оригинал из A с первого шага, вкладка «Шаг 2»";
});

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин */
for (const [viz, vars] of [
 ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6, 5] }],
 ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w: [0, 4, 2, 6, 5] }],
]) as [string, Record<string, unknown>][] {
 await test("D. графы путей", `${viz}: вершины 0...4 в графе`);
 await mountViz(viz);
 const before = allButtons().length;
 const playEnabled = (findButton(/^Пуск|Пуск$/)) as HTMLElement;
 await act(async () => {
 emitVizState(viz, { line: 5, func: "<module>", vars });
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\s+/g, " ");
 if (!/данные из компилятора/.test(txt)) throw new Error("не данные");
}

```

```

 });

 if (!VIZ_REGISTRY[chapterId]) return `шагов ${result.steps.length}`;
 await mountViz(chapterId);
 const finalVars = result.finalValues ?? result.steps[result.steps.length - 1].finalValues;
 await act(async () => {
 emitVizState(chapterId, { line: result.steps[result.steps.length - 1].line,
 vars: Object.keys(finalVars) });
 await sleep(40);
 });
 const txt = (host?.textContent ?? "").replace(/\\s/g, " ");
 if (!/данные из компилятора/.test(txt)) throw new Error("некорректные данные");
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в визуализации: ${defects.join(", ")}`);
 return `шагов ${result.steps.length}${result.steps[result.steps.length - 1].finalValues ? " с переменными " : ""}`;
 });
}

```

/\* — 6. Поведение редактора: пустой код, ошибки, глаголы

```

const ch = "dijkstra";

```

```

await test("F. редактор", "только комментарии: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# просто комментарий\n# и ещё один\n");
 const done = await waitFor(() => bodyText().include("комментарий"));
 if (!done) throw new Error("не дождался сообщения");
 if (!textarea()) throw new Error("редактор исчез после клика");
 if (!/встроенное демо/.test(bodyText())) throw new Error("некорректные данные");
 return "редактор на месте, бейдж «встроенное демо»";
});

```

```

await test("F. редактор", "синтаксическая ошибка: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");

```

```

 if (!/встроенное демо/.test(bodyText())) throw new Error("не встроенное демо");
 if (allButtons().length < 5) throw new Error(`кнопок должно быть не менее 5`);
 return `пустой код: кнопок ${allButtons().length},
 });

```

```

await test("F. редактор", "глаз: эталон попадает в редактор");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "поиск");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
const init = ta.value;
const editorButtons = allButtons();
const ref = PAGE_SYNC[ch]?.code ?? "";
if (!ref) throw new Error("у страницы нет эталонного кода");
const done0 = await waitFor(() => RUN_DONE.some((m) => m === "done"));
if (!done0) throw new Error(`инициализация не выполнена`);

```

```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор должен его отобразить
await click(findButton("Вставить эталонный код"), "вставить");
const trace = await waitFor(() => !!findButton(/Остаток кода/));
if (!trace) throw new Error(`эталон не запустился. проверьте код`);
if (!textarea()) throw new Error("редактор исчез во время теста");
if (/Ошибка:/.test(panelText())) throw new Error(`ошибка: ${panelText()}`);
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.startsWith("#"));
if (!ta2?.value.includes(firstRefLine)) {
 throw new Error(`выполняется не тот текст, что лежит в эталоне`);
}

```

```

const mustBe = [
 { what: "пуск/автопроход", re: /Запустить код|Продолжить/ },
 { what: "копирование кода", re: /Скопировать код|Копировать/ },
 { what: "заккрыть панель", re: /Заккрыть компилятор/ },
 { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
 { what: "шаг назад", re: /Шаг назад|назад/i },
 { what: "свернуть листинг", re: /Свернуть листинг/ },
];
const btns = allButtons().join(" | ");

```

```

 await mountApp("floyd");
 await click(findButton("Боковая панель Python"), "п
 const ta2 = textarea();
 if (!ta2) throw new Error("на новой странице нет ре
 if (ta2.value.includes("MY-MARKER-14")) throw new E
 if (!ta2.value.includes("# «")) throw new Error("на
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta3 = textarea();
 if (!ta3?.value.includes("MY-MARKER-14")) throw new
 return "у каждой страницы свой код, возврат сохраня
 });

```

```

await test("F. редактор", "редактор живёт и во время
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`трасса не собралась. Па
 if (!textarea()) throw new Error("во время прогона
 if (!findButton("/Остановить прогон трассы/")) throw
 // Листинг сворачивается (освобождая место редактор
 // и редактор остаются на месте.
 const codeLines = () => panel()?.querySelectorAll("
 if (codeLines() === 0) throw new Error("в раскрытой
 await click(findButton("Свернуть листинг трассы"),
 await settle(80);
 if (!textarea()) throw new Error("после сворачивани
 if (codeLines() !== 0) throw new Error(`листинг не
 if (!findButton("В конец трассы")) throw new Error(
 await click(findButton("Показать листинг трассы"),
 await settle(80);
 if (codeLines() === 0) throw new Error("листинг не
 // Esc: прогон остановлен, трасса убрана, редактор
 await key("Escape");
 await settle(150);
 if (!textarea()) throw new Error("Esc убрал редактор
 if (findButton("/Остановить прогон трассы/")) throw n
 if (!/встроенное демо/.test(bodyText())) throw new

```

```
return `редактор: ${editorBtns} кнопок, листинг: ${
});
```

```
await test("Н. доступность", "бейдж честно различает
 await mountApp(ch);
 // 1) демо: компилятор ещё ничего не присылал
 if (!/встроенное демо/.test(bodyText())) throw new Error("демо не работает");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не пришли");
 await click(findButton("Боковая панель Python"), "показать панель");
 const listing = await waitFor(() => !!findButton(/Описание кода/));
 if (!listing) throw new Error("панель не показала трассу");
 // 2) код выполнен, отладчик на первой строке: переключить на код
 const awaiting = await waitFor(() => /переменных пока нет/);
 if (!awaiting) throw new Error("после запуска бейдж не переключился");
 const code = $(bodyText().match(/встроенное демо|данные из компилятора/));
 // 3) дошли до конца трассы — картинка обязана жить
 await click(findButton("В конец трассы"), "в конец трассы");
 const linked = await waitFor(() => /данные из компилятора/);
 if (!linked) throw new Error("в конце трассы визуализация не появилась");
 // 4) сняли связь (Esc + пустой код) — снова демо
 await key("Escape");
 await settle(80);
 const ta = textarea();
 if (!ta) throw new Error("редактор не вернулся по Escape");
 typeInto(ta, "");
 const cleared = await waitFor(() => /встроенное демо/);
 if (!cleared) throw new Error("после очистки кода бейдж не переключился");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не пришли");
 return "демо → выполнено/переменных нет → данные из компилятора";
});
```

/\* — 7. Текстовый режим для PDF/парсеров —————

/\* — 9. Ссылки между билетами, разделы содержания, бейдж

```
await test("Н. доступность", "содержание сжимается в
 await mountApp("bridges-code");
 const root = () => document.querySelector<HTMLElement>("body");
 const toggle = () => findButton(/Сжать содержание|С
```



```

await test("I. ссылки", "упоминание «билет 5» кликабе.
 await mountApp("sparse-table");
 const links = Array.from(document.querySelectorAll<
 if (links.length === 0) throw new Error("на страниц
 const toPrefix = links.find((a) => a.dataset.goto ==
 if (!toPrefix) throw new Error(`нет ссылки на билет
 if (!/билет 5/i.test(toPrefix.textContent ?? "")) th
 `);
 const href = toPrefix.getAttribute("href") ?? "";
 if (!href.includes("topic=prefix-sums-2d")) throw n
 await click(toPrefix, "ссылка на билет 5");
 await settle(120);
 const txt = bodyText();
 if (!/Двумерная матрица префиксных сумм/.test(txt))
 throw new Error(`клик по «билет 5» не открыл стра
 }
 return `ссылок на странице: ${links.length}, перехо
));

```

```

await test("I. ссылки", "содержание: разделы по поряд
 await mountApp("bridges-code");
 const sidebar = document.querySelector("#sidebar-ro
 if (!sidebar) throw new Error("нет сайдбара содержа
 const headers = Array.from(sidebar.querySelectorAll
 .map((el) => (el.textContent ?? "").trim()))
 .filter((t) => SECTIONS.some((sec) => sec.title ==
 if (headers.length !== SECTIONS.length) {
 throw new Error(`в содержании ${headers.length} p
 }
 const expected = SECTIONS.map((sec) => sec.title).j
 if (headers.join(" | ") !== expected) throw new Err
 // 11 «Мосты» и 12 «Точки сочленения» читаются пар
 const items = Array.from(sidebar.querySelectorAll("
 const iBridges = items.findIndex((t) => t.includes(
 const iArtic = items.findIndex((t) => t.includes("T
 if (iBridges < 0 || iArtic < 0) throw new Error(`в
 if (Math.abs(iArtic - iBridges) !== 1) throw new Er

```

```

 }
 return "введение = куча/очередь/бинпоиск/сортировка";
 });

```

```

await test("I. блиц", "вопросы отвечают, объясняют и
 await mountApp("sparse-table");
 const list = quizzes["sparse-table"] ?? [];
 if (list.length === 0) throw new Error("у билета 2
 const section = Array.from(document.querySelectorAll(
 (el.getAttribute("aria-label") ?? "").includes("Б
));
 if (!section) throw new Error("блиц не отрисован на
 const ask = (label: string | RegExp) =>
 Array.from(section.querySelectorAll("button")).fi
 const hay = [b.getAttribute("aria-label"), b.te
 return typeof label === "string" ? hay.includes
 });
 const shown = () => (section.textContent ?? "").rep
 if (!shown().includes(list[0].question.slice(0, 24)
 throw new Error(`первый вопрос не показан: ${list
 }
 const wrong = list[0].options.findIndex((_, i) => i
 await click(ask(list[0].options[wrong].slice(0, 20)
 await settle(80);
 if (!/Неверно/.test(shown())) throw new Error("на н
 if (!shown().includes(list[0].explanation.slice(0,
 throw new Error("после ответа нет разбора – польз
 }
 const nextBtn = ask("Следующий вопрос");
 if (!nextBtn) throw new Error("нет кнопки «Следующи
 await click(nextBtn, "дальше");
 await settle(80);
 if (!shown().includes(list[1].question.slice(0, 24)
 // верный ответ
 await click(ask(list[1].options[list[1].correctInde
 await settle(80);
 if (!/Верно/.test(shown())) throw new Error("на вер
 if (!/вопрос 2\\//.test(shown())) throw new Error(`с

```



```

 " for i in range(n - (1 << j) + 1):",
 " st[i][j] = min(st[i][j - 1], st[i + (1 << j) - 1])",
 "print(n, st[0][2], st[1][1])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("готово"));
if (!ready) throw new Error(`своя таблица не построена`);
if (!/6 1 1/.test(panelText())) throw new Error(`отсутствует таблица`);
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в дефайнах`);
return "свой массив из 6 чисел: st[0][2] = 1, st[1][1] = 1";
});

```

```

await test("К. свой код", "Косарайю: эталон сам считает", async () => {
 await mountApp("scc-kosaraju");
 await click(findButton("Боковая панель Python"), "панель Python");
 if (!textarea()) throw new Error("нет редактора");
 await click(findButton("Вставить эталонный код"), "вставить код");
 const ready = await waitFor(() => bodyText().includes("готово"));
 if (!ready) throw new Error(`эталон Косарайю не выпущен`);
 if (/Ошибка:/.test(panelText())) throw new Error(`эталон не принят`);
 const out = panelText();
 if (!/порядок выхода:/.test(out)) throw new Error(`некорректный порядок`);
 // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SCC
 if (!/компонент: 2/.test(out)) throw new Error(`в графе не 2 компонента`);
 if (!textarea()) throw new Error("редактор исчез, панель Python не появилась");
 await click(findButton("В конец трассы"), "в конец трассы");
 const linked = await waitFor(() => /данные из компилятора/);
 if (!linked) throw new Error("демонстрация не принята");
 const defects = domDefects(document.querySelector("#defects"));
 if (defects.length) throw new Error(`артефакты в дефайнах`);
 return "эталон Косарайю: order считается DFS-ом, нарисован корректно";
});

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без", async () => {
 await mountApp("lite");
 await click(findButton("lite"), "lite");
 const ready = await waitFor(() => bodyText().includes("готово"));
 if (!ready) throw new Error(`lite не выпущен`);
 return "lite-режим: текст без панели Python";
});

```

```

 const tocLinks = (html.match(/href="#ch-/g) ?? []).length;
 if (tocLinks < chapters.length) throw new Error(`ссылки на главы некорректны`);
 if (!/Словарик терминов пособия/.test(html)) throw new Error(`не словарь терминов`);
 if (!/↑ к оглавлению/.test(html)) throw new Error(`не оглавление`);
 if (html.length < 200_000) throw new Error(`книга слишком маленькая`);
 return `${(html.length / 1024).toFixed(0)} КБ, глав: ${chapters.length}`;
 });
}

console.error = realConsoleError;
return results;
}

/** Сводка для оркестратора. */
export function summary() {
 const byGroup = new Map<string, { ok: number; fail: number }>();
 for (const r of results) {
 const g = byGroup.get(r.group) ?? { ok: 0, fail: 0 };
 g[r.ok ? "ok" : "fail"]++;
 byGroup.set(r.group, g);
 }
 return { total: results.length, passed: results.filter(r => r.ok).length, failed: results.filter(r => !r.ok).length, byGroup };
}

/** Инициализация страницы из vizSync – нужна для проверки ссылок на главы
export const initTemplateOf = (id: string) => buildInitPage(id);

ФАЙЛ 100/111: scripts/verify/check-links.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 1.2 КБ

/**
 * Аудит внутренних ссылок пособия: ничего не должно вестись за пределы
 *
 * npm run verify:links
 */
```

```

platform: "browser",
target: "es2022",
jsx: "automatic",
LogLevel: "error",
define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
},
loader: { ".css": "empty", ".png": "dataurl", ".jpg":
});
const { chapters, chapterTopics, SECTIONS, sectionOf, G
 await import(`${process.cwd()}/${OUT}?run=${Date.now(

const chapterIds = new Set(chapters.map((c) => c.id));
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подск
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs",

/* 1. билеты 1-24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopic
 ok(chapterIds.has(id), `chapterTopics ссылается на не
 tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 24; n += 1) {
 ok(covered.has(n), `билет ${n} не закрыт ни одной стр
}

/* 2. разделы содержания */
for (const chapter of chapters) {
 const sec = sectionOf(chapter.id);
 ok(!sec, `страница «${chapter.id}» не попала ни в од
 const tickets = chapterTopics[chapter.id] ?? [];

```

```

 mentionsHere += 1;
 for (let n = from; n <= to; n += 1) {
 ok(n >= 1 && n <= 24 && covered.has(n), `«${chapter
 }
 }
 ticketMentions += mentionsHere;

// 4. явные переходы
for (const m of html.matchAll(/data-goto="([^\"]+)/g)) {
 explicitLinks += 1;
 ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
}
for (const m of html.matchAll(/href="\?(?:lite=1&)?top/g)) {
 explicitLinks += 1;
 ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
}

// 5. якоря
for (const m of html.matchAll(/href="#([^\"]+)/g)) {
 ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
}

if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
 if (term.simulator) {
 ok(registryIds.includes(term.simulator), `глоссарий
 ok(
 chapterIds.has(term.simulator) || EXTRA_DEMOS.has
 `глоссарий «${term.id}»: simulator "${term.simula
 ые компилятора`
);
 }
 if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

```

```

 console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
 FAILS.slice(0, 40).forEach((f) => console.log(`  x ${f}`));
 process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");

```

ФАЙЛ 101/111: scripts/verify/check-reference-code.mjs  
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 133 | РАЗМЕР: 1.1 КБ

```

/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code
 *
 * Каждый референсный Python-код из PAGE_SYNC выполняет
 * (pyodide из node_modules) под тем же трейсером, что и
 * проверяется:
 * 1. код не падает и не обрывается по лимиту шагов;
 * 2. трасса не пустая (иначе панель показала бы пустую трассу);
 * 3. каждая переменная, обещанная странице, реально существует
 * иначе визуализация не может за ним следовать;
 * 4. в снимках нет «мусорных» значений, которые уронят трассу.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
 `${OUT_DIR}/contract-entry.ts`,
 `import { buildDebugRunner } from "../../src/data/debugRunner";
import { PAGE_SYNC } from "../../src/data/vizSync";
`
);
```



```

try {
 result = JSON.parse(String(await py.runPythonAsync(
} catch (error) {
 problems++;
 rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: $
 continue;
}
const seen = new Set();
for (const step of result.steps) for (const name of 0)
for (const name of Object.keys(result.finalValues ??

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result
if (result.truncated) issues.push("трасса оборвана по
if (result.steps.length === 0) issues.push("нет ни од
const missing = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .filter((a) => !a.some((x) => ident(x) && seen.has(
 .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «зашитый» результат вместо работы алгоритма. Именн
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
 Array.isArray(v) ? v.length === 0
 : typeof v === "string" ? v.length === 0
 : typeof v === "object" && v !== null ? Object.keys
 : false;
const neverFilled = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .map((a) => a.find((x) => ident(x) && seen.has(x)))
 .filter(Boolean)
 .filter((name) => {

```

```
/**
 * Браузерное окружение для проверок: jsdom + те API, к
 * Создаётся ПОСЛЕ загрузки Pyodide (иначе pyodide решит
 */
```

```
import { JSDOM, VirtualConsole } from "jsdom";
```

```
const OBSERVER_STUB = class {
 observe() {}
 unobserve() {}
 disconnect() {}
 takeRecords() {
 return [];
 }
};
```

```
export function createDom({ url = "http://localhost/?top" }) {
 const virtualConsole = new VirtualConsole();
 const jsdomErrors = [];
 virtualConsole.on("jsdomError", (error) => {
 jsdomErrors.push(error?.message || String(error));
 onError?.(error);
 });
 // console.error/warn приложения собираем отдельно —
 const consoleErrors = [];
 virtualConsole.on("error", (...args) => consoleErrors.push(...args));
 virtualConsole.on("warn", () => {});
```

```
const dom = new JSDOM(
 `<!doctype html><html lang="ru"><head><meta charset="utf-8"><title></title></head><body><div id="root"></div></body></html>`,
 { url, pretendToBeVisual: true, runScripts: "outside-only" },
);
const { window } = dom;
```

```
window.ResizeObserver = window.ResizeObserver ?? OBSERVER_STUB;
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER_STUB;
window.matchMedia =
 window.matchMedia ??
```

```

 "KeyboardEvent",
 "InputEvent",
 "getComputedStyle",
 "requestAnimationFrame",
 "cancelAnimationFrame",
 "DOMParser",
 "SVGElement",
 "Text",
 "DocumentFragment",
 "NodeList",
 "HTMLCollection",
 "Range",
 "MutationObserver",
 "localStorage",
 "sessionStorage",
 "Blob",
 "File",
 "FileReader",
];
 for (const key of copy) {
 try {
 Object.defineProperty(globalThis, key, { value: w
 } catch {
 /* read-only в Node – не критично */
 }
 }
 globalThis.IS_REACT_ACT_ENVIRONMENT = true;

 return { dom, window, document: window.document, cons
}

```

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМ

/\*\*

```

const shape = (v) => {
 const r = m(v);
 return r ? { rows: r.rows, cols: r.cols, label: r.size };
};
const cells = (v) => {
 const r = m(v);
 if (!r) return null;
 return Array.from({ length: r.rows }, (_, i) => Array
};
```

```
/* — liveMatrix: формы данных -----
check("квадратная 3x3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
check("не квадратная 2x4", shape([[1, 2, 3, 4], [5, 6, 7, 8]]));
check("значения ячеек 2x2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]]);
check("рванные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
check("рванные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
check("at() вне диапазона → null", m([[1, 2], [3, 4]]).at(2));
check("at() с отрицательным индексом → null", m([[1, 2], [3, 4]]).at(-2));
check("список словарей с числовыми ключами", cells([{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]));
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }));
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }));
check('ключи-пары "{i, j}" после снапшота tuple', cells([["i", "j"], ["k", "l"]]));
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5 }));
```

```
/* — liveMatrix: что матрицей НЕ является -----
check("плоский список чисел → null", m([1, 2, 3, 4, 5]));
check("словарь по именам → null", m({ dist: 1, prev: 2 }));
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c", "d"]));
```

```
/* — liveMatrix: особые значения -----
check('"inf" → +Infinity', cells([["inf", 1], [2, " -inf"]]));
check('числа-строки "42" → 42', cells([["1", "2"], ["3", "4"]]));
```

```

import { createRoot } from "react-dom/client";
import App from "../../src/App";

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="panel"]');
}
function editor(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>('text-area');
}
function footer(): string {
 const divs = Array.from(panel()?.querySelectorAll("div"));
 const f = divs.find((d) => d.className.includes("text-area"));
 return (f?.textContent ?? "").replace(/\s+/g, " ");
}
function state(tag: string) {
 const p = panel();
 const ed = editor();
 const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
 const step = txt.match(/var[^\s]*\s*\d+\s*\//\s*\d+/i);
 process.stdout.write(
 `  [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет"}\n` +
 `    step ? step[0] : "-" | футер: ${footer()}\n`
);
 process.stdout.write(`    бейдж страницы: ${step ? "1" : "0"}\n`);
 /встроенное демо/.test(document.body.textContent ?? "")
}
function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window.HTMLTextAreaElement.prototype, "value");
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubbles: true }));
 });
}

```

```

 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 });
}
const ed2 = editor();
if (!ed2) throw new Error("редактор так и не появился
typeInto(ed2, "# только комментариев\n# и ещё\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}

process.stdout.write("\n=== 3. вводим рабочий код x=4
const ed3 = editor();
if (!ed3) throw new Error("редактор пропал после комм
typeInto(ed3, "x = 41\nx += 1\nprint(x)\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}

process.stdout.write("\n=== 4. очищаем редактор полно
const ed4 = editor();
if (ed4) {
 typeInto(ed4, "");
 state("очистили");
 for (const t of [200, 800, 2000]) {
 await settle(t);
 state(`t+${t}`);
 }
} else {
 process.stdout.write(" редактора нет (листинг) — ж
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 });
});

```

-----  
### `verify:code` – `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется настоящим `sys.settrace`-трейсером, что и панель компилятора. Протрасса не оборвана по лимиту шагов, шагов больше нуля (или пустой листинг), и \*\*каждая переменная, обещанная страницей кодом\*\* – иначе визуализация физически не может за ним следовать. Предупреждения интерпретатора, которые иначе пойдут в «Результат».

### `verify:dom` – `browser-checks.tsx`

Настоящее приложение монтируется в jsdom (`App` целиком в главе, визуализация, панель Python), сценарии кликают кнопки в редакторе.

| Группа | Что делает |

| --- | --- |

| А. страницы (22 × 3) | каждая глава монтируется; бейдж автозапуска инициализации настоящим Python не роняет и не возвращает `[object Object]`; нет кнопок без имени |

| В. Флойд: размерности (10) | 5×5, 10×10 (больше демо – больше список, 80×80 (обрезка), пустая матрица, чужое имя |

| С. матричные страницы (6) | префиксные суммы 2D и разности |

| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами, гаснущими и оживающими |

| Е. эталонный код (46) | прогон кода в CPython → финальный результат и показать мусор |

| F. редактор (6) | код из одних комментариев, синтаксический анализ эталон и возвращает инициализацию; код не перетекает |

| G. lite-режим (1) | `?lite=1` отдаёт текст без навигации |

| H. доступность (2) | в панели подписаны все кнопки в меню |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

## Как это заведено в приложение

\* `window.\_\_algv0PythonRuntime` – точка подмены интерпретатора в `PythonCompiler.tsx`. В браузере пользователя поле не

```

log(` Python готов за ${Date.now() - t0} мс`);

log("поднимаю jsdom...");
const { window, document, consoleErrors } = createDom({
window.__algv0PythonRuntime = py;

log("собираю приложение esbuild'ом.");
mkdirSync(OUT_DIR, { recursive: true });
await esbuild.build({
 entryPoints: ["scripts/verify/browser-checks.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 // Vite-специфичные глобалы, которых нет вне dev-серв
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/b

process.on("unhandledRejection", (reason) => {
 jsdomErrors.push(`unhandledRejection: ${reason instan
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document
const sum = mod.summary();
```



```
* Зонд панели Python: печатает состояние редактора/тра
* Не тест, а диагностика — когда нужно понять, что пан
* ввода кода (автозапуск, переход в листинг, сообщения
*
* npm run verify:probe
*/
import * as esbuild from "esbuild";
import { mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "./dom-env.mjs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
const log = (m) => process.stdout.write(`[verify:probe]

log("гружаю CPython...");
const py = await loadPyodide({ indexURL: "node_modules/
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
```

```
id-token: write
```

```
concurrency:
```

```
 group: pages
```

```
 cancel-in-progress: true
```

```
jobs:
```

```
 build:
```

```
 name: Build static site
```

```
 runs-on: ubuntu-latest
```

```
 steps:
```

```
 - name: Checkout
```

```
 uses: actions/checkout@v6
```

```
 - name: Setup Node.js
```

```
 uses: actions/setup-node@v7
```

```
 with:
```

```
 node-version: 24
```

```
 cache: npm
```

```
 - name: Install dependencies
```

```
 run: npm ci --no-audit --no-fund
```

```
 - name: Build for the repository subpath
```

```
 env:
```

```
 VITE_BASE: /algv0/
```

```
 run: npm run build
```

```
 - name: Make GitHub Pages fallback
```

```
 run: cp dist/index.html dist/404.html
```

```
 - name: Upload Pages artifact
```

```
 uses: actions/upload-pages-artifact@v5
```

```
 with:
```

```
 path: ./dist
```

```
 deploy:
```

```
 name: Publish to GitHub Pages
```

Универсальное пособие • полный исходный код

```

↳ full_code_all_page
#
Готовые TXT и PDF коммитятся обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолч
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[
 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
```

- ```
# headless-браузер для снимков: chrome ставит  
# этот вызов — страховка, если кэш оказался п  
npx puppeteer browsers install chrome || true  
npm run build  
mkdir -p tmp  
nohup npx vite preview --host 127.0.0.1 --por  
for i in $(seq 1 40); do  
    if curl -sf http://127.0.0.1:4173/ > /dev/n  
    sleep 1  
done
```
- name: Step 5 — снимки визуализаций (headless-бр
 run: npm run export:shots
 continue-on-error: true
 - name: Step 6 — PDF-конспект со снимками
 run: |
 npm run export:guide
 # снимки публикуются вместе с приложением: HT
 # в автономный файл прямо из /export/viz-shot
 mkdir -p public/export/viz-shots
 cp tmp/viz-shots/*.png tmp/viz-shots/manifest
 - name: Type-check приложения
 run: npx tsc --noEmit
 continue-on-error: true
 - name: Summary
 run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full_code_all_pages
 echo "| \`public/export/full_code_all_pages
 echo "| \`public/export/guide\_lite.pdf\` | :
 echo "| Снимков визуализаций | \$(ls tmp/viz

Универсальное пособие • полный исходный код

```
name: Verify
```

```
# Проверки приложения на каждом пуше и PR:
```

```
#
```

```
#   verify:unit   – разбор «живых» данных компилятора (L
```

```
#   verify:code   – все эталонные Python-коды страниц вы
```

```
#                   под трейсером панели и создают обеща
```

```
#   verify:dom     – приложение монтируется в jsdom: стра
```

```
#                   матрицы чужих размерностей, кнопки, 
```

```
#
```

```
# Headless-браузер не нужен: jsdom + pyodide из node_mo
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - "**"
```

```
    paths-ignore:
```

```
      - "public/export/**"
```

```
      - "**/*.md"
```

```
  pull_request:
```

```
  workflow_dispatch:
```

```
concurrency:
```

```
  group: verify-${{ github.ref }}
```

```
  cancel-in-progress: true
```

```
permissions:
```

```
  contents: read
```

```
jobs:
```

```
  verify:
```

```
    name: unit + эталонный код + DOM
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

ФАЙЛ 111/111: .ai_skills/universal_guide_authoring/SKIL
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для ****создания и правки контента** репозитории. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем ****только**** из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо.

1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
 {
 id: "dijkstra", // = ключ VIZ_REGISTRY и
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html", // всегда "html": страни
 category: "Графы. Пути", // группа в навигации
 content: `...HTML ниже...`,
 },
];
```
```

Правила:

- `id` – kebab-case, уникальный, совпадает с ключом `VI

```
-----  
| `border-amber-500` + `text-amber-400` | ловушки, част  
| `border-rose-500` + `text-rose-400` | ограничения, «к
```

3. Библиотека блоков (точные классы)

3.1 Определение / строгое правило – всегда по центру

```
```html  
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-center">
 <p class="text-lg text-emerald-300 italic mb-2">Строчка
 <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

3.2 Обычный абзац

```
```html  
<p class="text-slate-300 text-sm mb-4">
 Текст с акцентом и инлайн-кодом
 dist[v]
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html  

```
```

3.3 Сетка ключевых чисел (3 ячейки)

```
```html  
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6">
 <div class="bg-slate-900 border border-slate-700 rounded">
 <p class="text-[10px] uppercase tracking-wider text-white">
 <p class="font-mono text-emerald-300 text-sm">0{(V+H
```

```

<table class="w-full text-xs text-left border-collapse"
 <thead>
 <tr class="text-slate-500 border-b border-slate-700">
 <th class="py-1.5 pr-2 font-bold">Колонка</th>
 <th class="py-1.5 pr-2 font-bold">Колонка</th>
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-800 align-top">
 <td class="py-2 pr-3 font-bold">Значение</td>
 <td class="py-2 pr-3 font-mono">O(log n)</td>
 </tr>
 </tbody>
</table>
</div>
```
```

3.6 Спойлер ``<details>`` – ****обязателен**** для кода,

```
```html
<details class="bg-slate-900/40 rounded-lg border border-slate-800"
 <summary class="p-4 font-bold text-slate-400 outline-slate-500">
 ▶ Код: Дейкстра на куче (Python)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto">...</code></pre>
 <p class="text-slate-300 text-sm">Комментарий после
 </div>
</details>
```
```

- Вложенные списки внутри спойлера: ``<ul class="list-disc">`
- Подзаголовок внутри блока: ``<h4 class="text-lg font-bold">` или ``<p class="text-lg font-bold text-white mt-8 mb-3">`

3.7 Ловушки / частые ошибки

- Поля записи: ``vizTitle``, ``stepNote`` (что считается оди...
``variables: [{ name, role, range }]``, ``code`` (референс...)
- Переменные, которые читает визуализация, должны быть...
(``int``, ``list``, ``dict``, ``str``): экземпляры классов сн...
до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – ``buildInit``
инициализация переменных до строки ``# --- тело алгоритма``

Проверка перед коммитом: ``npm run dev`` → страница открыта
подставляет код, шаг демонстрации подсвечивает нужные п...

6. Экспорт в PDF (``npm run export:guide``)

Текст страниц парсится ``scripts/export/html-to-blocks.m``
``public/export/guide_lite.pdf`` (текст + визуализации, б...
компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только ``h2/h3/h4``; списки – ``ul/ol/li``; т...
код – ``pre``; определения – обычный ``r`` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF использ...
нет эмодзи-глифов – пиктограммы вырезаются, остаётся
« Операции на отрезках» доедет как «Операции на отр...
а вот « значит да / значит нет» потеряет смысл, пи...
- спойлеры в PDF раскрываются полностью – не прячьте ту...
минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из ``CHAPTER_IMAGES`` (``script``...
снимки демонстраций – из ``tmp/viz-shots/*.png`` (их ре...
``npm run export:shots`` в CI); локально без браузера в...
описание вкладок и переменных из ``PAGE_SYNC``.

Цель по объёму: 40–60 страниц, жёсткий предел – 100.